

Universidad Nacional de Rosario
Facultad de Ciencias Exactas, Ingeniería y Agrimensura
Escuela de Ingeniería Electrónica
Proyecto II

Control de Acceso para institución educativa

Año: 2026

Autor: Ramonda, José

Legajo: R-4423/7

Director:

Prof. Ing. Daniel Marquez

Rosario, 31 de agosto de 2026

1. Resumen

En el presente documento se desarrolla el proyecto de finalización de carrera “Control de acceso para institución educativa”, el cual implementa un sistema de gestión y registro de ingresos. El sistema permite el acceso regular del personal autorizado mediante credenciales basadas en tecnología NFC (*Near Field Communication*) y la validación remota de visitantes externos por parte de un operador humano, utilizando el reconocimiento visual a partir de fotografías recibidas en un dispositivo móvil.

Índice

1. Resumen	2
2. Problemática	5
3. Funcionamiento del sistema	6
3.1. Arquitectura del sistema	7
3.2. Actores del sistema	8
3.3. Interfaces de usuario	9
3.4. Requerimientos del sistema	9
4. Alcance del proyecto	9
5. Terminales	10
5.1. Selección de hardware	10
5.1.1. Microcontrolador	10
5.1.2. Cámara	11
5.1.3. Comunicación RS-485	11
5.1.4. Lector NFC	11
5.1.5. Interfaces de campo	11
5.1.6. Alimentación	11
5.1.7. Diseño de circuito impreso	12
5.2. Firmware del controlador	13
5.2.1. Sistemas operativos en tiempo real	13
5.2.2. Comunicación con el servidor	15
5.2.3. Protocolo Ramodbus	17
5.2.4. Tareas reactivas al protocolo	22
5.2.5. Manejo de entradas y salidas digitales	23
5.2.6. Módulo de Identificación NFC	23
5.2.7. Comunicación con la ESP32-CAM	26
5.2.8. Comando de la conectividad WiFi	27
5.2.9. Comando de la cámara	27
5.2.10. Prioridades de las tareas	28
5.3. Firmware del ESP32-CAM	28
5.3.1. Comunicación UART	29
5.3.2. Conexión WiFi	29
5.3.3. Cámara	30
6. Servidor	32
6.1. Programa principal	32
6.1.1. Módulo de interfaz serial	33
6.1.2. Módulo de control	34
6.1.3. Módulo MQTT	34
6.1.4. Módulo de gestión de red	34
6.1.5. Módulo de validación de accesos por tags NFC	35
6.2. Instancia de Node-RED	36
6.3. Interfaz de Usuario	38
6.3.1. Arquitectura y Máquina de Estados	38

6.3.2.	Módulo de Autenticación y Roles	39
6.3.3.	Navegación y Menús Interactivos	39
6.3.4.	Integración MQTT y Notificaciones Asíncronas	40
6.3.5.	Gestión Dinámica del Padrón de Accesos	41
6.4.	Selección del hardware del servidor y migración a Linux embebido . .	42
7.	Seguridad	43
7.1.	Posibles ataques y vulnerabilidades	43
7.2.	Sistema anticlonación	44
7.3.	Protección de los datos del sistema	45
7.3.1.	Acceso físico a los terminales	45
7.3.2.	Red local de la institución	46
7.3.3.	Bus serial RS-485	46
7.3.4.	Acceso al servidor	47
7.3.5.	Interfaz de usuario	47
7.4.	Interfaz serial inalámbrica: consideraciones extra	48
8.	Ensayos	49
8.1.	Pruebas de funcionalidad y robustez	49
8.2.	Ensayos de estrés del canal RS-485	50
8.3.	Ensayo de tiempo de respuesta	50
8.3.1.	Medición de tiempo de apertura por acceso autorizado	50
8.3.2.	Medición de tiempo de fotografías bajo demanda	51
8.4.	Definición de las condiciones de operación del sistema	52
8.5.	Desempeño del servidor embebido	53
9.	Conclusiones	54
10.	Trabajo futuro	55
11.	Referencias	56
12.	Anexos	57
12.1.	Anexo 1: Diseño de PCB y Esquemáticos	57
12.2.	Anexo 2: Trazas de ejecución del sistema (Logs)	61
12.2.1.	Ensayo a 460800 baudios sin intervalo entre encuestas	61
12.2.2.	Ensayo a 115200 baudios sin intervalo entre encuestas	64
12.2.3.	Ensayo a 115200 baudios con retardo entre encuestas de 10 ms	67
12.3.	Anexo 3: Hoja de datos NTAG21x - Fragmento	71

2. Problemática

La problemática fue planteada por la dirección de la E.E.S.O. N.º 214 “Mariano Moreno”, ubicada en Santa Isabel, provincia de Santa Fe. Asumiendo el rol de cliente, la institución expuso la siguiente situación respecto al control de acceso y la seguridad: las puertas del establecimiento no pueden permanecer abiertas de forma ininterrumpida; sin embargo, resulta inviable destinar personal exclusivo para habilitar el ingreso de individuos autorizados o visitantes externos (como tutores legales) fuera de los horarios de entrada masiva. Frecuentemente, las oficinas de dirección se encuentran vacías y el personal asistente se halla realizando tareas de mantenimiento en zonas donde no es posible percibir el timbre sonoro. A esto se suma la ineficiencia que supone el desplazamiento físico con llaves mecánicas hacia cualquiera de los múltiples puntos de acceso cada vez que se requiere habilitar una entrada.

Para dar respuesta a estas necesidades, se requiere el diseño de un sistema que:

- Permita el acceso autónomo del personal de la escuela (docente y no docente), guardando los registros correspondientes para auditoría.
- Permita que un operador autorizado habilite el acceso a un visitante externo de forma remota desde un dispositivo móvil, validando la identidad del individuo mediante una captura fotográfica.

Esta modalidad también permite gestionar los ingresos durante el horario de clases, momento en el cual las puertas deben permanecer bloqueadas, a diferencia de los horarios de entrada y salida regular. Por fuera del horario de actividades escolares, el edificio se asegura convencionalmente mediante cerraduras mecánicas.

Un condicionante crítico para el diseño es el factor económico, dado que los costos de implementación deben ajustarse estrictamente a las limitaciones presupuestarias de una escuela pública.

Respecto a las características arquitectónicas, el edificio ocupa una manzana completa y el sistema debe gestionar tres puntos físicos de acceso: la entrada de alumnos (patio principal), la entrada de docentes y el portón del biciclero, el cual comunica con el patio exterior destinado a las clases de educación física. Las distancias de cableado desde cada una de estas puertas hasta la oficina designada para alojar el servidor central son de 7,5 m, 3 m y 40 m, respectivamente.



Figura 1: Plano de la institución resaltando los accesos y ubicación del servidor

3. Funcionamiento del sistema

El sistema funciona en un horario determinado correspondiente al horario escolar. Por fuera de ese horario el sistema permanece inactivo y las puertas cerradas con llave. Durante los horarios de ingreso, las puertas permanecen abiertas para evitar “cuellos de botella” por parte de los alumnos, pero una vez iniciado el periodo de clases, las puertas permanecen cerradas. Del lado de afuera se debe instalar un manijón que no permita que se accione el resbalón, ya que este lo acciona un destrabador eléctrico comandado por el sistema. Del lado interior se coloca un picaporte, que permita el egreso en todo momento por motivos de seguridad ante una posible evacuación. En casos de fallas del suministro energético, cuando el destrabador no pueda accionarse, basta con accionar el resbalón desde dentro con el picaporte o desde fuera con la llave.

En caso de que un visitante externo, por ejemplo el tutor de un alumno, visite la escuela, al tocar el pulsador del timbre de alguna de las puertas se le toma una fotografía que llega al dispositivo móvil de la persona a cargo, quien decide si comandar o no una apertura. El timbre sonoro debe seguir funcionando normalmente.

El personal de la institución contará con credenciales de acceso en formato de *tags* NFC, con los que podrán acceder sin mediación humana. El sistema deberá llevar registro de cada acceso por cualquiera de los dos medios mencionados.

3.1. Arquitectura del sistema

El sistema se compone de un nodo servidor implementado en una *single board computer* (SBC) comunicado con tres nodos llamados terminales, construidos en base a microcontroladores, ubicados físicamente en cada punto de acceso. El servidor gestiona las comunicaciones, los registros, los permisos de acceso y por supuesto, comanda a los terminales. Los terminales son los que comandan, bajo demanda del servidor, el desbloqueo de las puertas mediante sendos relés que accionan los destrabadores eléctricos. También controlan las cámaras que toman las fotografías del exterior y los sensores NFC para transmitir la información de estos al servidor.

Existen dos canales de comunicación entre nodos:

- Comunicación serial cableada mediante protocolo RS-485 para el flujo de comandos, configuraciones y eventos, ya que es un estandar robusto utilizado en la industria, de velocidades considerables y pensado para grandes distancias.
- Conexión a la red local, en el caso de los terminales via WiFi, mientras que el servidor puede conectarse via wifi o mediante cable de red si fuera posible, para transmitir las fotografías tomadas, dado que su transmisión no es crítica en tiempo y que una fotografía se corresponde a un volumen de datos mayor que puede congestionar el canal principal.

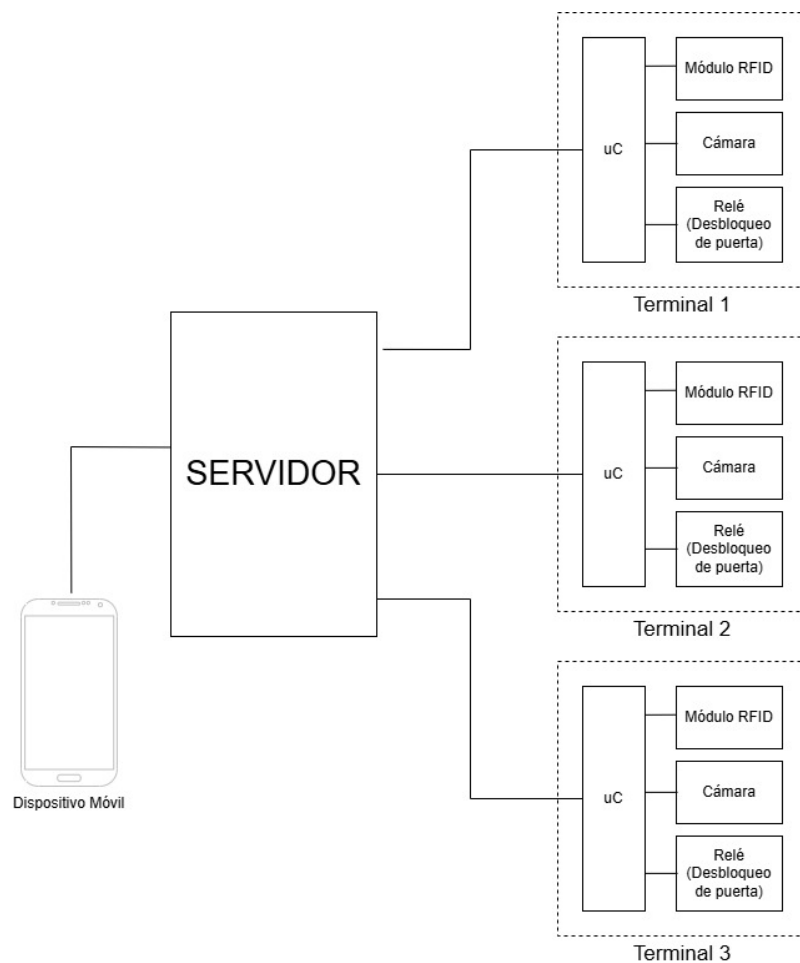


Figura 2: Diagrama de bloques del sistema

3.2. Actores del sistema

El sistema reconoce tres tipos de actores humanos.

Los **accesos autorizados** son aquellos que cuenten con una credencial o “tag” nfc autorizado y puedan acceder al presentarlo frente al lector. El lector debe validar la identidad del tag mediante contraseña, recolectar la identificación y contador de accesos y enviarla al servidor, para que este contraste con la lista de tags permitidos y responda con la orden de apertura o de no apertura. El Funcionamiento de la lectura y validación se detalla en la sección 5.2.6. El sistema debe tener capacidad para al menos 50 accesos autorizados, ya que representan al personal de la escuela (docente y no-docente) compuesto de alrededor de 30 personas. Se deja capacidad de reserva para cambios, incorporaciones y repuestos.

Los **visitantes externos** son aquellas personas que no tienen autorización de ingresar fuera del horario de apertura. Estos, al pulsar el boton del timbre, son fotografiados por la cámara del terminal. La fotografía es enviada hacia el servidor, el cual lo envía hacia el dispositivo móvil donde un usuario final valida la identidad del visitante y concede o no el acceso.

Los **usuarios finales** son aquellos con acceso a la interfaz de usuario de los dispositivos móviles. Para acceder a la interfaz estos deben iniciar sesión con credenciales (usuario y contraseña). Según las credenciales pueden tener distintos roles. El más básico es el de usuario, cuya sesión permanece abierta en el dispositivo a menos que se cierre explícitamente o que se reabra en otro dispositivo. Este rol permite solo acciones básicas tales como comandar aperturas o solicitar fotografías. Este rol está dedicado al personal no docente, directivo o administrativo. El sistema admite hasta 6 usuarios con este rol. Ciertos usuarios pueden, mediante credenciales extra, abrir una sesión temporal de administrador, este rol permite configurar credenciales y añadir autorizaciones de accesos. Estas credenciales pertenecen solo a los directivos. Finalmente, queda un rol de desarrollador al cual se accede con otras credenciales mediante un comando no disponible en el menú. Este rol no está pensado para el personal de la institución sino a quienes esten a cargo de la instalación y el mantenimiento del sistema, y permite realizar acciones simples sobre este sin la necesidad de acceder al servidor mediante SSH.

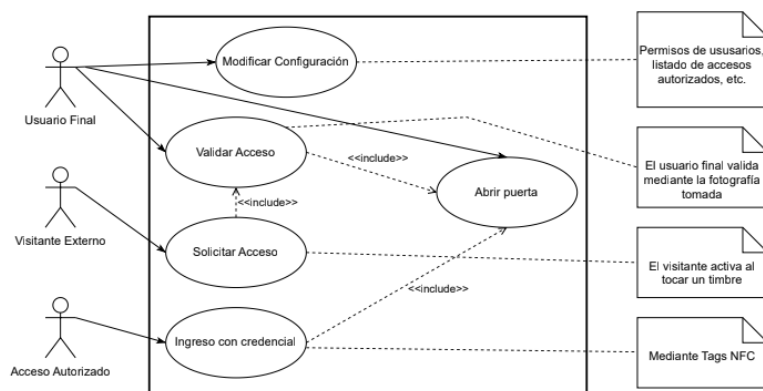


Figura 3: Diagrama UML de casos de uso.

3.3. Interfaces de usuario

La interfaz principal para los usuarios finales funciona a través de un bot de la aplicación de mensajería instantánea “Telegram”, ya que la API es amigable con los desarrolladores y permite utilizar menús y comandos de forma sencilla, además de que soluciona la conectividad con el exterior, sin tener que configurar manualmente puertos ni firewalls. Otra ventaja respecto a desarrollar una aplicación nativa es que no hay que hacer un desarrollo distinto para cada sistema operativo móvil.

El bot es en esencia un programa que se ejecuta permanentemente en el servidor y mediante llamadas a API recibe, procesa y envía mensajes al chat del bot.

Los registros históricos de accesos se dan en formatos legibles y se cargan diariamente a google drive mediante su API. Esto permite un fácil acceso a los datos y un ahorro de memoria del servidor al almacenar en la nube.

3.4. Requerimientos del sistema

Los requisitos temporales del sistema no son estrictos al punto de requerir una respuesta en tiempo real; sin embargo, el uso de este debe percibirse fluido para los actores.

Se establece entonces que el tiempo de respuesta para los accesos autorizados debe estar en el orden de los dos segundos, contando desde que el tag se presenta ante el sensor hasta que se acciona el relé que destraba la puerta.

En el caso del acceso de visitantes externos, el tiempo de respuesta se enmascara con la demora de la reacción humana, por lo que se establece una respuesta del orden de los cinco segundos del sistema para enviar la foto al usuario final, y del mismo orden para la apertura desde el comando manual, no pudiéndose controlar el tiempo de respuesta del usuario.

Otro requerimiento es la robustez frente a fallas del suministro energético y la conexión a internet. Para eso se propone conectar el servidor a una unidad UPS para mantenerlo activo ante cortes de energía, aunque sin funcionamiento. El sistema se habilita solo en un horario determinado, en caso de un desfazaje del reloj del servidor, por ejemplo por un reinicio sin conexión a internet para ponerse en hora, se puede mediante un tag especial forzar el funcionamiento del sistema fuera de horario, al menos hasta que se retome la conectividad y el sistema vuelva a funcionar con normalidad.

El último requerimiento crítico es el de seguridad del sistema, el cual se abarca en la sección 7.

4. Alcance del proyecto

El presente proyecto tiene como alcance el desarrollo de un prototipo funcional adaptable al entorno real descrito en la problemática, el cual contempla:

- Construcción y programación de al menos un terminal, incluyendo el microcontrolador, la cámara, el módulo RFID y las comunicaciones con el servidor.
- Programación del servidor central, abarcando la administración de la comunicación multipunto, el procesamiento de datos de entrada, la historización de eventos relevantes y la integración con dispositivos móviles externos.

- Implementación de los canales y protocolos de comunicación bidireccional entre los distintos nodos del sistema.
- Configuración y programación de credenciales físicas (*tags* NFC) para la validación y prueba integral del sistema.

5. Terminales

Los terminales son los subsistemas expuestos al campo, cada uno representa un punto de acceso. A diferencia del servidor, que es único, el sistema se compone de varios terminales en cada punto de acceso, en este caso tres, pero en una aplicación similar el número de terminales podría extenderse. Por este motivo, los terminales fueron diseñados para ser fácilmente replicables, adaptables y de bajo costo, dado que el costo total del sistema aumenta marginalmente con cada nueva unidad incorporada.

5.1. Selección de hardware

Un terminal se compone de un microcontrolador, diversos módulos electrónicos y dispositivos de interfaz con el entorno físico. Entre ellos se encuentran:

- Una cámara
- Un sensor NFC
- Un transceptor RS-485
- Una antena WiFi
- Dos relés, uno para permitir el ingreso destrabando la puerta y otro para hacer sonar el timbre
- Un interruptor, para que los visitantes externos soliciten el acceso

La primera opción para utilizar como controlador principal del proyecto fue el kit de desarrollo ESP32-CAM, basado en el módulo WROOM-32, el cual integra un microcontrolador ESP32 junto con una antena de radio e interfaces para conectividades WiFi, Bluetooth, entre otras, con la particularidad de que incluye un zócalo (o *socket*) para conectar una cámara, por ejemplo la OV2640, la cual se vende junto con el kit. El mayor problema del módulo es la limitada cantidad de pines GPIO disponibles y utilizables, haciéndolo poco viable para ser el controlador principal del terminal, a pesar de ser computacionalmente capaz de cumplir con las funciones requeridas.

Se optó por una arquitectura distribuida, en la que el ESP32-CAM queda dedicado exclusivamente a la adquisición y transmisión de imágenes, mientras que el procesamiento principal y el control de los periféricos recaen sobre un segundo ESP32.

5.1.1. Microcontrolador

El ESP32 [1] fue seleccionado por ofrecer un adecuado equilibrio entre capacidad de procesamiento, memoria, periféricos integrados y costo, además de incorporar múltiples interfaces de comunicación (UART, SPI, I²C, WiFi y Bluetooth)[2]. Otra ventaja es su procesador de doble núcleo que permite simultaneidad real. Para el prototipo se utilizará un kit de desarrollo ESP32 DEV KIT V1, pudiéndose

reemplazar en producción por el microcontrolador suelto, ya que esta etapa del sistema no requiere conectividad WiFi.

5.1.2. Cámara

Para el prototipo se optó, como ya se mencionó, al utilizar el kit de desarrollo ESP32-CAM, el cual se vende en conjunto con la cámara OV2640. Esta elección se basa en el fácil acceso y el bajo precio del módulo, el cual resuelve varios aspectos de la toma y transmisión de fotografías. La cámara cuenta con una resolución de 2 MP, suficiente para un reconocimiento de un rostro por parte de un validador humano. El módulo además incorpora un LED de alta luminosidad que funciona como flash de la cámara, y una memoria PSRAM externa de 4 MB que permite almacenar temporalmente las imágenes. La conectividad WiFi incorporada se utiliza para transmitir las imágenes al servidor mediante el protocolo HTTP.

Se comunica con el microcontrolador principal mediante UART para recibir comandos y compartir información.

5.1.3. Comunicación RS-485

Para la comunicación con el servidor, el ESP32 debe comunicarse con un transceptor que “traduzca” UART a 485. Se seleccionó un módulo basado en el circuito integrado MAX485, ampliamente utilizado para la conversión entre UART y RS-485.

5.1.4. Lector NFC

Para la lectura y escritura de las credenciales se seleccionó un módulo basado en el circuito integrado PN532 de NXP. Este dispositivo implementa internamente los protocolos necesarios para la comunicación con etiquetas RFID/NFC de alta frecuencia compatibles, proporcionando una interfaz de programación de alto nivel que simplifica el desarrollo del firmware. Asimismo, ofrece soporte para operaciones de lectura, escritura y autenticación sobre las etiquetas empleadas en el sistema.

Para este desarrollo se optó por las etiquetas NTAG-21X [3] como credenciales de acceso.

El módulo cuenta con interfaz SPI para comunicarse con el controlador principal.

5.1.5. Interfaces de campo

Para la activación del timbre y el desbloqueo de la puerta se incorporan sendos relés en cada terminal, comandados por salidas digitales del controlador. Ambos equipos operan con tensión de red de 220 V CA. Para el pulsador del timbre, se conectó entre GND y la entrada digital del ESP32 una bornera, que permita aprovechar el pulsador ya existente en la instalación.

5.1.6. Alimentación

Los ESP32 operan a 3,3 V, pero pueden alimentarse con 5 V, ya que las placas cuentan con reguladores de voltaje LDO incorporadas. El PN532 también cuenta con un regulador y tanto el transceptor como los relés se deben alimentar con 5 V. Se propone entonces alimentar mediante un conector micro USB tipo A (u opcionalmente un tipo C), para poder alimentar el circuito desde una fuente externa, como por

ejemplo un cargador de teléfono celular. Se añade además un capacitor de $470\mu\text{F}$ para mitigar los picos de consumo de corriente que se producen durante el encendido de la cámara o de la interfaz WiFi.

5.1.7. Diseño de circuito impreso

Para simplificar el conexionado y dotar de robustez mecánica al prototipo, se diseñó mediante el entorno KiCad el esquemático del circuito y su correspondiente placa de circuito impreso o PCB (*Printed Circuit Board*). El esquemático base puede observarse en el Anexo 12.1. En este se incluye, para cada microcontrolador, un par de pines asociados a **GND** y al terminal **TX** de la interfaz **UART0**, permitiendo conectar un adaptador serie-USB para monitorizar la consola de depuración y analizar los registros (*logs*) del sistema.

Dado que la placa se fabricó a simple faz (una sola capa de cobre) y el ruteo no permitía resolver todos los cruces de pistas en el plano, se generó una variante del esquemático documentada en el mismo anexo, donde se incorporan terminales y resistencias de 0Ω que actúan como puentes (*jumpers*) de interconexión.

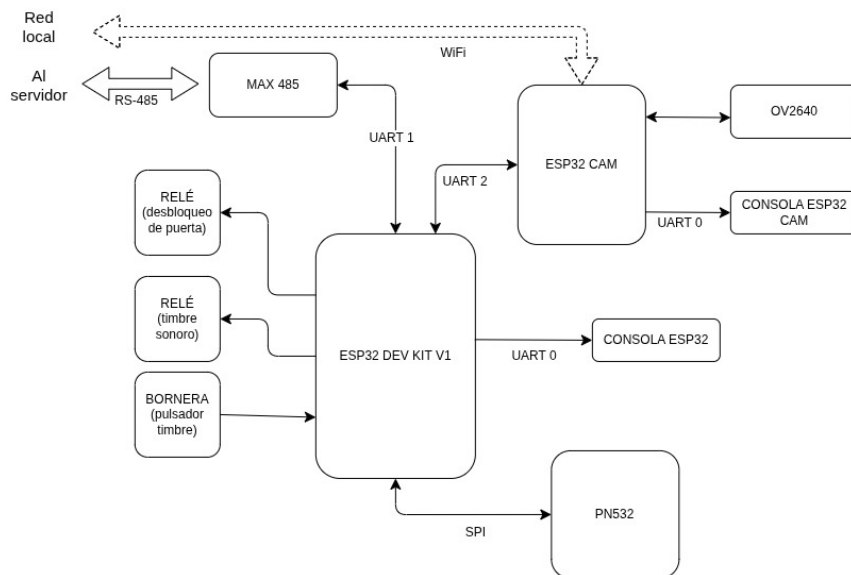


Figura 4: Diagrama de bloques de interfaces de cada terminal.

La PCB fue fabricada con el router CNC disponible en el espacio maker de la FCEIA.

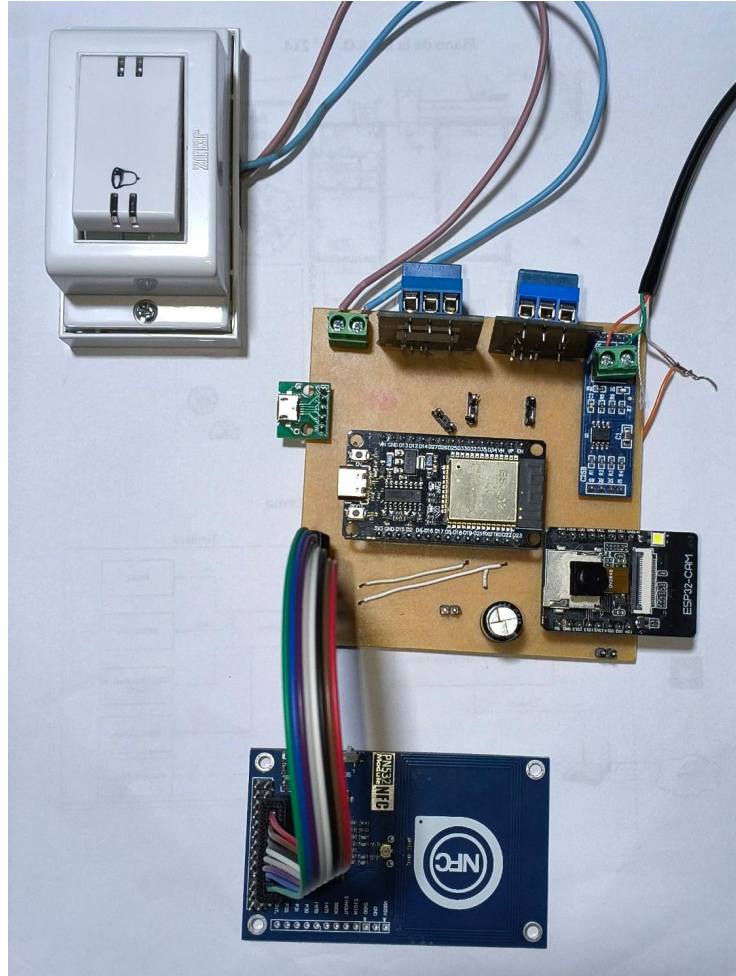


Figura 5: Montaje en PCB del prototipo.

5.2. Firmware del controlador

Para programar el firmware del controlador, se utilizó el framework oficial de Espressif, ESP-IDF [4] [5], a través de la extensión del entorno Visual Studio Code. Dado que el controlador debe ejecutar de forma concurrente tareas de comunicación, adquisición de datos, procesamiento y control de dispositivos externos, se optó por desarrollar el firmware utilizando el sistema operativo de tiempo real FreeRTOS, el cual se incorpora en forma nativa en el SDK de ESP32, por lo que la integración es orgánica.

5.2.1. Sistemas operativos en tiempo real

Una aplicación bare-metal suele organizarse alrededor de un lazo principal (*superloop*) que ejecuta secuencialmente todas las funciones del sistema. En cambio, un RTOS permite dividir la aplicación en múltiples tareas concurrentes, administradas por un planificador que decide cuál debe ejecutarse en cada instante. En FreeRTOS [6][7] [8] una tarea corresponde a una función independiente, normalmente implementada como un bucle infinito, aunque puede finalizar cuando su ejecución concluye. La ejecución de las tareas se alternan orquestadas por el *scheduler* o planificador.

Esta arquitectura permite modularizar el código, asignar prioridades y encapsular las responsabilidades.

Otra funcionalidad de los RTOS es, como su nombre lo indica, la posibilidad de asegurar la ejecución de ciertos ciclos con limitaciones de tiempo real, aunque en esta aplicación los requisitos temporales no son estrictos.

Las tareas pueden encontrarse en cuatro distintos estados:

- **Running** o corriendo, la tarea se está ejecutando.
- **Ready** o lista, la tarea está lista para ejecutarse, pero la CPU está tomada por otra tarea de mayor o igual prioridad.
- **Suspended** o suspendida, la tarea se encuentra “dormida”, es decir temporalmente desactivada, solo puede ser reactivada por otra tarea.
- **Blocked** o bloqueada, la tarea está esperando a un evento para estar lista.

Solo las tareas en estado Running consumen ciclos de CPU, por lo que se dice que los demás estados componen un “superestado” llamado *Not Running*.

Para definir cuál tarea de todas las que están listas se ejecuta hay dos formas de trabajar, la operación colaborativa, donde cada tarea cede el procesador en determinado momento, y la operación apropiativa, donde el scheduler interrumpe una tarea para darle CPU a otra. FreeRTOS para ESP-IDF utiliza planificación apropiativa basada en prioridades.

La apropiación puede ser para ejecutar una tarea de mayor prioridad o bien para alternar entre varias tareas de igual prioridad.

La suspensión se da cuando una tarea debe dejar de ejecutarse temporalmente, una tarea puede suspenderse a sí misma y a otras, pero solo puede ser retomada desde otra tarea.

El bloqueo se da cuando una tarea debe esperar un evento para continuar su ejecución, tal como la finalización de un timer, el cambio de una bandera o bien el aviso de otra tarea. La espera de una interrupción no puede bloquear directamente a una tarea, pero puede enviarse el aviso de desbloqueo desde el callback de dicha interrupción.

Las tareas interactúan entre sí mediante herramientas de comunicación y de sincronismo.

Las principales herramientas de sincronismo son los semáforos. Un semáforo contiene un contador. “Ceder” o “dar” un semáforo aumenta el contador y tomarlo lo disminuye, si se intenta tomar un semáforo que está en cero, la tarea se bloquea.

Un caso particular es el semáforo binario, el cual solo puede aumentar hasta uno, es decir, reconoce dos estados. Es particularmente útil para desbloquear una tarea desde otra de forma genérica.

Un caso particular del semáforo binario es el semáforo de exclusión mutua o “mutex”, el cual solo puede ser retornado por la misma tarea que lo tomó, se utiliza para proteger espacios de memoria de una escritura por parte de otra tarea, pudiendo dejar los datos compartidos en un estado inconsistente.

Otro método de sincronismo es la notificación, la cual es más rápida y eficiente que el semáforo binario, pero sirve solo para notificar una tarea en específico.

Las estructuras de comunicación tienen como funcionalidad principal enviar datos de una tarea a la otra, aunque también pueden utilizarse para la sincronización. Ejemplos de estas estructuras son el *stream buffer*, el cual transmite cadenas de bytes enteros de longitud variable, ideal para flujo continuo de datos, el *message*

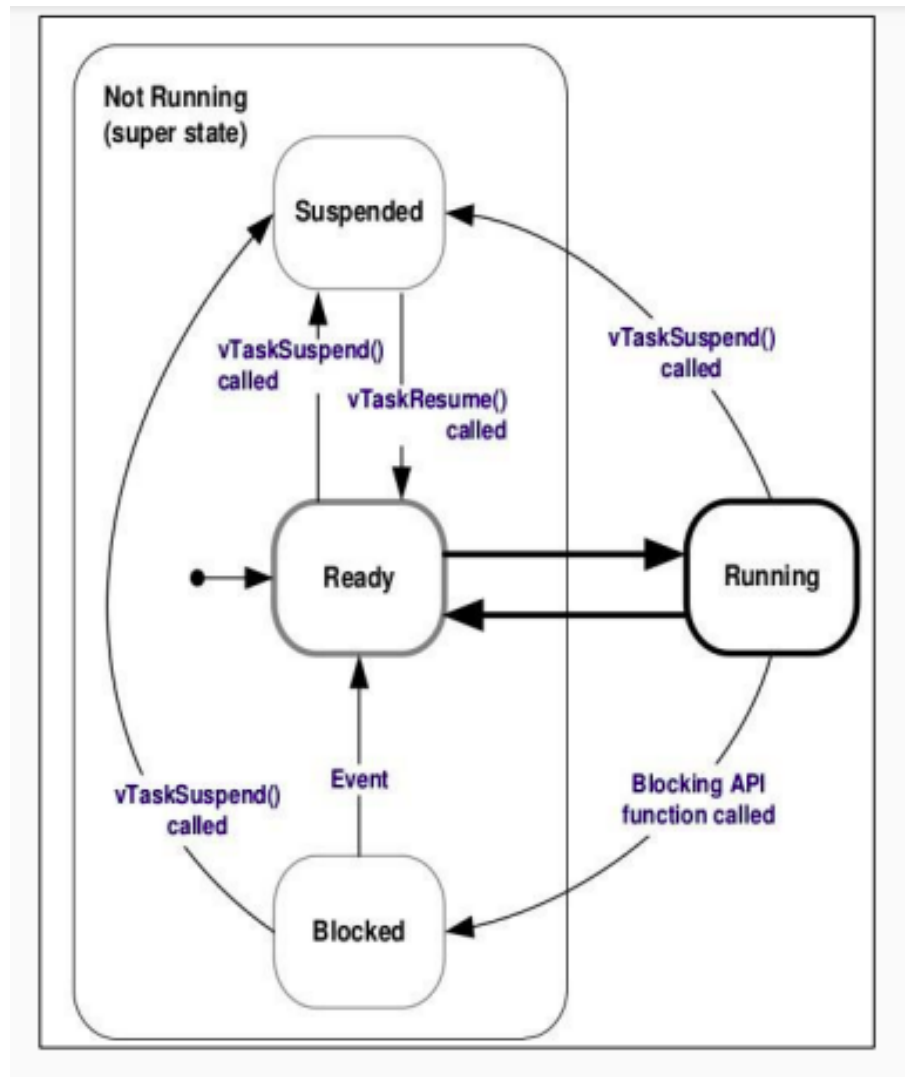


Figura 6: Diagrama de estados de una tarea en un RTOS.

buffer, el cual envía mensajes de bytes enteros de longitud definida, mejor para datos ya filtrados, y la *queue* o cola de mensajes, la cual puede transmitir elementos de tamaño definido, por ejemplo estructuras de datos.

5.2.2. Comunicación con el servidor

Para comunicar el controlador con el servidor se seleccionó, en la capa física del modelo OSI el protocolo serial RS-485 como ya se indicó. El transceptor “traduce” las señales diferenciales al protocolo UART por el cual se comunica el microcontrolador.

Para el microcontrolador se desarrollaron dos librerías que gestionen la comunicación. La primera, llamada `app_uart` se ubica en la capa de enlace de datos y se ocupa de gestionar la recepción y el envío de tramas crudas, sin ningún tipo de procesamiento. La segunda, llamada `protocol`, se ubica en la capa de aplicación y le corresponde procesar las tramas entrantes así como preparar las salientes.

Los datos entrantes son recibidos en una tarea dedicada de la librería `app_uart` y cargados en un stream buffer, el cual debe ser leído por la tarea correspondiente de la librería del protocolo.

La librería `app_uart` se compone de tres funciones y una tarea:

```
1 void uart_init(void);
2 void uart_rx_task(void *pvParameters);
3
4 void app_uart_send(uint8_t *trama, int len);
5 StreamBufferHandle_t uart_get_rx_streambuffer(void);
```

La función de inicialización primero crea el stream buffer, el cual es local, por lo que para recuperar los datos se debe acceder a él mediante un “getter”, es decir una función pública que devuelve la estructura de datos.

Luego se configura la operación del periférico, basado en las guías de la documentación oficial y códigos de ejemplo, y luego adaptado a las necesidades de este proyecto. Se crea una estructura de configuración donde se definen los parámetros de operación, se elimina el bit de paridad y el control de flujo por hardware, ya que el método de detección de errores se aplica en la capa de aplicación y el flujo se programa a mano. La inicialización configura el puerto UART1 para trabajar en modo RS-485 y reaccionando a eventos mediante interrupciones, todo esto de forma transparente al desarrollador.

Lo siguiente es instalar el driver, en este caso se definen los tamaños de los buffers de recepción y de transmisión (este último en 0). Estos buffers son “invisibles” para el programador, por ejemplo el buffer de recepción es circular y se va llenando con los bytes entrantes y vaciando con los que se leen explícitamente, el tamaño sirve para ver cuánto se pueden acumular. Se configuró en 2048 para asegurar el funcionamiento de forma conservadora, lo que se corresponde con más de 100 tramas de longitud máxima según se define en el protocolo. El driver también se le carga una cola de eventos junto con su tamaño. Esto configura a la UART para trabajar por interrupciones que se ejecutan de forma transparente al usuario, cada vez que llegan datos, el sistema operativo interrumpe y carga en la cola un evento de bytes entrantes.

Posteriormente, se cargan las configuraciones de la estructura, y se asignan los pines que se utilizaran. Finalmente, se establece el modo de trabajo, en este caso RS-485, que configura el pin antes definido como RTS para comandar el transceptor según la dirección de los datos. El pin restante es para control de flujo por hardware por lo que se deja sin usar.

La tarea `uart_rx_task` reacciona a los eventos y ante un evento del tipo `UART_DATA`, pone los datos en un Stream Buffer static del archivo. Esto lo hace leyendo la cola de eventos.

La cola bloquea la tarea hasta la ocurrencia de un evento, en el caso de que el evento sea de datos se leen y se guardan en el buffer. Hay más tipos de eventos relativos a posibles fallos, como un desborde del buffer de recepción o bien problemas al recibir los datos. Se implementaron logs para estos casos sin desarrollar. Quedan como mejora futura el tratamiento de estos eventos cuando se acote el buffer de recepción y se suba la velocidad.

Para enviar los datos por UART se creó la función `app_uart_send`, a la cual se le pasa como parámetro un puntero a un buffer (arreglo de enteros sin signo de 8 bits) y un campo de longitud.

La elección de esta arquitectura se debe a que la implementación del protocolo requiere que los datos entrantes se entreguen en un streambuffer y los salientes se reciban mediante un puntero. Manteniendo estas estructuras se podrían desarrollar

librerías para que el protocolo opere con diferentes capas físicas, como por ejemplo CAN bus.

5.2.3. Protocolo Ramodbus

La implementación de este módulo conllevó un doble desafío, primero la definición de un protocolo de capa de aplicación y segundo la implantación de este como librería, manteniendo robustez, desacoplamiento y portabilidad. Se decidió llamarlo “Ramodbus” por la inspiración a Modbus y el apellido del desarrollador.

Se evaluó utilizar Modbus RTU, dado que es un protocolo robusto, altamente probado y muy documentado con librerías en múltiples plataformas, además de estar diseñado para trabajar sobre RS-485.

La desventaja radica en que, al ser un protocolo orientado a comunicaciones con PLCs, su uso en una aplicación IoT requiere adaptaciones sustanciales para integrar comandos y tipos de datos diferentes. Se decidió entonces desarrollar un protocolo propietario inspirado en Modbus RTU pero adaptado a las necesidades de este proyecto, con la flexibilidad suficiente para ser reutilizado en aplicaciones similares.

De Modbus RTU se mantienen las siguientes características:

- Comunicación maestro - esclavo multipunto con múltiples esclavos
- Arbitraje mediante polling
- Funcionamiento en base a comandos
- Verificación de errores mediante CRC de 16 bits

La primera diferenciación viene de la complejidad de implementar un sincronismo temporal como Modbus RTU, en lugar de manejar los inicios y finales de trama mediante silencios y timeouts, se decidió incluir un byte de inicio de trama y un campo de longitud en el encabezado. De esta forma la trama queda de la siguiente manera:

START	NODO_ID	CMD	LEN	PAYLOAD	CRC16
0xAA	0x0A	0x00	0x00	long variable	0x85 0x8C

La principal innovación viene de los comandos, esta idea surgió en la etapa de implementación como método para optimizar la memoria reservada. Como Modbus RTU es orientado a registros, tiene pocos comandos, pero siempre mantiene un payload mínimo, ya que requiere por ejemplo para encender una salida, la dirección de esta.

En una aplicación IoT se pueden tener funcionalidades muy simples que requieran solo de una orden, como por ejemplo encender el relé que desbloquea la puerta en este proyecto. Se decidió entonces que Ramodbus tenga tantos comandos como sea necesario para cada aplicación específica.

Esto determina en principio dos tipos de comandos: los obligatorios del protocolo y los comandos de la aplicación. Existen tres comandos obligatorios.

El comando 0x00 que del lado del maestro es un *polling*, una encuesta que pregunta al esclavo si hubo algún evento. En el caso de que el maestro requiera enviar un comando, simplemente reemplaza el comando de polling por el del comando. Desde el esclavo un comando 0x00 es un *acknowledge* (ACK).

El comportamiento del esclavo es responder de inmediato con los datos que tenga listos, si el maestro le pide datos, el esclavo probablemente responda con un ACK de inmediato y entregará los datos como respuesta a un ciclo de polling posterior. Esto puede parecer en principio una debilidad, porque se debe configurar el maestro para esperar las respuestas y confirmaciones por varios ciclos hasta considerar reenviar la solicitud. Sin embargo, esto trae una potencial mejora para aplicaciones críticas en tiempo con mayor número de esclavos, ya que al ofrecer una respuesta inmediata, se puede acelerar el ciclo de encuestas, incluso aprovechando el determinismo del RTOS. Esto es uno de los dos factores que tienen el potencial de compensar la diferencia de longitud en las tramas.

El segundo comando obligatorio es el 0x01, solo de parte del esclavo y representa el *not-acknowledge* (NACK), se dispara el CRC se valida mal o si el campo len tiene un valor fuera de rango. Quedan a futuro posibilidades de agregarle más validaciones.

El tercer comando obligatorio es el 0x02, el cual desde el lado del maestro es una orden de reset por software, mientras que desde el lado del esclavo es una confirmación de inicialización exitosa.

Los comandos también se dividen en *Control commands* (comandos de control) y *Stream commands* (comandos de flujo). Los comandos de control son aquellos con índices (decimales) entre el 0 y el 99 y no llevan payload y se usan para disparar eventos de forma directa. La versatilidad del protocolo es que uno define el significado desde el comando 0x03 en adelante, como analogía a Modbus debería tomarse, por ejemplo, un número distinto para encender cada bobina. Esto es el segundo factor que compensa la longitud del encabezado, ya que permite en muchos casos prescindir de payload y enviar tramas más breves. Al enviar un comando de control el campo len se pone en 0. Los comandos de flujo son los numerados decimalmente entre el 100 y el 255 y son todos aquellos que requieran de un payload. Estos permite ,si se quiere replicar algo más similar a Modbus RTU se pueda diseñar la tarea para que lea a qué salida atacar en el payload, pero el uso pensado es el intercambio de datos.

Se sugiere que los comandos de la aplicación tengan correspondencia entre maestro y esclavo. Por ejemplo si un comando desde el lado del maestro es encender una salida, se utilice el mismo desde el lado del esclavo como confirmación.

De la misma manera si los comandos fueran de diferente tipo, por ejemplo el maestro envía una solicitud de datos (control) y el esclavo devuelve los datos (flujo), se pueden usar dos comandos diferentes o bien utilizar en ambas direcciones el mismo stream command, esto es más prolijo, pero por cuestiones de implementación debe incluirse una trama “dummy” en el payload de la orden.

La cantidad de comandos de cada tipo (incluyendo los obligatorios) son un parámetro de inicialización del protocolo. Debido a la implementación, el protocolo especifica que la numeración de los comandos debe ser consecutiva, es decir si se va a usar 10 comandos de flujo, se debe inicializar el protocolo con 10 stream commands y definirlos del 100 al 110. Si se quiere usar un comando 130, pero no los del medio, se inicializa con 30 stream commands, con el costo de memoria que esto conlleva.

El protocolo se implementa en una librería propia, cuyo archivo de cabecera define algunas macros de uso interno pero accesibles tales como el byte de start, la

longitud del encabezado y los índices de comando obligatorios. También define varias estructuras tanto de uso interno como de uso público vía API.

El protocolo se ejecuta mediante dos tareas: el *parser*, el cual recibe los datos, los valida y los pasa a la tarea correspondiente a cada comando, y el *dispatcher* el cual se ocupa de gestionar la transmisión de datos. Ambas funciones son privadas y las tareas se crean mediante la función de inicialización `protocol_init`, la cual lleva como argumento un puntero a un tipo de dato de configuración:

```
1 typedef struct{
2     int ctrl_cmds;
3     int st_cmds;
4     uint8_t masterid;
5     uint8_t nodoid;
6
7     int parser_stack;
8     parser_interface_func buffer_getter;
9     int parser_priority;
10
11    int dispatcher_stack;
12    dispatcher_interface_func sender;
13    int dispatcher_priority;
14
15 } protocol_params_t;
```

Los primeros campos son las cantidades de comandos de cada tipo, luego le siguen las id del nodo propio y el maestro. Le siguen algunos de los parámetros de configuración de las tareas para que el usuario de la api pueda definir en cada una el stack y la prioridad. `Buffer_getter` y `sender` son datos del tipo

```
1 typedef void (*dispatcher_interface_func)(uint8_t*, int);
2 typedef StreamBufferHandle_t (*parser_interface_func)(void);
3 ;
```

Que son punteros a funciones, hacen de interface y se les pasa en el main las funciones de la API de `app_uart`, esto para mantener las funciones compatibles con otros protocolos.

El protocolo es reactivo, para gestionar la comunicación y el sincronismo entre el parser y las tareas que reaccionen a cada comando se parte desde la premisa de que a cada comando que recibe el esclavo le corresponde una tarea de forma unívoca. La idea original sin diferenciar comandos proponía unificar sincronismo y comunicación mediante message buffers. La ventaja que ofrecen las funciones de FreeRTOS es que la función de recepción puede bloquear una tarea en ausencia de datos. Se propuso crear un arreglo de estos, de forma tal que el índice del arreglo se corresponda con el número de comando, de aquí surge la necesidad de definir los comandos de forma contigua. Esto permite transmitir solo los datos sin ninguna información extra, ya que el comando define que tarea lo recibe.

El uso únicamente de buffers de mensaje tiene dos grandes problemas y ambos nacen del hecho de tener comandos que no transportan datos: El primero es que, al tener que definir el arreglo con buffers de igual tamaño, se desperdicia memoria. El segundo es que al no haber payload, los buffers de recepción nunca despertarían a la tarea, por lo que habría que enviar siempre un byte “dummy”.

Frente a esto surge la idea de separar los comandos, aquellos que transporten datos se llamaron comandos de flujo y mantuvieron la arquitectura, desfasando en 100 unidades los índices. Los comandos cortos, los de control, se sincronizan con

sus respectivas tareas usandose de forma análoga a sus contrapartes como índices de un arreglo de semáforos binarios. El parser libera el semáforo correspondiente desbloqueando la tarea, que debe luego tomar el semáforo para volver a bloquearse al cumplir su función.

Como no se puede crear un arreglo estático dentro de una función, ambos arreglos se crean como punteros estáticos en el archivo de implementación.

```
1 static MessageBufferHandle_t *cmd_buff = NULL;
2 static SemaphoreHandle_t *cmd_smph = NULL;
```

Posteriormente, la función de inicialización luego de llenar los valores estáticos de las id, y validar las cantidades de comandos, crea ambos arreglos.

Como los objetos se deben crear con funciones específicas, primero se reserva la memoria mediante `malloc` y luego mediante un bucle `for` se crea cada buffer y cada semáforo.

Luego, cada tarea, mediante una función getter pública, recupera según corresponda el buffer de mensaje o el semáforo binario correspondiente.

La tarea parser mediante la función asignada como parámetro utiliza un stream buffer para despertar e ir pre procesando los datos. También hay un buffer temporal donde se van guardando los datos válidos entrantes, y se va sobrescribiendo en las iteraciones

El funcionamiento es mediante una máquina de estados finita, la cual comienza en un estado de espera con la tarea bloqueada. Al recibir un dato la tarea despierta y se compara hasta coincidir con el byte de inicio de trama. En ese caso evoluciona a un estado de validación de id. En caso de que la id sea la de otro nodo, el dato se descarta y se vuelve a la espera, en el caso de que el mensaje sea para ese nodo, se guarda el dato en el buffer y se evoluciona a un estado que lee y guarda el campo de comando y el campo de longitud. Si la longitud del payload es mayor a 0 se pasa a un estado intermedio que guarda el payload, y en ambos casos se termina en un estado de validación del CRC. Si el CRC es válido se pasa a un estado de parseo donde se libera el semáforo o bien se envía el payload por el buffer correspondiente, para luego mediante una notificación de tarea avisar al dispatcher que envíe una respuesta. Si el CRC no es válido se pasa a un estado de rechazo donde se notifica a la tarea de dispatcher que envíe un NACK.

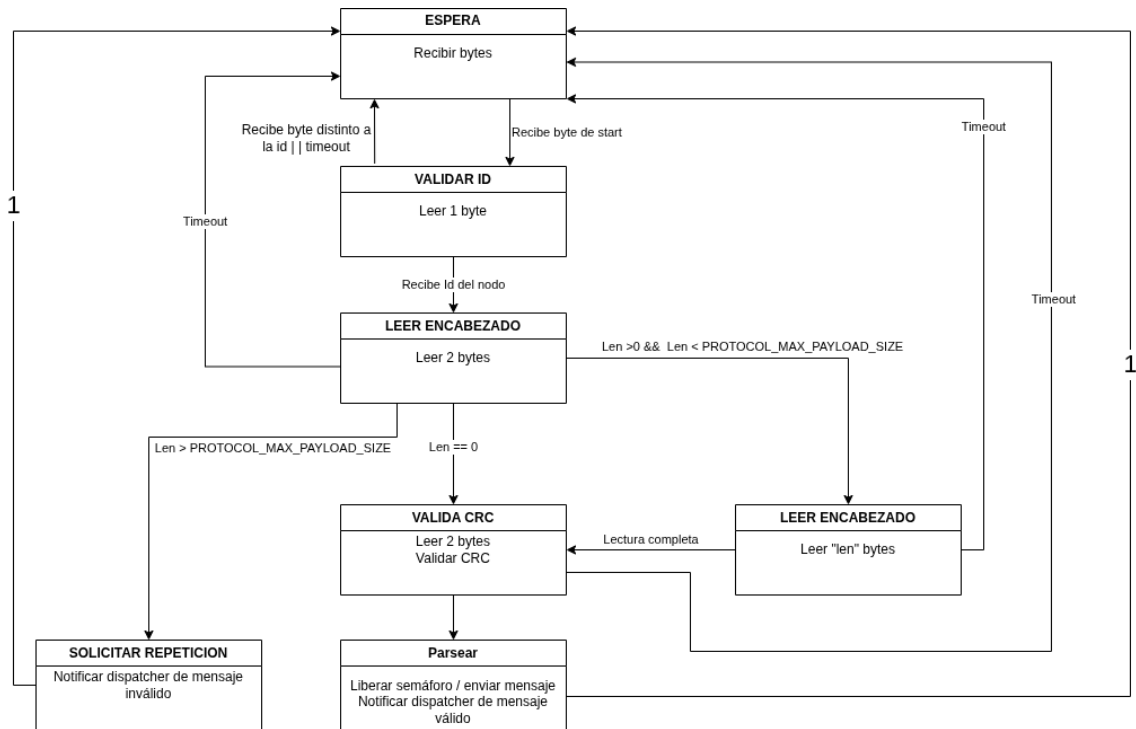


Figura 7: Máquina de estados finitos del parser del protocolo.

Cuando una tarea de algún módulo quiere mandar un dato debe utilizar la función *composer*. Esta recibe como argumentos el comando a enviar, la longitud del payload y un puntero a un buffer para los datos. Esto para no pasar la trama por copia, haciendo el transporte más eficiente. Esto conlleva un riesgo por lo que se añade como argumento un semáforo que bloquee la escritura de este.

Para la comunicación entre el composer y el dispatcher se utiliza una cola de mensajes, ideal para transmitir estructuras como las creadas para el protocolo:

```

1      typedef struct {
2
3          uint8_t id;
4          uint8_t cmd;
5          uint8_t len;
6          uint8_t *payload;
7          uint16_t CRC;
8      } msj_t;
9
10     typedef struct {
11
12         msj_t msj;
13         SemaphoreHandle_t semaforo;
14     } q_msj_t;
  
```

La estructura de cola `q_msj_t` contiene al semáforo y a una variable de la estructura `msj_t`. La función calcula el CRC en base a los datos recibidos y llena la estructura para encolarla. Previamente, si se utilizó un semáforo, lo toma para proteger al buffer de ser modificado durante la transmisión.

La tarea dispatcher es la encargada de llamar a la función (en este caso `app_uart_send`) que transmite los mensajes. Esta guarda el mensaje en un buffer y lo envía mediante la función descrita anteriormente

La tarea se mantiene bloqueada hasta recibir una notificación del parser, es decir solo transmite en respuesta a las encuestas. En el caso de que la notificación sea de fallo transmite un comando de NACK. En el caso de que la notificación sea de aviso de mensaje válido, va a revisar si en la cola del composer hay datos en espera que transmitir, para armar el mensaje y enviarlo. Luego de enviarlo libera el semáforo para permitir la reescritura del buffer de datos. En caso de que no haya datos en cola, construye y envía un comando de ACK.

Este comportamiento revela dos características importantes del protocolo: Primero que, dada la alta prioridad que deben tener las tareas de este respecto a las demás de la aplicación para mantener la fluidez, lo más probable es que la respuesta a cualquier comando siempre sea un ACK y que los datos esperados se transmitan en un ciclo de polling siguiente.

Lo siguiente es que los datos se transmiten en orden de llegada, este orden vendrá dado en la aplicación por la prioridad que se le asigne a cada tarea.

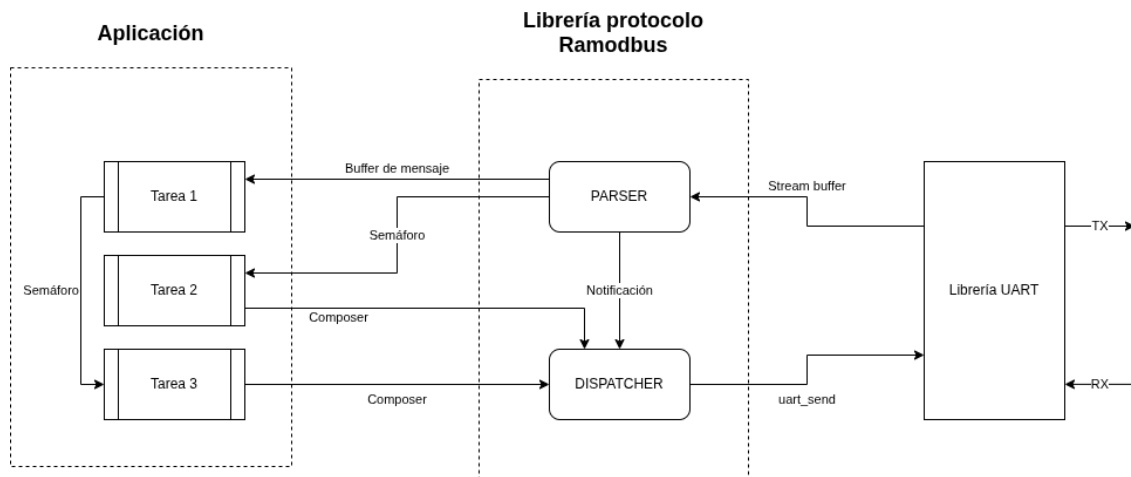


Figura 8: Esquema de operación del protocolo.

Si bien a cada comando (exceptuando los dos primeros) se le corresponde una tarea, esto no necesariamente se cumple en el sentido inverso. No todas las tareas tienen que devolver confirmaciones. Además, puede que haya tareas que encolen datos, pero que reaccionen no a un comando de un protocolo sino a cualquier otro evento, como una interrupción o la orden de otra tarea.

5.2.4. Tareas reactivas al protocolo

La estructura de una tarea que reacciona a un comando del protocolo es, en principio, sencilla. Durante la inicialización obtiene, mediante la función *getter* correspondiente, el semáforo binario o message buffer asociado al comando y verifica que la operación haya sido exitosa. Luego entra en un bucle infinito en el que permanece bloqueada indefinidamente hasta recibir un mensaje o liberarse el semáforo. Una vez desbloqueada, ejecuta la acción correspondiente y vuelve al estado bloqueado al finalizar. Como ejemplo, la tarea que implementa el comando obligatorio de reinicio se muestra a continuación.

```

1      void reset_task(void *pvParameters) {
2
3          SemaphoreHandle_t cmd_sem = protocol_get_ctrl_sem(CMD_RESET);
4          if (cmd_sem == NULL) {
5              ESP_LOGE("RESET", "No se pudo obtener el semaforo de reset");
6              esp_restart();
7          }
8
9          while(1){
10             if (xSemaphoreTake(cmd_sem, portMAX_DELAY) == pdTRUE) {
11                 esp_restart();
12             }
13         }
14     }

```

5.2.5. Manejo de entradas y salidas digitales

Este módulo implementa el control de las dos salidas digitales del controlador, correspondientes al relé de apertura de la puerta y al relé del timbre, además de la lectura de la entrada digital asociada al pulsador del timbre.

Durante la inicialización se configuran los pines de entrada y salida, se obtienen mediante la API del protocolo los semáforos correspondientes a los comandos de apertura de puerta y captura de fotografía, y se crea un temporizador de software utilizado para controlar la duración del accionamiento del relé del timbre.

La apertura de la puerta se implementa mediante una tarea reactiva al protocolo. Esta permanece bloqueada sobre el semáforo asociado al comando correspondiente y, al recibir la orden, activa el relé durante un tiempo configurable. Antes de volver al estado de espera envía la confirmación de ejecución mediante el protocolo de comunicaciones.

La detección de la pulsación del timbre se realiza mediante una interrupción externa configurada sobre el flanco descendente de la señal. Cuando esta ocurre, el manejador de interrupción libera el semáforo asociado a la captura de fotografías para despertar la tarea correspondiente y activa el relé del timbre. Como las rutinas de interrupción deben ejecutarse en el menor tiempo posible, la desactivación del relé no se realiza desde la propia interrupción sino mediante un temporizador de software, cuyo callback se ejecuta una vez transcurrido el tiempo programado.

Esta arquitectura permite minimizar el tiempo de ejecución de la rutina de interrupción, delegando el procesamiento de mayor duración a las tareas del sistema operativo y a los mecanismos de temporización provistos por FreeRTOS.

5.2.6. Módulo de Identificación NFC

Para la lectura de credenciales de acceso, se utilizó un módulo basado en el chip PN532. La elección de este integrado radica en su amplia compatibilidad con protocolos RFID a 13,56 MHz, en particular la norma ISO/IEC 14443A, lo cual permite interactuar fluidamente con etiquetas de la familia NTAG21x (NTAG213, NTAG215 y NTAG216).

El control del chip se resolvió integrando el componente de software libre “garag__esp-idf-pn532”[9]. Esta biblioteca abstrae la comunicación SPI de bajo nivel y proporciona una API para la inicialización del controlador, la configuración

de sus temporizadores y el envío de comandos al circuito integrado PN532.

El funcionamiento del módulo está gestionado por el RTOS mediante la creación de tres tareas concurrentes: *Lectora*, *Programadora* y *Árbitro*.

El sistema opera bajo dos estados mutuamente excluyentes: Modo Lectura y Modo Programación. Dado que ambas tareas (`nfc_task` y `nfc_config_task`) compiten por el uso del mismo periférico SPI y el mismo handler del PN532, se implementó un mecanismo de exclusión donde una tarea suspende a la otra (`vTaskSuspend` y `vTaskResume`) según el estado dictaminado por la tarea de arbitraje.

El árbitro, a su vez, reacciona directamente a las peticiones del coordinador central. Utiliza un semáforo binario cedido por maestro para alternar el estado. Si se recibe el comando de programación (`CMD_PROGMODE`), el sistema entra en una ventana de tiempo limitada (*timeout*) para configurar una nueva etiqueta. Si no se detecta actividad o finaliza el tiempo, el sistema retorna de forma segura a su estado natural de lectura, garantizando la operatividad ininterrumpida del control de acceso.

Uno de los mayores desafíos en sistemas de identificación por radiofrecuencia es la mitigación de ataques de clonación y repetición. Para abordar esto, el sistema no se limita a leer el UID (Identificador Único) de 7 bytes de la tarjeta, sino que implementa un esquema de autenticación dinámica y contadores de lectura, aprovechando el contador ya incorporado en el firmware de los tags.

Durante el modo de programación, al detectar una tarjeta NTAG virgen, el sistema ejecuta una función de derivación de claves utilizando HMAC-SHA256 (mediante la librería `mbedtls`). Se utiliza una clave maestra residente en el firmware del equipo y el UID único de la tarjeta para generar un hash de 32 bytes. Los primeros 6 bytes de este hash se truncan y se inyectan en la memoria de la NTAG para configurar el *Password* (PWD) de 4 bytes y el *Password Acknowledge* (PACK) de 2 bytes.

De esta forma, cada credencial del sistema posee una contraseña única matemáticamente vinculada a su hardware que se calcula en el acto, evitando la vulnerabilidad de alojar contraseñas en una base de datos.

El funcionamiento del sistema anti clonación se detalla en la sección 7

Adicionalmente, se configuran los registros de control de la NTAG para activar la función *ASCII Mirror*. Esta característica escribe automáticamente el valor actual del contador de interacciones interno de la etiqueta en la memoria de usuario cada vez que la tarjeta es leída. Se le llama espejado ASCII, ya que el valor del contador, alojado internamente en 3 bytes, se escribe en formato ASCII, es decir que el número que se guarda es el valor ASCII del caracter con el que se leería el número en formato hexadecimal, ocupando el doble de espacio. Por ejemplo el byte 0x2F (47 en decimal) se guardaría como 50 70, es decir los valores del caracter “2” y el caracter “F”. 12.3

Página (Hex)	Byte 0	Byte 1	Byte 2	Byte 3
0x29/0x83/0xE3	MIRROR	RFUI	MIRROR_PAGE	AUTH0
0x2A/0x84/0xE4	ACCES	RFUI	RFUI	RFUI
0x2B/0x85/0xE5	PWD0	PWD1	PWD2	PWD3
0x2C/0x86/0xE6	PACK0	PACK1	RFUI	RFUI

Cuadro 1: Mapa de configuración de registros de seguridad y espejado en familia NTAG213/215/216.

El byte MIRROR contiene las configuraciones del espejado ASCII, se configura

con el valor 0x85. Los dos bits más significativos indican que el espejado se realizará sobre el contador (siendo las otras opciones espejar la UID, ambos o ninguno). Los siguientes dos indican en que posición de la página seleccionada se comienza a escribir el dato, en este caso al principio. El bit 2 es la habilitación de modulación fuerte, la cual se deja por defecto en 1 mientras que los demás bits están reservados y se mantienen en 0.

Todos los bytes notados “RFUI” son de reserva y se mantienen en su valor por defecto 0x00. El byte MIRROR_PAGE determina en que página se escribirá el espejo contador. El byte AUTH0 define a partir de qué página se protegen los datos con contraseña, el valor por defecto es 0xFF, es decir que se permite la lectoescritura de todas las páginas, lo habitual es configurarlo en 0x04, es decir dejar libres las páginas de información de fábrica y proteger el resto.

El byte ACCES se configura con el valor 0x98, el cual si se analiza bit a bit indica que la contraseña protege contra escritura y lectura, y que el contador está activado. Además, permite establecer un número máximo de intentos de lectura y activar un bloqueo permanente del tag. Estas opciones se descartaron por el riesgo de dejarlo inutilizado accidentalmente, dado que los errores de lectura pueden darse por motivos físicos, teniendo en cuenta que no existe información pública de que se halla logrado descifrar una contraseña de un tag mediante fuerza bruta.

La página PWD guarda la contraseña del tag, aquí se escribe la contraseña de 4 bytes calculada mediante el hash. La página PACK contiene los dos bytes de confirmación de la autenticación con contraseña que también se calcula mediante el hash.

En su régimen nominal de lectura, el flujo de ejecución sigue una estricta secuencia de validación. Al acercar una tarjeta, se lee su UID y se calcula localmente el hash esperado. Posteriormente, se envía un comando de autenticación al PN532 pasándole la contraseña calculada (NTAG_COMMAND_PWD_AUTH). Si la NTAG devuelve el PACK correcto, se confirma la legitimidad de la tarjeta.

Superada la barrera de seguridad, se emite un comando de lectura rápida (NTAG_COMMAND_FAST_READ) apuntando a la dirección configurada previamente para el *Mirror*. El valor retornado, que representa el contador de lecturas en formato de caracteres hexadecimales ASCII, es convertido a un entero de 32 bits (`strtoul`).

Finalmente, la información a reportar se condensa en un *payload* de 10 bytes: los 7 bytes correspondientes al UID físico de la tarjeta, seguidos de 3 bytes que representan el valor del contador dinámico. Este bloque de datos se despacha mediante la función *composer* de Ramodbus bajo el comando CMD_NFC, sincronizando el envío mediante un semáforo para asegurar la integridad de la memoria compartida hasta que el despachador de la capa de red confirme su transmisión hacia el servidor.

Se puede, mediante un comando de las NTAG, acceder al valor del contador de forma directa, sin tener que leer un registro ni convertirlo desde ASCII a un número, pero este comando no aumenta el contador como una lectura, por lo que para usarlo debería realizarse una lectura a un registro cualquiera, lo cual agrega puntos de falla en la comunicación del tag con el módulo.

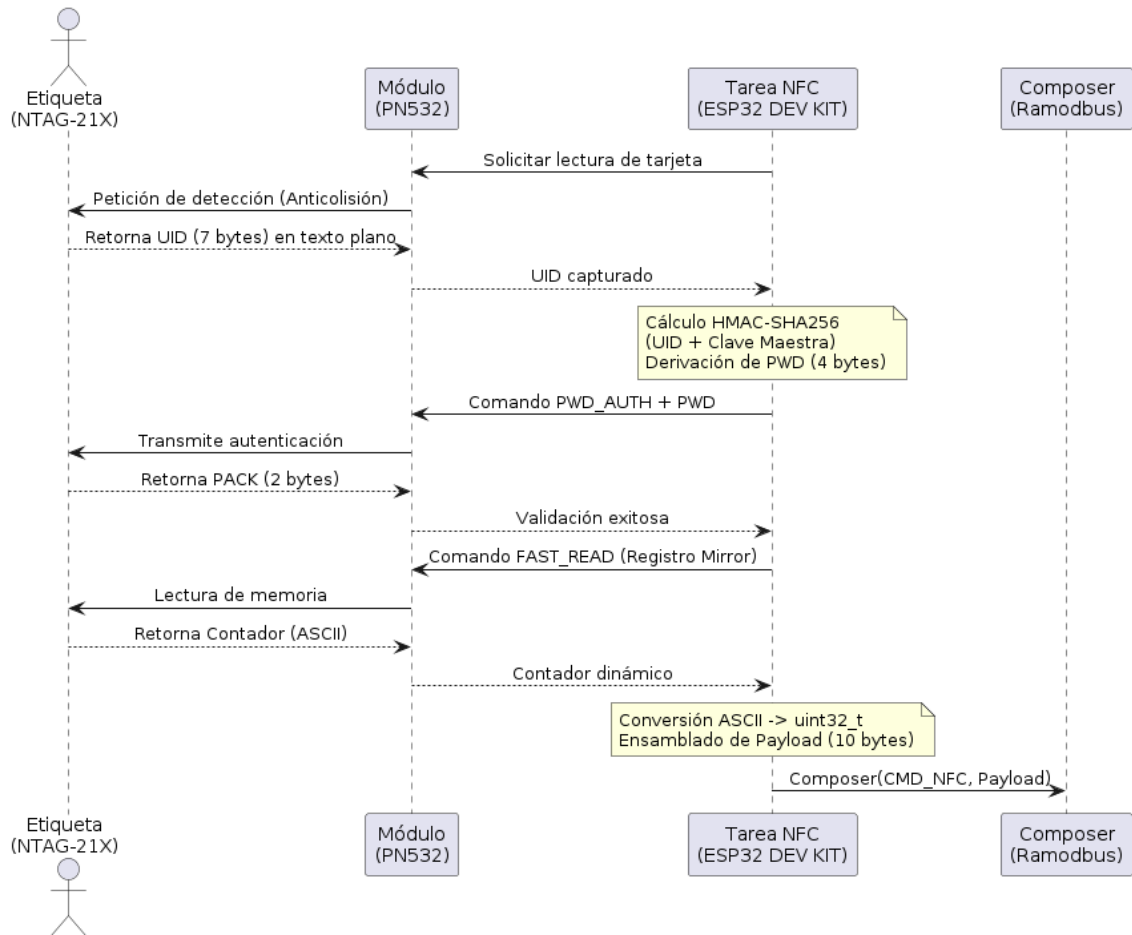


Figura 9: Diagrama UML de secuencia de la validación de un tag NFC

5.2.7. Comunicación con la ESP32-CAM

Para comunicar el controlador principal con el módulo de la cámara, se estableció una comunicación UART, asociada al periférico UART 2 del ESP32.

El protocolo UART está diseñado para conexiones punto a punto y permite la comunicación *full duplex*, lo que simplifica esta implementación respecto a la del servidor, dado que no es necesario preocuparse por arbitrajes ni colisiones. Asimismo, al ser una comunicación menos crítica y de menor distancia, se reemplazó el CRC como método de verificación de tramas por un *checksum*.

La librería `uart_cam` abarca tanto la recepción y el envío de datos, como su procesamiento, “resumiendo” las dos bibliotecas que se utilizan en la comunicación principal.

La tarea `cam_uart_rx_task` es muy similar a su contraparte de la UART 1, solo que valida en el acto el checksum y, si correspondiera envía el mensaje de error. Posteriormente, retransmite la información recibida (confirmaciones de acciones) al servidor mediante el *composer*

Para el envío de datos se programó la función `cam_uart_send`, que calcula el checksum y envía la trama. Esta simpleza se debe que al tratarse de una comunicación full duplex, el ESP32CAM siempre está “escuchando” y no se requieren de condiciones previas para transmitir un mensaje.

Se desarrollaron también módulos relativos a las funcionalidades de la ESP32-

CAM, que actúan como puentes entre esta y el servidor.

5.2.8. Comando de la conectividad WiFi

Para administrar la conexión de la ESP32-CAM a la red local de la institución se desarrolló un módulo intermedio que se comunica con el servidor de forma directa y replica las órdenes al periférico. El WiFi se comanda mediante un comando de flujo, es decir que tiene un payload determinado. Se decidió utilizar un solo comando para administrar todos los casos posibles desde la misma tarea. Se definió que el payload máximo admisible en una trama sea de 10 bytes, la cantidad exacta para las UID y los tres bytes del contador en las transacciones NFC. No se definió un número mayor para ahorrar memoria, ya que se reserva memoria para n buffers de mensaje de longitud igual. Esto pone en apuros mensajes que puedan llegar a ser más extensos, como por ejemplo las credenciales de conexión a la red. Es por esto que se estableció que, de ser necesario, se subdivida el payload en trozos o *chunks*, quedando el payload definido de la siguiente forma:

- **Byte 0:** Subcomando, reconoce cuatro posibles, dos de ellos no requieren más payload y los otros dos sí.
- **Byte 1:** chunk actual
- **Byte 2:** cantidad total de chunks
- **Bytes 3 al 9:** datos

De forma tal que se envían de máximo siete bytes de información por vez. Los subcomandos que conllevan payload son los de cambio de SSID (Nombre de la red) y de contraseña de acceso. Los restantes son una orden de conectar y desconectarse de la red, esto para mantener el sistema desconectado fuera de horario de uso, ya que la antena de radio requiere un consumo de energía no despreciable.

La tarea que reacciona a la llegada de mensajes se dedica a parsear los subcomandos, en el caso de las credenciales, las almacena localmente ya armadas, y apenas se cambien ambas las envía a la ESP32-CAM. En el caso de las órdenes directas, las retransmite directamente.

5.2.9. Comando de la cámara

El módulo intermedio de la cámara realiza funciones similares. Se compone de dos tareas. Una reacciona al semáforo del comando `CMD_TAKE_PH`, liberado por una orden del servidor o bien por el callback de la interrupción del pulsador, y envía la orden de tomar una fotografía al módulo de la cámara.

La otra tarea reacciona al comando de flujo que establece la URL del servidor HTTP al cual deben transmitirse las imágenes. El payload de este comando consta de cuatro bytes que conforman la dirección IP del servidor y otros dos que representan el número de puerto, la tarea genera una string del tipo “`http://192.168.XX.XX:XXXX/upload`” y la transmite al controlador secundario.

5.2.10. Prioridades de las tareas

Dada la naturaleza apropiativa del código, resulta indispensable definir correctamente las prioridades considerando la importancia de la ejecución de estas, en primer lugar para el correcto funcionamiento del sistema y en segundo para garantizar una experiencia de usuario fluida.

Módulo	Tarea	Prioridad FreeRTOS
Sistema	Reset	20
UART-RS485	Rx	10
Protocolo	Dispatcher	9
Protocolo	Parser	8
DIO	Puerta	7
NFC	Lectura de tags	6
UART-Camara	Rx - cámara	5
NFC	Arbitraje	4
Wifi	Wifi	3
Camara	Camara	3
NFC	Configuración	3

Cuadro 2: Distribución y esquema de prioridades de las tareas en FreeRTOS.

La prioridad máxima se corresponde a la tarea que ataja los comandos de reset, dado que si se manda a reiniciar por software no tiene sentido que se proceda la ejecución de otras tareas.

Le sigue el receptor de tramas del bus serial, dado que el sistema debe estar siempre atento a las encuestas del maestro. La siguiente tarea en despertar cronológicamente es el parser, el cual luego de procesar la trama recibida notifica al dispatcher, el cual se configuró con prioridad mayor para priorizar las respuestas inmediatas. La máxima prioridad del sistema es responder al servidor.

Luego le siguen las funcionalidades, donde la más prioritaria es la de abrir la puerta, ya que es lo más perceptible por el usuario. Siguiendo una lógica similar, la lectura de tags debe realizarse de forma rápida para asegurar fluidez en el acceso.

Las funcionalidades relativas a la ESP32-CAM son menos críticas en tiempo y demorarse ante las demás sin que esto afecte el sistema o sea percibido por algún actor humano

Finalmente, la tarea de configuración de tags se crea dentro de la tarea NFC principal, por lo que debe crearse con menor prioridad. El árbitro tiene menor prioridad que la tarea de lectura para que no pueda interrumpir un intento de acceso.

5.3. Firmware del ESP32-CAM

Para el kit de desarrollo ESP32-CAM, la arquitectura de software resulta a grandes rasgos una versión simplificada de la del controlador principal, manteniendo la arquitectura de tareas de FreeRTOS para manejar las distintas responsabilidades y reaccionar a los distintos eventos de forma concurrente, respondiendo a comandos externos y devolviendo confirmaciones e información relevante.

5.3.1. Comunicación UART

La ESP32-CAM ocupa el rol de un periférico, por lo que si bien la comunicación es punto a punto y no podemos hablar de un “esclavo”, conceptualmente se puede decir que está en una jerarquía inferior al controlador principal, es por eso que el comportamiento de la librería `uart` se asemeja más al de un esclavo `Ramodbus` simplificado. Se tomó como base la misma librería `uart_cam` pero adaptada. La configuración se mantuvo tal cual, utilizando incluso el mismo periférico UART 2.

La tarea `cam_uart_rx_task` es bastante similar a su contraparte, la recepción y validación se mantienen, mientras que el parseo se comporta de manera similar a `ramodbus`, liberando semáforos o transmitiendo buffers de mensaje según corresponda, resolviéndose de forma “manual” dentro de la misma tarea dada la reducida cantidad de opciones de ruteo.

Se añaden funciones *getter* para el buffer de mensajes de WiFi, el buffer de URL del servidor HTTP y el semáforo que indica tomar una fotografía.

Se mantiene también la función de envío `cam_uart_send`.

5.3.2. Conexión WiFi

El módulo de gestión WiFi en la ESP32-CAM tiene como propósito administrar la interfaz de red inalámbrica en modo *Station*. Al operar como un periférico subordinado al controlador principal, su comportamiento está supeditado a los comandos recibidos por UART, pero mantiene cierta autonomía para gestionar la persistencia del enlace y el ahorro de energía.

Como se detalló en el apartado del controlador principal, si bien este recibe las credenciales divididas en *chunks* debido a las limitaciones de memoria de los buffers de mensaje, este se encarga de reensamblarlas. Por consiguiente, la ESP32-CAM recibe las credenciales (*SSID* y *Password*) en estructuras planas y completas.

Para manejar el flujo de red de forma concurrente sin bloquear la recepción de comandos, la arquitectura se dividió aprovechando las herramientas de FreeRTOS y el *framework* nativo del microcontrolador:

- **Recepción de comandos (`app_wifi_com_task`):** Esta tarea permanece bloqueada aguardando datos en un *MessageBuffer* alimentado por la recepción de la UART. Al recibir una trama, evalúa el primer byte para identificar el comando. Si se trata de credenciales de red, actualiza los arreglos globales en RAM de forma segura. Si recibe un comando de acción (`WFON` o `WFOFF`), invoca las funciones del driver WiFi para encender, apagar o reconectar el periférico, estableciendo una bandera fundamental: `WIFI_MUSTBE_BIT`.
- **Gestión de eventos (`event_handler`):** Actúa como un *callback* que reacciona a los eventos internos del sistema (como desconexiones físicas o la obtención de una IP). Aquí reside la resiliencia del enlace: si ocurre una desconexión accidental, se verifica el estado de la bandera `WIFI_MUSTBE_BIT`. Si el sistema “debería” estar conectado, el *handler* dispara automáticamente una secuencia de reintentos sin necesidad de intervención externa.
- **Mantenimiento y persistencia (`app_wifi_task`):** Una tarea de bajo peso dedicada a reaccionar a las banderas de un *EventGroup* (`s_wifi_event_group`). Su función principal es descargar de tareas bloqueantes al flujo principal,

encargándose de los ciclos de reconexión y de escribir las nuevas credenciales en la memoria no volátil (NVS), garantizando que la cámara recuerde la red tras un reinicio.

Finalmente, la comunicación fluye también en sentido inverso. Cuando la ESP32-CAM logra conectarse a la red y el router le asigna una dirección, el evento `IP_EVENT_STA_GOT_IP` empaqueta los 4 bytes correspondientes a la dirección IPv4 obtenida y los transmite de regreso al ESP mediante la función `cam_uart_send` para que este los retransmita al servidor. Esta confirmación es crítica para que el servidor central sepa a qué IP exacta debe realizar las peticiones HTTP a la hora de capturar una fotografía.

5.3.3. Cámara

El módulo encargado de la captura de imágenes toma como base el componente oficial del fabricante `espressif/esp32-camera`. Esta biblioteca provee una API de alto nivel que abstrae la complejidad de los buses I2S e I2C (SCCB), requiriendo únicamente la inicialización mediante una estructura de configuración. Los parámetros se adaptaron de los ejemplos provistos por el fabricante, estableciendo el formato de salida en JPEG y la resolución en SVGA para equilibrar calidad de imagen y uso de memoria.

Para mantener la concurrencia del sistema, la lógica se dividió en dos tareas independientes de FreeRTOS:

La `url_task` aguarda la recepción de una trama UART proveniente del controlador principal que contiene la dirección IP y el puerto del servidor HTTP. Al recibirla, formatea la URL de destino y la almacena en la memoria no volátil (NVS) mediante `nvs_set_str`. Esta persistencia permite que el periférico mantenga su configuración ante reinicios o cortes de energía, inicializando el cliente HTTP sin requerir una nueva intervención del controlador principal.

Mientras tanto la `camara_task` es la tarea central del módulo. Permanece bloqueada a la espera de que se libere el semáforo `cmd_sem`, disparado por un evento de acceso (como la validación de una credencial NFC). Al activarse, la rutina ejecuta una secuencia crítica para asegurar la calidad de la captura:

1. **Purga de *Frame Buffer*:** Debido al comportamiento del controlador DMA de la cámara, el búfer suele retener una imagen capturada instantes antes o presentar artefactos visuales tras períodos de inactividad. Para solucionar esto, el firmware solicita una primera captura que es descartada inmediatamente (*dummy frame*), forzando al hardware a exponer y capturar una nueva imagen.
2. **Iluminación y Captura:** Simultáneamente a la solicitud del segundo *frame* (la captura real), se enciende el LED de alta potencia de la placa para garantizar la correcta iluminación del sujeto, apagándose inmediatamente al obtener los datos.
3. **Transmisión HTTP y Tolerancia a Fallos:** La imagen en formato JPEG se envía mediante un método POST al servidor. Se implementó una lógica de resiliencia que realiza hasta tres reintentos en caso de fallos de red o de respuesta del servidor (tiempos de espera o rechazos). Si el fallo persiste, se

liberan y reconstruyen los recursos del cliente HTTP para evitar bloqueos futuros.

Finalmente, de forma independiente al resultado de la transacción HTTP, se transmite una trama de confirmación por UART (`UART_CAM_PH_CMD`). Esta señal impacta en el controlador principal para que libere el flujo lógico e informe al servidor central, el cual será el responsable final de verificar el ingreso físico del nuevo archivo de imagen.

6. Servidor

El servidor constituye el núcleo del sistema. Consiste en una computadora con una distribución de Linux como sistema operativo, conectada por un lado al bus RS-485 mediante una interfaz serie y, por otro, tanto a la red de área local de la institución como a Internet.

La elección de Linux como entorno de ejecución otorga portabilidad al sistema, permitiendo su despliegue sobre diversas plataformas de hardware según la disponibilidad y los requerimientos de la instalación. Durante la etapa de desarrollo y pruebas, se utilizó una computadora portátil con Ubuntu junto a un convertidor USB a RS-485 estándar. Para la puesta en producción en campo, el cambio fundamental a nivel de hardware consiste en la incorporación de un convertidor de grado industrial con aislamiento galvánico (optoacoplado y con protecciones ante transitorios), lo cual previene bucles de masa (*ground loops*) y garantiza la seguridad eléctrica tanto del equipo anfitrión como del bus de comunicaciones.

Cumplido este recaudo, el software puede desplegarse en cualquier servidor o PC convencional que ejecute Linux. Asimismo, una alternativa viable y compacta es el empleo de una computadora de placa reducida (*Single Board Computer* o SBC), como una Raspberry Pi. Esta opción resulta especialmente conveniente cuando no se dispone de infraestructura de servidores previa o cuando se requiere ubicar la unidad de control en proximidad física al tendido del bus. Entre las ventajas añadidas del uso de una SBC destacan su bajo consumo energético y la posibilidad de prescindir de convertidores USB intermedios mediante el conexionado de un transceptor RS-485 directamente a los pines UART/GPIO del procesador.

La arquitectura consta de cuatro procesos:

- Un gestor de la interfaz serial que se encarga del polling y los comandos hacia los esclavos.
- La interfaz de usuario.
- El servidor HTTP.
- Un broker MQTT para comunicar los procesos entre sí.

La interfaz serial no solo gestiona la comunicación con los terminales, sino que también se ocupa del monitoreo de la conectividad de estos por ambas vías y resuelve internamente todo lo relacionado con los accesos mediante tags NFC.

El servidor HTTP se implementa sobre una instancia de Node-RED y también incorpora algunas funcionalidades auxiliares.

La interfaz de usuario está programada en Python y utiliza la API de la aplicación de mensajería “Telegram”.

6.1. Programa principal

El dominio de la interfaz serial se ocupa de realizar las encuestas periódicas, verificar la conectividad y comandar a los esclavos, así como de procesar los datos recibidos por estos.

Está programado en Python y se compone de varios módulos. Cada módulo incluye funciones que pueden ser llamadas desde otros archivos así como procesos que se disparan en hilos paralelos de ejecución (librería `threading.py`). El programa principal `main.py` crea los procesos y dispara los hilos.

En principio esta arquitectura presenta similitudes con la de los terminales implementados en FreeRTOS, pero existen diferencias clave que hay que mencionar, la primera es que si bien los hilos se ejecutan mediante una estructura apropiativa gestionada por un *scheduler* que vive en el kernel del sistema operativo no existe en este caso tal cosa como asignación de prioridades, todos los hilos en estado ready comparten *time slices* del mismo tamaño.

Otro punto clave es el GIL de Python o “Global Interpreter Lock”, un semáforo de exclusión mutua (*mutex*) diseñado para proteger las estructuras de memoria internas y el mecanismo de conteo de referencias frente a condiciones de carrera. Por esto, solo un hilo ejecuta código Python a la vez por proceso, impidiendo el paralelismo real en tareas de cálculo intensivo. Sin embargo, para operaciones de entrada/salida (como la comunicación serie o de red), el GIL se libera durante las esperas, permitiendo una concurrencia fluida y eficiente entre módulos.

A continuación se describen los distintos módulos que componen el sistema.

6.1.1. Módulo de interfaz serial

Este módulo es la interfaz serial del servidor e implementa la ejecución de un maestro del protocolo “Ramodbus”. El hilo principal de ejecución no requiere de una máquina de estados, sino que es secuencial. Intercambia información con los otros módulos mediante una cola de entrada y sendas colas de salida para cada nodo.

Las direcciones o identificadores de los nodos activos se definen en una tupla dentro del archivo de configuración. El hilo de comunicaciones itera cíclicamente sobre estos identificadores ejecutando la siguiente secuencia por cada nodo:

1. Revisa el estado del nodo, si está marcado como desconectado, se omite su consulta para no penalizar el tiempo de ciclo.
2. Comprueba si existen comandos pendientes en la cola asociada a dicho nodo.
3. Si la cola contiene elementos, extrae el comando y lo transmite; en caso contrario, envía una trama periódica de consulta (*polling*).
4. Espera la respuesta del esclavo, luego de que responda o de un timeout pasa al siguiente nodo y repite.

Si un nodo no responde, se incrementa un contador de fallos consecutivos que se reinicia ante cualquier recepción válida. Al superarse un umbral predefinido de reintentos fallidos, el nodo pasa al estado *desconectado* y se emite una notificación al resto del sistema

Cuando se recibe una respuesta válida, si se trata de una simple confirmación (ACK) se descarta. Si en cambio contiene datos útiles, se encapsula en un diccionario con el formato { "id_nodo": id_nodo, "cmd": cmd_recibido, "payload": payload_recibido }

La construcción de tramas se centraliza en una función encargada de estructurar el encabezado y computar el código de redundancia cíclica (CRC). Por su parte, la rutina de recepción bloqueante valida la dirección del destinatario, el encabezado y la coherencia del CRC, retornando la carga útil decodificada o el código de error correspondiente.

En el archivo de configuración se definen tres parámetros que afectan a la velocidad de la comunicación.

- **Velocidad de transmisión (*Baud rate*):** Tasa de transferencia en baudios del enlace físico.
- **Tiempo de espera (*Timeout*):** Tiempo límite que el maestro aguarda la respuesta del esclavo. Se establece con un margen superior al tiempo de respuesta nominal; por ello, la exclusión de nodos desconectados resulta indispensable para evitar retardos ociosos en el barrido.
- **Intervalo entre consultas (*Inter-polling delay*):** Pausa deliberada entre transacciones consecutivas para no saturar el medio y otorgar tiempo de procesamiento a los terminales.

6.1.2. Módulo de control

Este módulo, implementado en el archivo `nexo.py`, centraliza la lógica de control del servidor y la coordinación interna. Define la función pública `encolar()`, empleada tanto por rutinas internas como por otros módulos del proceso para ingresar comandos en las colas de salida sin interactuar directamente con el puerto serie. Asimismo, ejecuta un hilo dedicado a procesar los elementos de la cola de entrada. Ante eventos de desconexión de terminales, publica una alerta en el tópico MQTT correspondiente; mientras que ante la recepción de tramas de datos, invoca a la rutina de análisis sintáctico (*parser*) para despachar la ejecución hacia la subrutina correspondiente según el comando recibido.

6.1.3. Módulo MQTT

Este módulo encapsula el cliente MQTT del proceso. Provee una función de publicación accesible por los demás módulos y ejecuta un hilo de escucha (*listener*) que, mediante su propio analizador sintáctico, procesa los mensajes entrantes según el tópico y la carga útil (*payload*) recibida.

En general hay tres familias de tópicos anidadas en el tópico “sbc”. El módulo `nexo` publica en el subtópico “sbc/notify” para disparar las alertas de desconexión o clonación de tags, y en “sbc/status/<nodo>” donde en <nodo> figura una string del número de nodo en formato hexadecimal. En estos tópicos se informan eventos en el payload, por ejemplo `OK_PUERTA` es el aviso de que el comando de apertura impactó sobre el terminal y el acceso fue desbloqueado y `OK_FOTO` informa que el terminal intentó (con o sin éxito) transmitir una imagen mediante HTTP.

La tercera rama corresponde a los tópicos de comandos bajo el formato `sbc/cmd/`, a los cuales el cliente permanece suscrito para recibir peticiones emitidas desde la interfaz de usuario. En este caso, la carga útil especifica el nodo destinatario; por ejemplo, si se recibe un mensaje en `sbc/cmd/abrir` con el valor `0x0A`, el módulo encola una orden de apertura dirigida al nodo 10, la cual se transmitirá físicamente durante su próxima consulta en el bus RS-485. El payload de algunos mensajes de este tópico incluyen además del número de nodo el ID del chat del usuario que los solicitó, el cual es utilizado en el proceso de Node-RED.

6.1.4. Módulo de gestión de red

El módulo de red realiza las funciones de conexión y mantenimiento de las conexiones a la red local de los terminales. Incorpora el módulo `subprocess`, el cual permite inyectar comandos a la terminal del sistema operativo. Implementa funciones

que mediante comandos de terminal obtienen la dirección IP del servidor, el nombre de la red wifi y la contraseña de acceso, para esta última es necesario tener privilegios de superusuario al ejecutar el script, esto no representa un problema en producción ya que, al correr como servicios todos los procesos se ejecutan desde el usuario raíz.

Los datos obtenidos sirven como argumentos de otras dos funciones que transmiten las configuraciones de red a los nodos, una toma la IP y arma la trama de 6 bytes de IP + puerto para el servidor HTTP (por defecto se utiliza el puerto 1880) y la otra toma el nombre de la red y la contraseña, los segmentan en *chunks* de no más de 7 bytes y arman las tramas de formato subcomando + chunk actual + chunk totales + data, para indicar al terminal que modifique sus credenciales de acceso a la red. Ambas funciones barren con la tupla de nodos y para cada uno encolan los mensajes que hagan falta para configurar la conexión. Estas funciones se ejecutan al inicio del programa desde el main para que el sistema inicie con los nodos configurados.

También se incluyó un hilo de monitoreo, el cual periódicamente recorre un diccionario que asocia cada nodo con su IP reportada y verifica la conexión mediante un ping. Ante sucesivas fallas en el ping, se encola un comando de reconexión al terminal.

El diccionario es global y está definido en el archivo de configuración. Lo rellena el parser como respuesta a los comandos de WiFi provenientes del nodo, los cuales reportan la IP de este.

Esto se definió así dado que al ocupar una red ya existente, el sistema puede adaptarse a las asignaciones de IP dinámicas por DHCP del router.

6.1.5. Módulo de validación de accesos por tags NFC

Este módulo implementa toda la lógica de accesos mediante tags NFC. Gestiona la actualización del “padrón” de accesos, la validación de solicitudes y el registro histórico local.

Incorpora las librerías `os` para operar con archivos, `csv` para leer y escribir archivos de este formato y `datetime` para poder incorporar fecha y hora en los registros.

El padrón es un archivo csv que incluye los campos UID, contador, titular y habilitación. el UID y el contador son los datos recibidos desde el terminal que deben validarse. El titular es el dueño del tag asociado y la habilitación es un campo booleano que permite desactivar tags ya sea manualmente mediante la modificación del archivo o de forma automática cuando se detecta una clonación.

Como la funcionalidad de acceso debe ser dinámica, no se puede consultar al archivo en cada solicitud, la función `cargar_padron_a_ram()` rellena un diccionario con las UID, contadores y titulares de los tags habilitados. Esta función se llama al iniciar el proceso o mediante la llegada de un comando por MQTT desde la interfaz de usuario.

La trama de que envían los terminales consta de 7 bytes para el UID y 3 bytes para el contador. El procesamiento de las tramas lo realiza el parser mediante funciones del módulo de acceso, primero, con los datos recibidos llama a la función `validar`, la cual revisa primero que el UID coincida con alguna de las cargadas en RAM y luego que el contador sea válido mediante los siguientes criterios:

- El contador recibido no puede ser menor al guardado, ya que esto implicaría una clonación

- El contador recibido no puede ser mucho mayor al guardado, ya que esto implicaría que fue leído por fuera del sistema, se deja un margen para posibles fallas, por ejemplo una lectura válida con el servidor caído.
- Un caso excepcional es si el contador guardado tiene el valor -1, en este caso se toma cualquier valor como válido ya que se le asigna -1 a los tags nuevos a incorporar, o por si se quiere volver a incorporar un tag que fue deshabilitado anteriormente, ya que pudo seguir siendo leído por los terminales y aumentando el contador de forma indefinida.

En el caso de que la solicitud sea válida, la función actualiza el contador tanto en RAM como en el archivo y retorna éxito. Si la función retorna éxito, el parser encola un comando de apertura y posteriormente llama a la función `registrar_evento_en_log` la cual registra en un csv la entrada válida. En el caso de que el UID no se encuentre en el padrón, la función de validación devuelve “DESCONOCIDO” y el parser registra el evento.

Si el fallo en la validación está en el contador, la función de validación deshabilita el tag tanto en RAM como en el padrón y devuelve “CLONACIÓN”. El parser además de registrar el evento, dispara una alarma por MQTT para que la interfaz notifique los usuarios.

6.2. Instancia de Node-RED

Para resolver la recepción HTTP y la automatización de tareas secundarias sin añadir complejidad computacional innecesaria al código principal en Python, se implementó una instancia de Node-RED. Esta herramienta permite orquestar flujos de datos de forma ágil mediante una interfaz gráfica basada en la interconexión de nodos, lo que resulta ideal para mantener un orden visual claro. Dado que su conveniencia disminuye frente a lógicas con demasiados escenarios condicionales, su integración en esta arquitectura se delimitó estrictamente a la ejecución de procesos simples, aprovechando su rapidez de implementación y la disponibilidad de bloques preconfigurados (incluyendo funciones programables en JavaScript) para interactuar con diversos protocolos.

El principal uso que se le da en el sistema es el de servidor HTTP. Mediante un bloque POST recibe las imágenes transmitidas por los terminales y las guarda en un directorio específico mediante un nodo de archivo.

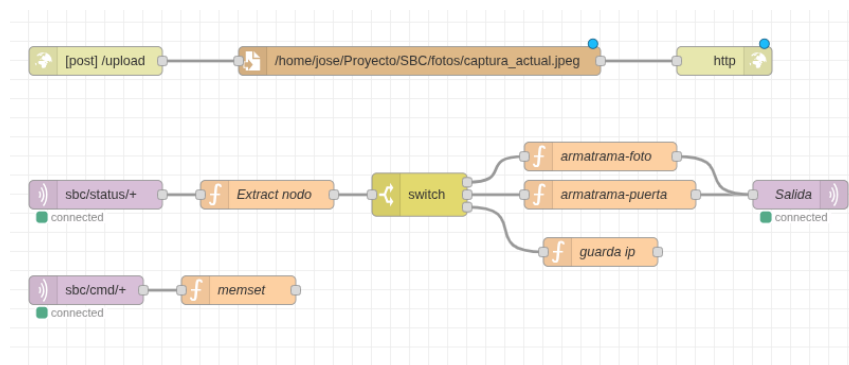


Figura 10: Flujo de automatización de endpoint HTTP.

Este flujo también incorpora un cliente MQTT que suscribe al tópico de comandos,

el cual guarda en memoria el nodo y el ID del chat del usuario que lo disparó. También suscribe al tópico de status, para retransmitir la información al tópico `sbc/notify`. Esto lo hace para el caso de los comandos de apertura, para enviar la confirmación solo al usuario que la solicitó. En el caso de las fotografías, revisa si fue solicitada por un usuario y lo utiliza como destino, si la fotografía se corresponde a alguien solicitando el acceso desde fuera, indica que se transmita a todos los usuarios. El payload de la foto es un `JSON` que indica el evento (si la foto fue solicitada por un usuario o “espontánea”, es decir por solicitud de acceso de un visitante), el destinatario y la ruta absoluta de la fotografía.

Otro flujo simple que se implementó es la carga histórica del registro de accesos. Mediante un nodo de inyección programado para dispararse diariamente a cierta hora, se envía un email con el archivo csv de registros que contiene los correspondientes a ese día. El flujo utiliza un nodo de archivo para saber dónde “buscar” el registro, luego un nodo de función para formatear el mensaje y finalmente un nodo e-mail, el cual está configurado con la dirección de destinatario así como con una clave de aplicación, en este caso de google para trabajar con Gmail.

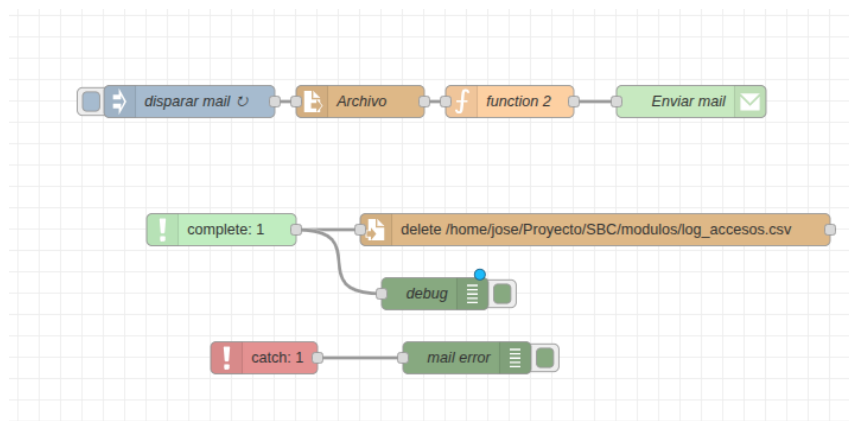


Figura 11: Flujo de automatización de envío periódico de registros de acceso.

El nodo `complete` se asocia a otros nodos y se dispara cada vez que el objetivo realiza su función, al apuntarlo al nodo de e-mail, sabe cuando el correo se envió exitosamente y comanda un nodo de archivo para que elimine el registro viejo, así cada archivo solo representa al día pasado. En el caso de error, por ejemplo que a la hora de enviar el correo el servidor pierda la conexión a Internet, no se borra el archivo y sigue acumulando registros hasta que se envíe.

Hay un tercer flujo desactivado que permite inyectar tramas por el bus RS-485. Utiliza nodos de inyección manual conectados a un nodo de función que calcula y añade el CRC y envía la trama a un nodo serial. También incluye un nodo serial de entrada conectado a un nodo de debug para observar las respuestas. Este flujo se utilizó en el desarrollo para probar el protocolo y se mantuvo para pruebas. Si bien se tuvo que desactivar para no generar conflictos con el uso del puerto serie del servidor, que ahora utiliza el programa principal, se mantiene como una herramienta de debug y pruebas de actualizaciones en el futuro.

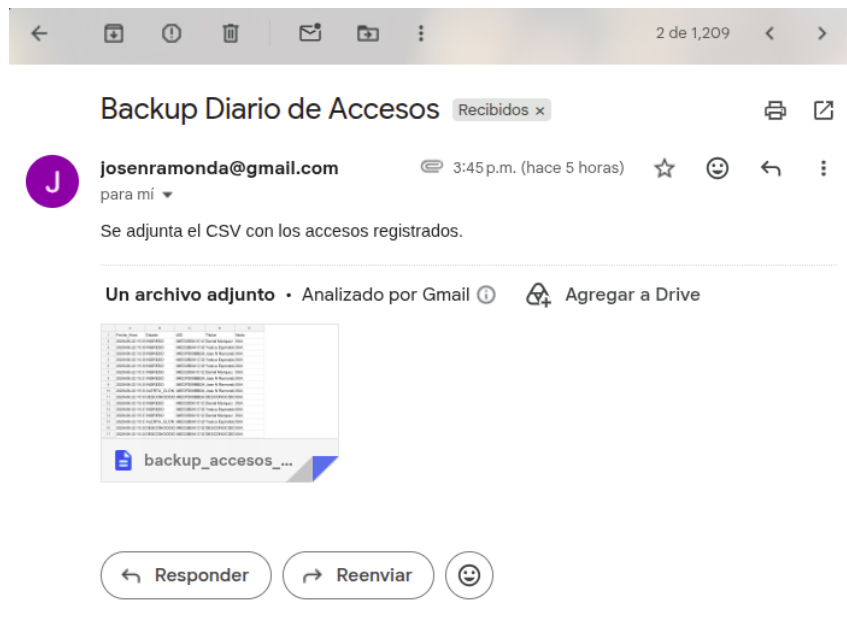


Figura 12: Correo electrónico con registro histórico enviado desde Node-Red.

6.3. Interfaz de Usuario

La interacción principal con el sistema de control de acceso se centraliza a través de un *bot* en la aplicación de mensajería instantánea Telegram. Esta plataforma fue seleccionada frente a alternativas populares como WhatsApp debido a que ofrece una API [10] gratuita, abierta, ampliamente documentada y accesible. En contraste, la API de WhatsApp requiere un registro empresarial, es de pago y presenta un entorno más cerrado. Adicionalmente, esta decisión responde al requerimiento explícito del cliente de mantener las funcionalidades de control de acceso separadas de sus comunicaciones cotidianas, evitando que los comandos y notificaciones de seguridad se pierdan entre otras conversaciones.

El *bot* permite automatizar respuestas en formato de chat e implementar comandos e interfaces gráficas basadas en menús. La aplicación fue desarrollada en Python y se ejecuta como un proceso completamente independiente. Esta decisión arquitectónica permite desacoplar la interfaz del resto de las tareas del sistema, aislando posibles puntos de fallo.

Asimismo, la biblioteca `Python-telegram-bot` requiere el uso nativo del módulo `asyncio` para el manejo de tareas asíncronas. Dado que este módulo implementa un esquema de concurrencia colaborativa, su convivencia con el módulo `threading` tradicional dentro del mismo proceso resulta compleja y propensa a colisiones, lo que refuerza la necesidad técnica de aislar el *bot* en su propio espacio de ejecución.

6.3.1. Arquitectura y Máquina de Estados

El núcleo de la interfaz se basa en una Máquina de Estados Finitos (MEF) implementada mediante un `ConversationHandler`. Este diseño previene comportamientos erráticos al restringir las acciones del usuario según su estado actual en el sistema. Los estados definidos son:

- **ESPERANDO_USER** y **ESPERANDO_CLAVE**: Flujo inicial de autenticación.

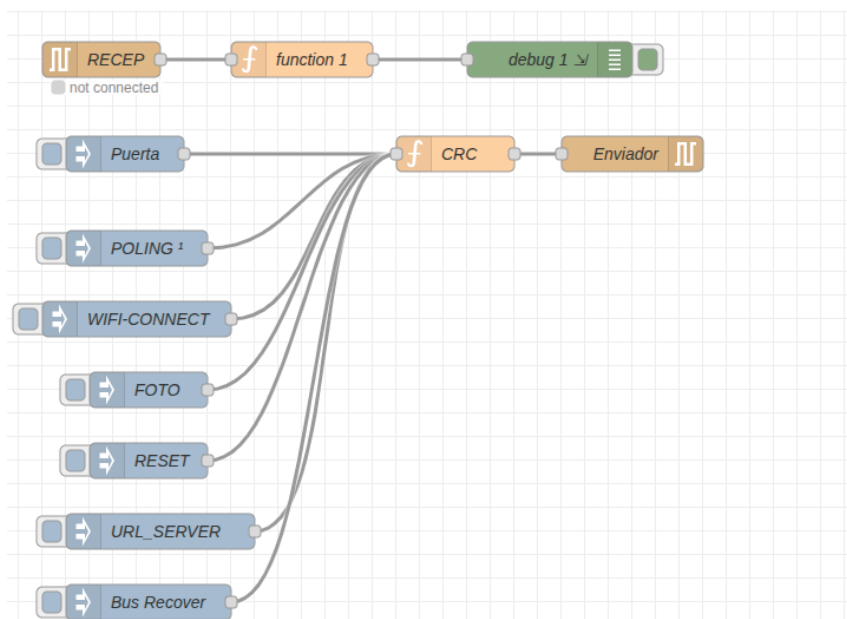


Figura 13: Flujo de inyección de comandos Ramodbus al bus serial.

- **EN_MENU_USUARIO:** Estado de reposo donde el sistema aguarda interacciones a través de los teclados en línea (botones).
- **ESPERANDO_CLAVE_ADMIN:** Estado transitorio para la elevación de privilegios.

Cualquier entrada de texto no reconocida mientras el sistema se encuentra en estado de menú es capturada e ignorada de forma segura para no romper el flujo de la conversación.

6.3.2. Módulo de Autenticación y Roles

La seguridad de la interfaz se gestiona de forma local a través del archivo `credenciales.json`. El sistema clasifica a los individuos en tres categorías: *Invitados* (sin acceso), *Usuarios Estándar* (pueden operar puertas y cámaras) y *Administradores* (pueden auditar el sistema y gestionar el hardware). Al ingresar exitosamente, el sistema vincula de forma permanente el `chat_id` numérico de Telegram con el perfil del usuario. Esto es fundamental para el posterior enrutamiento de notificaciones asíncronas y alertas de seguridad específicas a cada individuo.

6.3.3. Navegación y Menús Interactivos

Para minimizar los errores de tipeo, la operación se realiza íntegramente mediante teclados en línea (`InlineKeyboardMarkup`). Los menús son dinámicos; por ejemplo, la selección de hardware lee directamente las direcciones físicas reales del bus de los nodos (como `0x0A` para el Acceso Frente) desde la configuración (`botconfig.py`) y las embebe en la carga útil (callback) de cada botón.

El Administrador cuenta con un panel avanzado que le permite:

- Listar el estado y privilegios de todos los usuarios registrados.
- Activar o detener el *Modo Programación* directamente en el hardware de los nodos remotos.
- Descargar y actualizar la base de datos física de accesos.



(a) Validación de acceso y menú de usuario.



(b) Gestión del padrón.

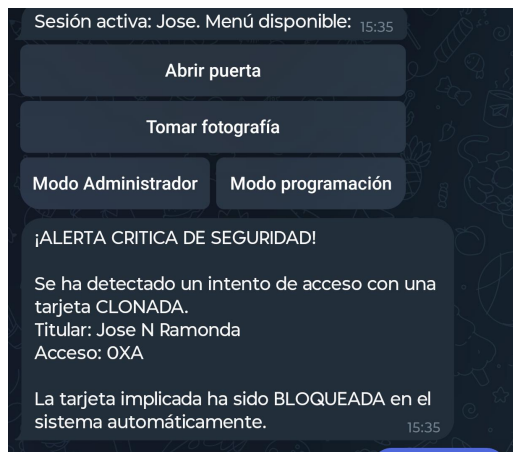
Figura 14: Interfaz de Telegram mostrando la operación normal del sistema.

6.3.4. Integración MQTT y Notificaciones Asíncronas

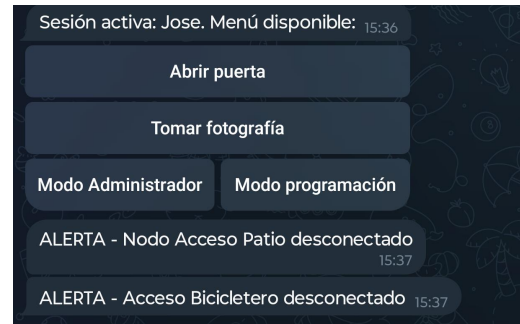
El módulo `MQTT.py` gestiona dos vías de comunicación simultáneas para comunicar este proceso con el resto del sistema:

1. **Emisión de comandos (`sbc/cmd/`):** Ejecuciones síncronas gatilladas por los botones del bot (aperturas, toma de fotografías, inicio de modo programación).
2. **Escucha de notificaciones (`sbc/notify`):** Un hilo secundario que se mantiene suscrito al bróker para atajar alertas provenientes de Node-RED o del hardware.

Este hilo secundario cuenta con un bucle de eventos propio (`asyncio`) que le permite despachar contenido multimedia a los usuarios sin bloquear la ejecución principal de la interfaz. Esto incluye capturas fotográficas de control, alertas tempranas por pulsación de timbres físicos y notificaciones críticas de seguridad (por ejemplo,



(a) Alarma de clonación de tags.



(b) Alarma de desconexión de nodos.

Figura 15: Mensajes de alarma configurados en el bot.

el bloqueo automático ante el escaneo de una tarjeta clonada).

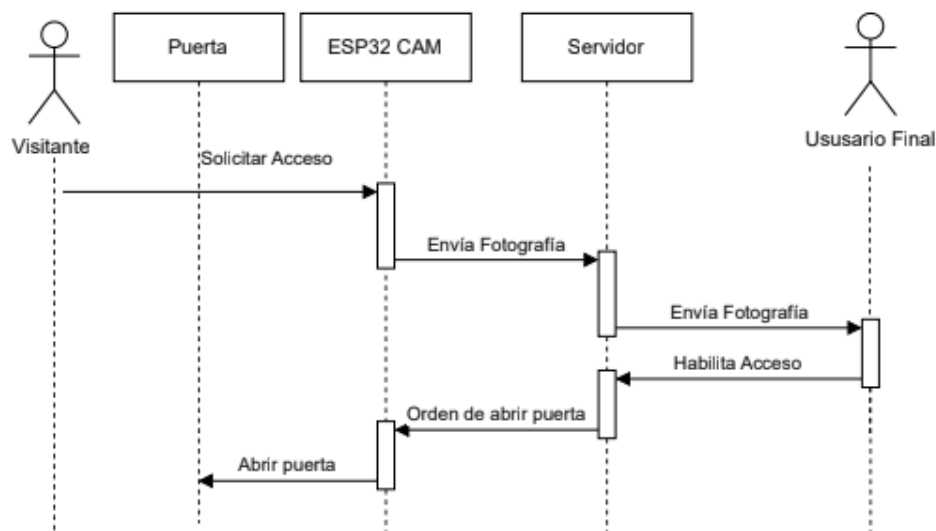


Figura 16: Diagrama UML de secuencia para la solicitud de acceso mediante fotografía.

6.3.5. Gestión Dinámica del Padrón de Accesos

El sistema resuelve la actualización de credenciales NFC de forma intuitiva. El administrador puede solicitar la descarga del archivo `accesos_autorizados.csv` directamente desde el chat. Una vez editado externamente, la actualización en el servidor SBC se logra adjuntando el nuevo documento a la conversación. El bot ataja el archivo entrante, valida su nomenclatura, lo descarga reemplazando el padrón anterior en el disco y publica un comando MQTT (`actualiza`) para que el hardware refresque sus listas locales de permisos.

6.4. Selección del hardware del servidor y migración a Linux embebido

Para el despliegue del servidor central y sus pruebas de validación, se migró el entorno de ejecución desde la computadora de desarrollo hacia una computadora monoplaca (*Single Board Computer*). Si bien las etapas preliminares de integración se llevaron a cabo sobre una *Raspberry Pi 3B+*, la elección final de producción recayó en la *Raspberry Pi Zero 2 W*.

Esta decisión responde a un criterio deliberado de optimización y dimensionamiento mínimo de hardware:

- **Costo y disponibilidad:** Es la opción más económica dentro de la familia Raspberry Pi con arquitectura de 64 bits, reduciendo el costo unitario del servidor a menos de la mitad respecto a otros modelos estandar.
- **Eficiencia energética y térmica:** Presenta el menor consumo de potencia (típicamente inferior a 1,5 W en carga moderada) y una baja disipación térmica, lo que permite su confinamiento continuo dentro de gabinetes cerrados sin requerir ventilación activa.
- **Garantía de interoperabilidad hacia arriba:** Al comprobarse experimentalmente que la placa más restrictiva de la familia (con 512 MB de RAM) soporta de forma holgada la carga combinada de los servicios, queda demostrado que cualquier modelo superior de la gama es plenamente compatible sin requerir adaptaciones de software.

Al tratarse de un nodo dedicado exclusivamente a tareas de control en segundo plano, se instaló la distribución *Raspberry Pi OS Lite* (sin entorno gráfico de escritorio), maximizando el espacio de memoria RAM disponible para los procesos del sistema. La migración del código base desde Ubuntu resultó directa gracias a la compatibilidad del entorno Debian, implementándose las siguientes adaptaciones de bajo nivel:

- **Control del bus RS-485 sin latencia de conmutación:** Se evitó el uso de módulos basados en el circuito integrado MAX485, los cuales delegan el control de flujo (transmisión/recepción mediante los pines DE/ $\bar{R}E$) a una línea GPIO controlada por software. Dado que el núcleo del sistema operativo Linux no opera en tiempo real estricto, la latencia en la conmutación por software generaba colisiones al recibir las respuestas inmediatas del microcontrolador. En su lugar, se adoptó un módulo transceptor con autodirección de flujo por hardware.
- **Habilitación de la UART principal nativa:** Se utilizaron los pines físicos de expansión (GPIO14/TXD y GPIO15/RXD) vinculados al periférico UART nativo (PL011). Mediante ajustes en el archivo de configuración del gestor de arranque (`config.txt`), se deshabilitó la consola serie del sistema operativo y se independizó el generador de reloj de la UART de la frecuencia dinámica de la GPU, asegurando una tasa de baudios estable frente a variaciones de carga del procesador.

- **Gestión de privilegios y seguridad:** Con el fin de evitar la ejecución de procesos bajo la cuenta de superusuario (`root`), el usuario estándar del sistema fue incorporado a los grupos `dialout` (para el acceso al puerto serie `/dev/ttyAMA0`) y `gpio`.
- **Aislamiento del entorno Python:** En cumplimiento de la directriz PEP 668 para distribuciones basadas en Debian moderno, las dependencias del proyecto se instalaron dentro de un entorno virtual (`venv`) aislado del árbol de paquetes del sistema.
- **Orquestación de servicios mediante `systemd`:** La lógica de control del bus (`main.py`) y la pasarela del bot de Telegram (`bot.py`) se desacoplaron como dos servicios independientes de segundo plano gestionados por `systemd`, configurados con reinicio automático ante excepciones para garantizar la disponibilidad continua del acceso.

7. Seguridad

Un sistema de control de acceso es un tipo de sistema de seguridad y como tal debe estar protegido. Su objetivo es restringir el ingreso a personas autorizadas y aumentar el nivel de protección de las instalaciones mediante mecanismos de identificación y validación de usuarios.

No obstante, es importante señalar que ningún sistema de seguridad puede considerarse completamente invulnerable. En la práctica, las medidas de seguridad no buscan eliminar por completo la posibilidad de un ataque o una intrusión, sino reducir su probabilidad de éxito y aumentar la dificultad, el tiempo y los recursos necesarios para llevarlo a cabo.

El nivel de seguridad implementado debe ser coherente con el contexto de uso, la probabilidad de ocurrencia de amenazas y las limitaciones técnicas y físicas existentes. En este caso, el sistema está destinado al control de acceso de una institución educativa, cuyas necesidades de seguridad difieren de las de entornos críticos como entidades bancarias o instalaciones de alta seguridad.

Asimismo, la eficacia del sistema se encuentra condicionada por características físicas de la infraestructura. Las puertas sobre las que actúa el sistema incorporan un destrabador eléctrico, pero no cuentan con mecanismos de refuerzo estructural adicionales. Por lo tanto, aunque el sistema puede impedir el acceso normal a personas no autorizadas, no puede evitar métodos de ingreso forzado que excedan las capacidades del mecanismo de cierre. En consecuencia, la solución propuesta debe entenderse como una mejora significativa en el control y la gestión de accesos, dentro de las limitaciones físicas y operativas del entorno donde será implementada.

7.1. Posibles ataques y vulnerabilidades

La principal vulnerabilidad del sistema es la física y abarca principalmente dos posibles ataques. Primero el ingreso forzado, en este caso el sistema en sí no representa una barrera física sino administrativa. No es lo mismo a nivel legal que una persona no autorizada simplemente abra la puerta y acceda a que ingrese mediante el uso de la violencia. La existencia de la medida de seguridad define la falta que constituye

vulnerarla. El otro caso es el de la afectación física del sistema, por ejemplo la desconexión del servidor, el sabotaje del cableado del bus serial, etc. Estos eventos, si bien no permiten el acceso no autorizado como tal, anulan total o parcialmente el funcionamiento del sistema.

Por fuera del sabotaje físico, si un atacante deseara poder acceder a voluntad pasando desapercibido, hay dos caminos posibles:

- Clonar un tag NFC válido.
- Tomar control del sistema en alguna de sus instancias inyectando comandos de apertura.

7.2. Sistema anticlonación

La familia de normas ISO/IEC 18000 establece los requisitos y especificaciones para los sistemas de identificación por radiofrecuencia (RFID) en diversas bandas del espectro. De manera general, un sistema RFID se compone de tres elementos fundamentales: el *tag* o transpondedor que contiene los datos de identificación, el lector que emite un campo electromagnético para energizar e interrogar a la etiqueta, y el sistema o software de gestión encargado de procesar la información obtenida.

Si bien numerosos sistemas tradicionales de control de acceso operan en la banda de baja frecuencia (LF, típicamente a 125 kHz), la disponibilidad, el costo de los módulos de lectura y la facilidad de integración comercial favorecen el uso de la banda de alta frecuencia (HF, 13,56 MHz), estandarizada bajo ISO/IEC 18000-3. Con el objetivo de priorizar la viabilidad económica, el bajo costo unitario de los transpondedores y la rápida reposición por parte de la institución, se optó por operar en dicha frecuencia. No obstante, la amplia difusión y accesibilidad de las herramientas para 13.56 MHz introducen desafíos de seguridad específicos.

En los sistemas elementales de identificación, la primera barrera de autenticación reside en el Identificador Único (“UID”) grabado de fábrica en la memoria de solo lectura del circuito integrado. Aunque existen en el mercado etiquetas con UID reescribible (comúnmente denominadas *Magic Cards* o CUID), la principal vulnerabilidad radica en la **emulación activa**. Mediante hardware comercial de bajo costo —como el propio transceptor PN532 operando en modo *Target* o cualquier teléfono inteligente equipado con NFC y herramientas de emulación de tarjetas (HCE)—, un atacante puede replicar el UID capturado previamente y responder a los comandos del lector de forma idéntica a una credencial legítima.

Dado que el UID se transmite en texto plano durante la fase de inicialización y anticolisión, constituye información pública para cualquier dispositivo que logre proximidad física con el *tag*. Por este motivo, resulta indispensable migrar de un esquema de identificación pasivo basado en UID hacia el estándar de proximidad **ISO/IEC 14443 Tipo A** (base de la tecnología NFC), aprovechando la memoria protegida del transpondedor.

Dentro de las alternativas disponibles bajo este estándar, las credenciales *Mifare Classic* quedaron descartadas debido a las vulnerabilidades estructurales demostradas en su algoritmo propietario *Crypto-1*. En su lugar, se seleccionaron etiquetas de la familia **NTAG21x** (específicamente NTAG213/215/216, clasificadas como NFC Forum Type 2 Tag). Estas permiten la protección de su contenido mediante una contraseña de 32 bits. Para que el lector establezca una sesión activa con el tag, debe autenticarse mediante la contraseña grabada en la memoria del chip, ante lo cual este

responde con un código de reconocimiento (*Password Acknowledge* o PACK). Ambos valores se derivan de forma dinámica mediante un algoritmo **HMAC-SHA-256**: este combina el UID del *tag* con una función *hash* criptográfica (SHA-256) y una clave secreta almacenada exclusivamente en el firmware del nodo, garantizando que dicha clave no quede expuesta en la memoria ni en las transmisiones con el servidor central.

El algoritmo **HMAC-SHA-256** devuelve siempre 32 bytes, por los cuales se trunca el resultado tomando seis bytes consecutivos, comenzando en determinada posición para incorporar una capa de seguridad ante una filtración de la clave secreta.

En este punto obtener los datos para emular un tag completo resulta considerablemente difícil y requeriría de conocimientos y equipo específico, un método podría ser acoplar al sistema una antena que intercepte todo el intercambio, luego estudiando el protocolo descifrar todas las etapas de la comunicación para finalmente emular la uid y transmitir en el momento apropiado el PACK y suplantar a la credencial original. Ante este improbable pero posible caso, se incorporó una medida adicional, que si bien no es preventiva puede detectar una eventual clonación.

Las NTAG cuentan con un contador por hardware, que al activarse aumenta el valor en un registro de tres bytes. Esto solo cuenta lecturas de registros y solo funciona luego de la energización del tag, por lo que si se deja la etiqueta sobre el lector indefinidamente, el contador no aumenta. En la programación de cada tag se incluye un registro al cual el contador se “espeja”, con lo cual el terminal al detectar un tag realiza la siguiente secuencia:

- Extrae el UID
- Calcula el hash
- Se identifica con el tag mediante la contraseña
- Valida el PACK devuelto
- Lee el registro del contador
- Envía al servidor los datos UID + contador

Con esto se tiene un primer filtro local en el nodo, que identifica si el tag fue programado para este sistema. Luego el servidor no solo valida que el UID pertenezca a las autorizadas, sino que revisa el valor del contador. En el caso más desfavorable, si un tag clonado se utiliza para acceder y aumenta su contador indefinidamente, al ingresar el titular con el tag original, el contador leído será menor al registrado detectándose la clonación. Ante este evento, el UID se bloquea y se notifica a los usuarios finales.

7.3. Protección de los datos del sistema

La otra forma de vulnerar la seguridad del sistema es mediante el acceso a este en cualquiera de sus tramos. A continuación se describen las consideraciones de seguridad de cada uno.

7.3.1. Acceso físico a los terminales

El acceso físico no autorizado a los terminales expone al sistema a vulnerabilidades elementales, como la apertura forzada de la cerradura electromecánica mediante un simple cortocircuito en los terminales del relé de accionamiento.

Más allá del ataque directo sobre los actuadores físicos, existe el riesgo de alteración lógica mediante la instalación de un firmware malicioso en los microcontroladores. Para mitigar este vector de ataque a través de la interfaz de diagnóstico, el diseño del prototipo expone únicamente el pin de transmisión (**Tx**) de la UART0. Al omitirse deliberadamente la conexión física del pin de recepción (**Rx**), se imposibilita la reprogramación directa del equipo a través de dicho puerto.

No obstante, al emplear placas de desarrollo genéricas (módulos ESP32) para la etapa de prototipado, persiste el riesgo de que un atacante con acceso al interior de la carcasa extraiga el módulo completo para reprogramarlo externamente, o bien lo reemplace por uno manipulado. En un entorno de producción, esta vulnerabilidad se mitiga integrando el microcontrolador soldado directamente sobre una placa de circuito impreso (PCB) diseñada a medida, lo cual dificulta enormemente su extracción y reemplazo rápido. El diseño y ruteo de dicha PCB definitiva se establece como trabajo futuro.

7.3.2. Red local de la institución

Los datos que viajan a través del WiFi están cifrados. La única información que se transmite por este medio son las fotografías capturadas por los terminales, la cual no es considerada información sensible por lo que no se tomaron medidas de seguridad adicionales para este medio.

Otro riesgo podría ser la inyección de comandos mediante un cliente MQTT. Por eso se mantiene el broker del servidor escuchando únicamente mensajes locales del equipo.

Si en el futuro se quisiera adicionar otro sistema IoT que utilice MQTT en la red local, deberán tomarse las precauciones adecuadas agregando autenticación de parte de los clientes.

Adicionalmente, en caso de desplegar el sistema sobre una red cableada (Ethernet), la protección contra intrusos físicos requeriría implementar protocolos de autenticación de puertos como IEEE 802.1X. Sin embargo, dado que en este proyecto se optó por utilizar una *Raspberry Pi Zero 2 W*, la cual carece nativamente de un puerto Ethernet RJ45—, el servidor queda inherentemente exento de ataques mediante conexión física directa a la placa, limitando el perímetro de exposición de red exclusivamente a la interfaz Wi-Fi.

7.3.3. Bus serial RS-485

Al acceder al canal serial se podrían interceptar e inyectar tramas que se camuflen como mensajes legítimos tanto del servidor como de los terminales, pudiendose inyectar un comando de apertura. La protección del bus es física, ya que los cables deben ser pasados por cañerías o instalarse interiormente a las paredes.

Si bien existen opciones para reforzar esta seguridad como por ejemplo el cifrado de las tramas, estas requieren un procesamiento adicional que compromete la respuesta temporal o hardware adicional que aumenta el costo. No se consideró que esta mejora lo justifique.

7.3.4. Acceso al servidor

El servidor central constituye el punto más crítico en la seguridad arquitectónica del sistema. Un acceso no autorizado a este componente podría derivar en la interrupción del servicio, el robo de credenciales, la alteración del padrón de usuarios o la apertura fraudulenta de los accesos físicos mediante la inyección de cargas útiles en los tópicos MQTT correspondientes.

Para mitigar los vectores de ataque por acceso físico directo, la administración del sistema operativo se restringe exclusivamente a conexiones remotas mediante el protocolo SSH. El entorno local exige autenticación de credenciales para cualquier interacción con la consola, bloqueando intentos de inicio de sesión no autorizados en caso de que se conecten periféricos de entrada y salida (teclado y monitor) directamente a la placa. Como medida adicional complementaria para despliegues sobre computadoras monoplaca (SBC), es posible deshabilitar los puertos USB y las salidas de video (HDMI) a nivel del kernel.

Por otro lado, aunque el motor de Node-RED se ejecuta localmente, su editor gráfico es accesible vía HTTP desde cualquier dispositivo dentro de la red de área local. Dejar esta interfaz expuesta conlleva riesgos críticos, tales como la alteración de la lógica de control, la interceptación de información confidencial o la inyección de comandos a nivel del sistema operativo. Para neutralizar esta vulnerabilidad, se habilitó el módulo de autenticación nativo (`adminAuth`) de Node-RED. La configuración se realiza generando un hash criptográfico con la herramienta `node-red-admin hash-pw` e inyectándolo en el archivo `~/.node-red/settings.js`. Tras esta configuración, cualquier intento de acceso al editor web es interceptado por un portal cautivo que exige el ingreso de credenciales válidas.

Finalmente, respecto a los datos en reposo, existe el riesgo de extracción de la tarjeta MicroSD y su posterior lectura en un equipo externo. Para proteger la información sensible, es necesario implementar técnicas de cifrado sobre el almacenamiento persistente. En este aspecto, es fundamental diferenciar el tratamiento de las credenciales: las contraseñas de acceso de los usuarios no deben guardarse en texto plano, sino mediante el uso de hashes criptográficos unidireccionales, mientras que los secretos operativos, como el token de la API de Telegram, requieren métodos de cifrado reversible o bóvedas seguras, ya que el sistema necesita recuperarlos íntegramente para operar. Dado que la implementación de estas medidas de seguridad en el sistema de archivos añade una complejidad técnica considerable al proceso de arranque desatendido del hardware embebido, estas optimizaciones exceden el alcance del prototipo actual y han sido delimitadas como líneas de trabajo futuro.

7.3.5. Interfaz de usuario

El tráfico del servidor que llega a Internet pasa por los servidores de Telegram, los cuales se consideran seguros, al ser una aplicación orientada a la privacidad. En este punto el mayor riesgo es humano, una filtración del link del bot, acompañado por un robo de credenciales, por ejemplo por un ataque de *phishing* a un usuario autorizado, puede llegar a darle acceso al bot a un atacante. En este caso la prevención es clave, la cual se logra mediante la capacitación de los usuarios.

7.4. Interfaz serial inalámbrica: consideraciones extra

En el análisis de soluciones se optó por utilizar RS-485 como capa física de la vía de comunicación principal entre el servidor y los terminales. Otra de las opciones consideradas fue utilizar una comunicación inalámbrica mediante módulos RF o LoRa. Si bien se optó por el bus cableado debido a su menor costo, queda abierta la posibilidad de utilizar una solución inalámbrica en instalaciones donde el costo y la complejidad del tendido de cables sean comparables o superiores al de los módulos de comunicación.

En el caso de utilizar un enlace inalámbrico, deberían incorporarse medidas de seguridad adicionales. Uno de los nuevos riesgos que aparecen es la **intercepción de datos**. Mientras que en el enlace cableado es necesario obtener acceso físico al bus para capturar las tramas, en una comunicación inalámbrica estas son transmitidas por el aire y pueden ser recibidas por terceros con el equipamiento adecuado. Además, un atacante que conozca el protocolo podría intentar generar e inyectar sus propias tramas. Por este motivo, además de la identificación de los terminales, sería necesario incorporar mecanismos de **cifrado y autenticación criptográfica** de las comunicaciones.

También debe evitarse la reproducción de tramas legítimas previamente capturadas. Para esto puede incorporarse un contador de mensajes, un *rolling code* u otro mecanismo *anti-replay*, de modo que una trama válida no pueda ser reutilizada indefinidamente.

Otro riesgo es la interrupción del funcionamiento mediante un ataque de *flooding*, consistente en transmitir una gran cantidad de mensajes con el objetivo de saturar los recursos del receptor o del canal de comunicación. En este caso resulta importante realizar el descarte de mensajes inválidos lo antes posible, además de incorporar mecanismos como límites de tasa, límites de buffers y control de retransmisiones. Esto podría requerir modificaciones respecto del protocolo utilizado sobre RS-485.

Otro ataque posible es el *jamm*ing, que consiste en generar interferencia deliberada sobre la frecuencia o banda utilizada por el sistema, impidiendo que el receptor pueda recibir correctamente las transmisiones legítimas. A diferencia del *flooding*, este ataque no requiere conocer el protocolo de aplicación, ya que actúa directamente sobre el medio físico. Su mitigación puede requerir técnicas como el cambio de frecuencia o canal, reintentos de transmisión y, en sistemas con mayores requerimientos de disponibilidad, medios de comunicación alternativos.

En ambos casos, los ataques pueden **mitigarse, pero no eliminarse completamente** si el atacante dispone de equipamiento y potencia de transmisión suficientes. Por este motivo, además de las medidas de seguridad propias del protocolo, el sistema debería diseñarse considerando que pueden producirse pérdidas frecuentes de conectividad. En particular, deberían reforzarse los mecanismos de detección de fallos, desconexión y reconexión del canal, así como definirse el comportamiento de los terminales durante períodos sin comunicación con el servidor.

8. Ensayos

Para validar el correcto funcionamiento del sistema se desarrollaron una serie de pruebas de banco para corroborar la funcionalidad y robustez del sistema.

Si bien las funcionalidades por separado se fueron probando en el desarrollo, esta sección remite a pruebas realizadas por el sistema en su conjunto.

Se ensambló el sistema con el servidor alojado en la *Raspberry Pi Zero 2 W* conectado a un solo terminal funcional.

Para probar la conexión por RS-485 se posicionó el sistema en un escenario de peor caso. Si bien en la institución la distancia máxima de un nodo al servidor es del orden de los 40 m, las pruebas se realizaron con un cable de 100 m de dos pares trenzados, sin blindaje y totalmente enrollado, empeorando las condiciones de resistividad, ruido y compatibilidad electromagnética.

8.1. Pruebas de funcionalidad y robustez

Para las primeras pruebas se configuró el bus serial a 9600 baudios y el intervalo entre ciclos de polling en el orden de los 500 ms.

Se probaron en conjunto las funcionalidades de

- Conexión serial y respuestas a las encuestas periódicas.
- Ejecución del bot y disponibilidad de la interfaz de usuario.
- Inicio de sesión de diferentes roles.
- Accesos mediante tags NFC autorizados.
- Aperturas comandadas desde la interfaz de usuario.
- Envío de fotografías bajo demanda solicitadas por la interfaz de usuario.
- Envío de fotografías ante interrupciones generadas por el pulsador del timbre.
- Actualización del padrón de accesos mediante la interfaz de usuario.
- Envío por correo electrónico de los registros históricos diarios.
- Alarmas de desconexión de nodos del bus RS-485.
- Monitoreo periódico de conectividad del terminal a la red local, junto con su reconexión automática ante una desconexión forzada.

Los resultados fueron satisfactorios, el sistema demostró ser funcional y sólido a nivel de software.

8.2. Ensayos de estrés del canal RS-485

Para evaluar los límites físicos y lógicos de la comunicación, se sometió al bus a pruebas de estrés incrementando la exigencia sobre el canal de transmisión hasta inducir el punto de fallo.

La primera variable ajustada fue la velocidad de modulación (*baud rate*). En el esquema de codificación empleado por las interfaces UART, un baudio equivale a un bit por segundo (bit s^{-1}). Se partió de una tasa base conservadora de 9600 baudios, incrementando progresivamente por los valores estándar definidos para comunicaciones serie (19200, 38400, 57600, 115200, 230400, 460800 y 921600 baudios).

Durante el ensayo, el sistema operó con total estabilidad hasta los 460800 baudios. A 921600 baudios, la comunicación falló por completo. La consola del ESP32 no indicó la recepción de mensajes y el servidor reportó el nodo como desconectado. En base a eso se infiere que la falla se debe a la gestión de interrupciones del sistema operativo en tiempo real (FreeRTOS) del microcontrolador. A dicha tasa, las interrupciones del periférico UART se disparan con una frecuencia tan elevada que el planificador de tareas no logra evacuar el búfer de recepción antes de la llegada de nuevos datos, provocando desbordamientos (*overflows*) en la capa de hardware.

La siguiente variable de ajuste fue el intervalo de tiempo entre ciclos de polling. Es importante recordar que este parámetro representa cuánto demora el servidor luego de recibir una respuesta (o de cumplirse el timeout) para lanzar una nueva encuesta. Este intervalo se redujo a cero, demostrando que el sistema funciona correctamente operando bajo ráfagas continuas de encuestas y respuestas inmediatas, sin requerir tiempos de relajación en el canal.

Para la prueba final de estrés, se configuró el sistema con los parámetros más exigentes (460800 baudios y un intervalo de *polling* nulo). El sistema se dejó operar en este régimen continuo durante un tiempo estimado de 5 min. Ambos extremos (servidor y terminal) se configuraron para registrar en *logs* cualquier pérdida de trama, desconexión o fallo en la validación del CRC. Transcurrido el ensayo, y tras haberse procesado un estimado de más de 100 000 intercambios de tramas, ningún nodo reportó fallas, lo que representó una tasa de éxito del 100 % en la transmisión.

8.3. Ensayo de tiempo de respuesta

Para medir los tiempos de respuesta del sistema se realizaron dos ensayos en tres condiciones de operación.

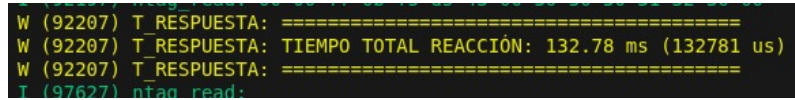
La primera condición es en el límite máximo de operación obtenido del ensayo de estrés a 460800 baudios y con un retardo entre encuestas nulo. La segunda condición reduce la tasa a 115200 baudios y la tercera incorpora un intervalo de polling de 10 ms, con el objetivo de ver cuanto afectan la variación de estos parámetros a la respuesta del sistema.

8.3.1. Medición de tiempo de apertura por acceso autorizado

Para el primer ensayo se modificó el firmware del terminal, declarando la variable global `int64_t ti` y añadiendo la biblioteca `esp_timer`.

En la tarea de lectura de tags NFC, al momento de la detección de un tag se llama a `esp_timer_get_time` para asignarle a `ti` el timestamp exacto en microsegundos. Luego, en la tarea que reacciona al comando de apertura, inmediatamente después

de accionar el relé, se vuelve a tomar el tiempo y se calcula la diferencia con el valor inicial imprimiéndose por pantalla el resultado, correspondiente al tiempo transcurrido desde la detección del tag hasta la apertura, abarcando la validación local del tag, la lectura del contador, el envío de los datos al servidor, la validación en el servidor, la respuesta con el comando de apertura y la activación efectiva del relé.



```

W (92207) T_RESPUESTA: =====
W (92207) T_RESPUESTA: TIEMPO TOTAL REACCIÓN: 132.78 ms (132781 us)
W (92207) T_RESPUESTA: =====
T (97627) ntag_read:

```

Figura 17: Salida del monitor ESP-IDF.

Se tomaron una docena de mediciones para cada caso y los resultados en promedio fueron los siguientes:

Escenario de ensayo	Tiempo promedio de respuesta
115200 bauds (10 ms)	160,64 ms
115200 bauds (0 ms)	134,85 ms
460800 bauds (0 ms)	128,90 ms

Cuadro 3: Tiempos promedio de respuesta del sistema medidos en microcontrolador para los tres escenarios de ensayo.

Este ensayo muestra que la disminución de la velocidad de transmisión aumenta el tiempo de respuesta en el orden de los 6 ms, mientras que, para la misma velocidad, un aumento entre intervalos de ciclos de encuesta de 10 ms añade 26 ms a la respuesta final, siendo este cambio considerablemente más significativo.

8.3.2. Medición de tiempo de fotografías bajo demanda

Para el segundo ensayo, se auditaron de forma combinada los registros del sistema operativo mediante el comando `journalctl -o short-precise` para capturar los eventos de ambos servicios con resolución de microsegundos, solicitando diez capturas por cada configuración.

Los registros se registraron en archivos cuyo contenido posee el siguiente formato:

```

1 Aug 23 13:57:03.400791 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
  cmd/foto | Payload: 0x0A,8816825051
2 Aug 23 13:57:03.402498 ramondapi python[1350]: [ACCION] Encolando pedido de
  fotografia para el nodo 10...
3 Aug 23 13:57:03.402498 ramondapi python[1350]: [LOGICA] Comando 7 encolado para el
  nodo 0xa
4 Aug 23 13:57:03.676382 ramondapi python[1350]: [DEBUG BUS] Encolando evento a la
  entrada evento 7
5 Aug 23 13:57:03.676713 ramondapi python[1350]: [PARSER - CAMARA] El nodo 0xa
  reporta fotografia tomada
6 Aug 23 13:57:03.677586 ramondapi python[1350]: [MQTT PUB SUCCESS] sbc/status/0xa ->
  OK_FOTO
7 Aug 23 13:57:03.682373 ramondapi python[1179]: [BOT-MQTT IN] Evento:
  FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
8 Aug 23 13:57:05.442759 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
  de Telegram.
9 Aug 23 13:57:05.443844 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
  : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg

```

El procesamiento local se inicia cuando el bot despacha la orden vía MQTT interno según se observa en el primer log. Los siguientes dos reportes los hace el proceso maestro tras recibir la petición y comandar el pedido de fotografía al terminal. Unos 274 ms después el terminal reporta la transmisión por HTTP de la imagen y el proceso master lo anuncia al broker. Casi instantáneamente el proceso del bot imprime el aviso de que recibió la ruta de la imagen y en el orden de los 2 s logra transmitirla a los servidores de Telegram.

El tiempo de procesamiento interno del sistema es de 281,6 ms. Se tiene tiempo de envío de las fotografías pero no el de entrada del comando. En pruebas informales el tiempo total cronometrado resulta en el orden de los 4 s, pero este tiempo no es controlable, ya que depende de la conexión a internet y el funcionamiento de los servidores de Telegram.

Promediando los tiempos de respuesta del sistema, para los dos casos sin intervalo de polling, el tiempo de procesamiento interno es de alrededor de 256 ms, con un máximo de 361 ms. En el caso de agregar una distancia temporal entre encuestas, el promedio asciende a unos 273 ms con un valor máximo registrado de 373 ms, un aumento que se corresponde con el retardo añadido a la ejecución.

Los tiempos de transferencia de las fotografías via Telegram rondan en promedio los 1,8 s con una demora máxima de 3,23 s. Sin embargo esto es anecdótico, ya que condiciones externas pueden variar a lo largo del experimento.

Las trazas completas pueden observarse en el anexo 12.2.

8.4. Definición de las condiciones de operación del sistema

Si bien los ensayos desarrollados con un solo terminal son un *worst case scenario* desde el punto de vista de los terminales, ya que obligan al ESP32 a responder de inmediato y de forma consecutiva, el terminal único resulta en un caso favorable para el sistema en conjunto, ya que en la práctica el tiempo de respuesta aumenta proporcionalmente a los n nodos conectados al bus.

El alcance de este proyecto abarca hasta la construcción de un prototipo del sistema con un solo terminal. Al no poder probarse el funcionamiento con los tres terminales conectados, se recurre a una estimación para inferir cuál sería el tiempo de respuesta real del sistema. De forma conservadora y manteniendo el criterio de suponer el peor escenario, consideramos que el tiempo de respuesta se triplicaría, es decir, para las condiciones ensayadas resultaría de entre 387 ms y 483 ms para el acceso autorizado mediante tags NFC. En ambos casos se encuentran por debajo del medio segundo, por lo que ambos resultarían satisfactorios.

Para el caso de las fotografías bajo demanda la estimación es más compleja, ya que el mayor tiempo del sistema es el de la transmisión HTTP local. Sin embargo, dado que los tiempos de intercambio de información con los servidores de Telegram son, aunque variables, en general mucho mayores, esta demora enmascara la de la respuesta del sistema.

En general el sistema puede funcionar según sus requerimientos con una tasa de 115200 baudios. Se opta por reducir el intervalo entre encuestas a 2 ms para reducir el impacto en el tiempo de respuesta al agregar más nodos.

8.5. Desempeño del servidor embebido

Como verificación final, con el sistema operando en las condiciones mencionadas se verificó la disponibilidad de recursos del servidor mediante utilidades estándar `free -h` y `top` en la placa de producción:

1		total	used	free	shared	buff/cache	available
2	Mem:	415Mi	272Mi	97Mi	556Ki	97Mi	142Mi
3	Swap:	414Mi	84Mi	330Mi			

De los 415 MiB de memoria física direccionable por el sistema operativo, el entorno en régimen de operación continuo ocupa de forma estable 272 MiB (65,5 %). Esto preserva un margen disponible de 142 MiB (34,5 %).

En cuanto a la capacidad de cómputo, la inspección del procesador arrojó los siguientes valores:

1	%Cpu(s): 25.5 us, 3.6 sy, 0.0 ni, 70.9 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st									
2	PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM
3	1205	jose	20	0	329504	23556	11296	S	91.7	5.5
									0:34.04	python

La carga de trabajo combinada de los cuatro núcleos ARM Cortex-A53 alcanza apenas un 29,1 % (suma del 25,5 % en espacio de usuario y 3,6 % del sistema), conservando un estado de inactividad (*idle*) del 70,9 %. Aunque el proceso principal responsable del bus RS-485 (PID 1205) registra un uso del 91,7 %, este valor es relativo a un único núcleo. Los tres núcleos restantes operan con un amplio margen para resolver simultáneamente la pila de red, el entorno de Node-RED y el encriptado de las comunicaciones del *bot* de Telegram.

Es importante destacar que el elevado consumo relativo a un único núcleo por parte del gestor serie es un comportamiento esperado y acotado. La comunicación serial es continua, con intervalos de sondeo casi nulos, a lo que se suma el procesamiento en tiempo real de las tramas entrantes. El hecho de que toda esta carga no se distribuya, sino que se asigne enteramente a un único núcleo del microprocesador, es una consecuencia directa del *Global Interpreter Lock* (GIL) de Python. Este mecanismo arquitectónico del lenguaje impide que los distintos hilos de ejecución de un mismo proceso operen simultáneamente en múltiples núcleos.

Por consiguiente, la escalabilidad a tres o más nodos no representa un aumento significativo de la carga computacional. Como el bus es compartido, el servidor realiza la misma cantidad de encuestas por segundo y procesa los datos iterando de igual manera, distribuyendo el tiempo entre los distintos terminales de forma secuencial. Si bien la cantidad de eventos válidos procesados (como lecturas de tarjetas o pulsaciones de timbre) aumentará al sumar más nodos, estos sucesos resultan esporádicos en comparación con la alta velocidad del ciclo de polling, por lo que no generan un impacto mensurable en la ocupación global del servidor.

Estos resultados demuestran de manera empírica que la *Raspberry Pi Zero 2 W* satisface con solvencia los requerimientos del servidor local, conformando una solución robusta que optimiza los costos de *hardware*, el consumo energético y el espacio de instalación.

9. Conclusiones

El presente trabajo final de ingeniería concluyó con el diseño, desarrollo, implementación física y validación integral de un sistema modular de control de acceso institucional. La totalidad de los requerimientos funcionales, de seguridad y de escalabilidad planteados en el alcance inicial fueron alcanzados satisfactoriamente.

A partir de los resultados obtenidos en las distintas fases de desarrollo y en los ensayos de banco, se desprenden las siguientes conclusiones técnicas:

- **Robustez del enlace físico y protocolo a medida:** El protocolo desarrollado sobre el estándar RS-485 demostró una fiabilidad del 100 % en condiciones de ensayo severas (cableado no blindado de 100 m en bobina a $460\,800\text{ bit s}^{-1}$). La caracterización temporal evidenció que la reducción a una tasa de trabajo conservadora de $115\,200\text{ bit s}^{-1}$ incrementa el tiempo de tránsito en apenas 6 ms, garantizando máxima inmunidad al ruido electromagnético sin penalizar la respuesta del sistema.
- **Determinismo local y desacoplamiento de latencias:** La arquitectura híbrida (procesamiento distribuido en microcontrolador y servidor Linux) cumplió el objetivo de mantener los tiempos críticos en tiempo real. La autenticación local mediante credenciales NFC seguras y el accionamiento de las cerraduras se resuelven de forma fiable y con un tiempo de respuesta que mejora ampliamente el requerido en la definición del alcance.
- **Seguridad criptográfica en credenciales físicas:** Se mitigó de forma efectiva la vulnerabilidad de clonación inherente a los sistemas de control de acceso comerciales basados en lectura estática de UID, implementando autenticación mutua y llevando una actualización dinámica de registros de uso de las credenciales.
- **Optimización de costos y recursos de hardware:** Los ensayos de consumo demostraron que la pila completa de servicios del servidor opera de forma estable utilizando únicamente el 65 % de la memoria RAM y un 29 % de uso de CPU sobre una plataforma de bajo costo y factor de forma reducido (*Raspberry Pi Zero 2 W*). Esto valida que el sistema no requiere hardware de cómputo sobredimensionado para su funcionamiento continuo en entornos embebidos e industriales.

Se concluye que el sistema cumple con todos los requerimientos establecidos. El código fuente del proyecto está disponible en un repositorio público [11] bajo un enfoque de código abierto.

10. Trabajo futuro

El prototipo desarrollado demostró un desempeño altamente satisfactorio durante las pruebas de banco. En consecuencia, la etapa subsiguiente consiste en la instalación en campo para someter al sistema a condiciones de operación reales y continuas.

Para consolidar el estado actual del proyecto en una versión desplegable, se identifican las siguientes tareas inmediatas de optimización:

- **Robustez de red:** Fortalecer las rutinas de monitoreo de conectividad, implementando lógicas de reconexión más resilientes ante microcortes o saturación de la red local.
- **Gestión de archivos multimedia:** Implementar la clasificación dinámica de las fotografías recibidas en el servidor, utilizando la dirección IP o el identificador del nodo para nombrar y organizar unívocamente los archivos según su terminal de origen.
- **Interfaz y Experiencia de Usuario (UI/UX):** Optimizar el *front-end* para ofrecer una navegación más fluida e intuitiva al personal de la institución.
- **Funcionalidades adicionales:** Implementar funcionalidades secundarias pendientes como la gestión de usuarios por parte de un administrador.
- **Automatización de instalación de servidor:** Para simplificar la portabilidad, elaborar un *bash script* que instale Mosquitto y Node-Red, clone el repositorio de git-hub, configure la Raspberry Pi y establezca los servicios.

A largo plazo, como líneas de investigación y desarrollo para una futura revisión arquitectónica del sistema (Versión 2.0), se proponen las siguientes mejoras estructurales:

- **Actualización remota:** Incorporar soporte para la actualización de *firmware* a distancia mediante FOTA (*Firmware Over-The-Air*), facilitando el mantenimiento y despliegue de parches sin requerir acceso físico a las placas.
- **Alimentación integrada:** Diseñar una nueva revisión de la placa de circuito impreso (PCB) de los terminales que integre una fuente de alimentación conmutada, permitiendo su conexión directa a la red eléctrica comercial.
- **Transición a topología inalámbrica:** Desarrollar capas adicionales de cifrado y autenticación a nivel de trama, sentando las bases de seguridad necesarias para reemplazar el bus RS-485 cableado por una red de comunicaciones inalámbricas por radiofrecuencia.

11. Referencias

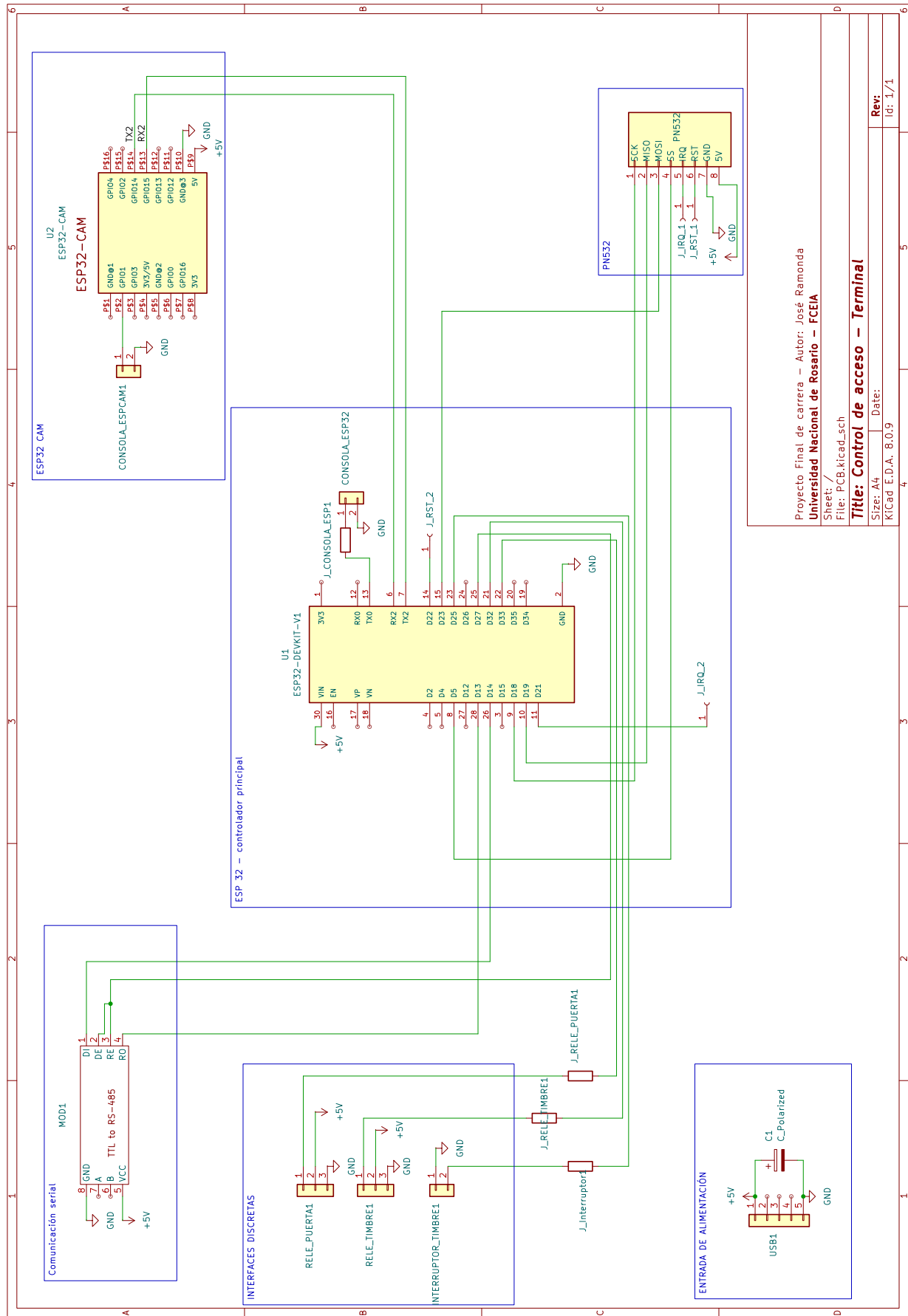
- [1] Espressif Systems, *ESP32 Series Datasheet*, Documentación técnica oficial, 2024. dirección: https://documentation.espressif.com/esp32_datasheet_en.pdf.
- [2] Espressif Systems, *ESP32 Technical Reference Manual*, Versión 5.6, 2024.
- [3] NXP Semiconductors, *NTAG213/215/216: NFC Forum Type 2 Tag compliant IC with 144/504/888 bytes user memory*, Rev. 3.0, Documento DS265330, 2013.
- [4] Espressif Systems, *ESP-IDF Programming Guide*, Release v5.3, 2024.
- [5] Espressif Systems. “ESP-IDF Programming Guide (Web Reference).” (2024), dirección: <https://docs.espressif.com/projects/esp-idf/en/v5.3/esp32/> (visitado 25-08-2026).
- [6] R. Barry y The FreeRTOS Team, *Mastering the FreeRTOS Real Time Kernel: A Hands-On Tutorial Guide*. 2016, Release Version 1.1.0.
- [7] Amazon Web Services, *The FreeRTOS Reference Manual: API Functions and Configuration Options*, 2017.
- [8] Amazon Web Services. “FreeRTOS API Reference.” (2026), dirección: <https://www.freertos.org/a00106.html> (visitado 25-08-2026).
- [9] Comunidad Open Source. “Librería y componente PN532 para ESP-IDF.” Repositorio de GitHub. (2026), dirección: <https://github.com/garag/esp-idf-pn532> (visitado 25-08-2026).
- [10] Telegram Messenger LLP. “Telegram Bot API Documentation.” (2026), dirección: <https://core.telegram.org/bots/api> (visitado 25-08-2026).
- [11] J. Ramonda. “Sistema de Control de Acceso para Institución Educativa.” Repositorio de código fuente del proyecto. (2026), dirección: <https://github.com/Jose-Ramonda/Control-de-acceso-para-institucion-educativa>.

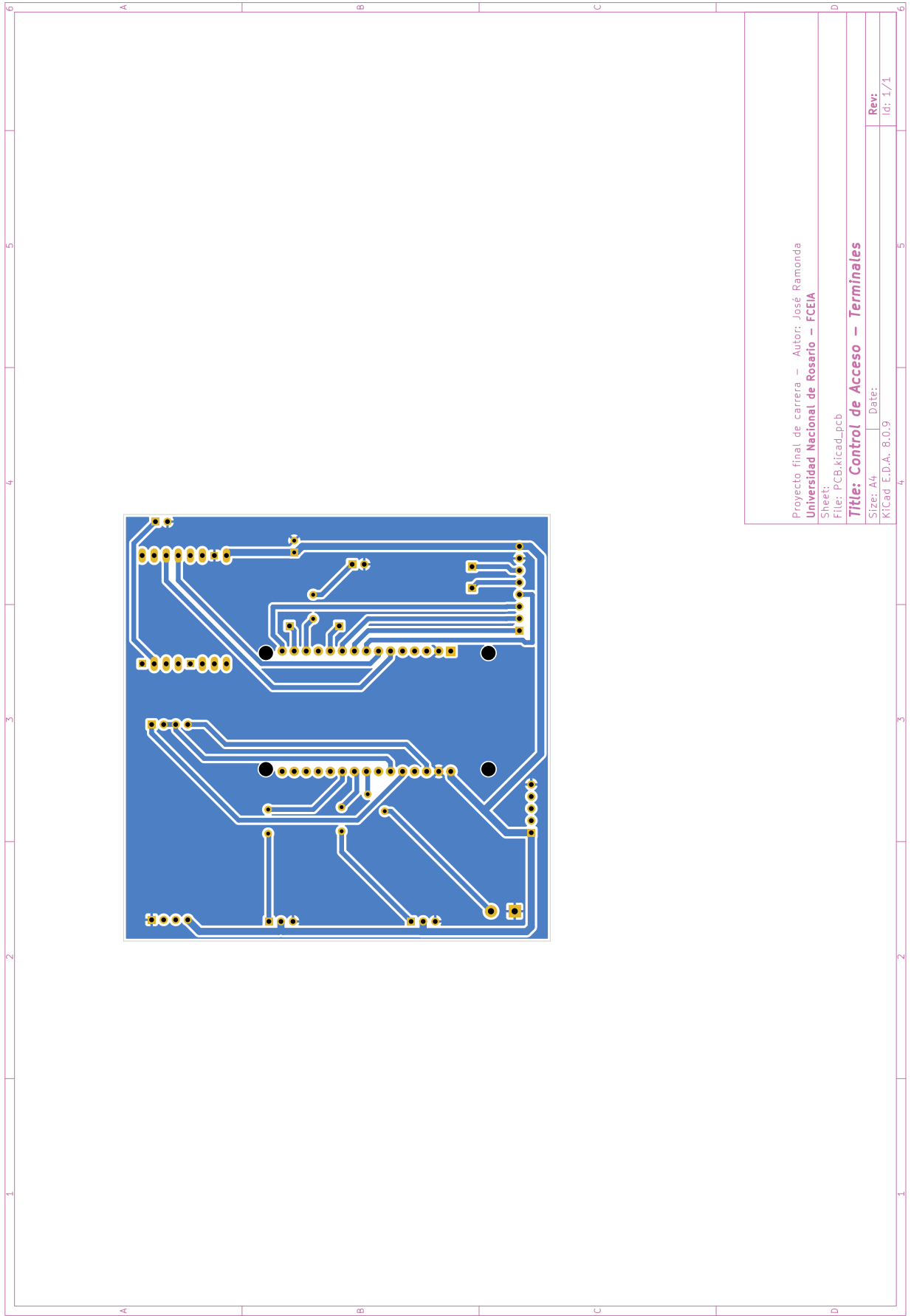


12. Anexos

12.1. Anexo 1: Diseño de PCB y Esquemáticos

Los siguientes diagramas detallan el diseño de hardware de los terminales de acceso. En primer lugar, se presenta el esquemático original de las conexiones. A continuación, se expone una versión modificada que incorpora resistencias de 0Ω a modo de puentes, implementada como solución ante las restricciones topológicas para el ruteo de las pistas en una placa de simple faz. Finalmente, se incluye el diseño final de la placa de circuito impreso (PCB).





12.2. Anexo 2: Trazas de ejecución del sistema (Logs)

A continuación se adjuntan los registros de ejecución (*logs*) extraídos a partir de la consola del servidor para los procesos de interfaz serial y ejecución del bot de Telegram.

12.2.1. Ensayo a 460800 baudios sin intervalo entre encuestas

Traza de servicio en Raspberri Pi Zero 2 W

```
1 Aug 23 14:02:46.226312 ramondapi python[1443]: [MQTT PUB SUCCESS] sbc/notify -> {"
  ↳ destino": "all", "evento": "alerta", "nodo": "Ox1e", "data": "DESCONECTADO"}
2 Aug 23 14:02:46.870591 ramondapi python[1443]: [DEBUG BUS] Encolando evento a la
  ↳ entrada evento 101
3 Aug 23 14:02:46.870942 ramondapi python[1443]: [PARSER - WIFI] El nodo 0xa reporta
  ↳ IP: 192.168.0.10
4 Aug 23 14:02:46.871331 ramondapi python[1443]: [MQTT PUB SUCCESS] sbc/status/0xa ->
  ↳ 192.168.0.10
5 Aug 23 14:02:47.312823 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
  ↳ de Telegram.
6 Aug 23 14:02:47.313997 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
  ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg
7 Aug 23 14:02:47.313997 ramondapi python[1179]: [BOT-MQTT IN] Evento: ALERTA | Nodo:
  ↳ Acceso Patio | Destino: all
8 Aug 23 14:02:48.545270 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
  ↳ de Telegram.
9 Aug 23 14:02:48.546551 ramondapi python[1179]: [BOT-MQTT IN] Evento: ALERTA | Nodo:
  ↳ Acceso Biciclatero | Destino: all
10 Aug 23 14:02:49.796689 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
  ↳ de Telegram.
11 Aug 23 14:03:32.776352 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
  ↳ cmd/abrir | Payload: 0x0A,8816825051
12 Aug 23 14:03:32.781376 ramondapi python[1443]: [ACCION] Ejecutando apertura del
  ↳ nodo 10 por bus serie...
13 Aug 23 14:03:32.781376 ramondapi python[1443]: [LOGICA] Comando 3 encolado para el
  ↳ nodo 0xa
14 Aug 23 14:03:32.782571 ramondapi python[1443]: [DEBUG BUS] Encolando evento a la
  ↳ entrada evento 3
15 Aug 23 14:03:32.782769 ramondapi python[1443]: [PARSER - PUERTA] El nodo 0xa
  ↳ reporta APERTURA DE LA PUERTA.
16 Aug 23 14:03:32.783879 ramondapi python[1443]: [MQTT PUB SUCCESS] sbc/status/0xa ->
  ↳ OK_PUERTA
17 Aug 23 14:03:36.948241 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
  ↳ cmd/foto | Payload: 0x0A,8816825051
18 Aug 23 14:03:36.949824 ramondapi python[1443]: [ACCION] Encolando pedido de
  ↳ fotografia para el nodo 10...
19 Aug 23 14:03:36.949824 ramondapi python[1443]: [LOGICA] Comando 7 encolado para el
  ↳ nodo 0xa
20 Aug 23 14:03:37.244801 ramondapi python[1443]: [DEBUG BUS] Encolando evento a la
  ↳ entrada evento 7
21 Aug 23 14:03:37.245197 ramondapi python[1443]: [PARSER - CAMARA] El nodo 0xa
  ↳ reporta fotografía tomada
22 Aug 23 14:03:37.245750 ramondapi python[1443]: [MQTT PUB SUCCESS] sbc/status/0xa ->
  ↳ OK_FOTO
23 Aug 23 14:03:37.251677 ramondapi python[1179]: [BOT-MQTT IN] Evento:
  ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
24 Aug 23 14:03:39.166555 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
  ↳ de Telegram.
25 Aug 23 14:03:39.167778 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
  ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg
26 Aug 23 14:03:43.906173 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
  ↳ cmd/foto | Payload: 0x0A,8816825051
27 Aug 23 14:03:43.908006 ramondapi python[1443]: [ACCION] Encolando pedido de
  ↳ fotografia para el nodo 10...
28 Aug 23 14:03:43.908006 ramondapi python[1443]: [LOGICA] Comando 7 encolado para el
  ↳ nodo 0xa
29 Aug 23 14:03:44.036453 ramondapi python[1443]: [DEBUG BUS] Encolando evento a la
  ↳ entrada evento 7
30 Aug 23 14:03:44.036827 ramondapi python[1443]: [PARSER - CAMARA] El nodo 0xa
  ↳ reporta fotografía tomada
```

```

31 Aug 23 14:03:44.037313 ramondapi python[1443]: [MQTT PUB SUCCESS] sbc/status/Oxa ->
    ↳ OK_FOTO
32 Aug 23 14:03:44.044269 ramondapi python[1179]: [BOT-MQTT IN] Evento:
    ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
33 Aug 23 14:03:45.290941 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
    ↳ de Telegram.
34 Aug 23 14:03:45.291936 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
    ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg
35 Aug 23 14:03:49.280263 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
    ↳ cmd/foto | Payload: 0x0A,8816825051
36 Aug 23 14:03:49.281804 ramondapi python[1443]: [ACCION] Encolando pedido de
    ↳ fotografia para el nodo 10...
37 Aug 23 14:03:49.281804 ramondapi python[1443]: [LOGICA] Comando 7 encolado para el
    ↳ nodo Oxa
38 Aug 23 14:03:49.551704 ramondapi python[1443]: [DEBUG BUS] Encolando evento a la
    ↳ entrada evento 7
39 Aug 23 14:03:49.551704 ramondapi python[1443]: [PARSER - CAMARA] El nodo Oxa
    ↳ reporta fotografia tomada
40 Aug 23 14:03:49.552639 ramondapi python[1443]: [MQTT PUB SUCCESS] sbc/status/Oxa ->
    ↳ OK_FOTO
41 Aug 23 14:03:49.559461 ramondapi python[1179]: [BOT-MQTT IN] Evento:
    ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
42 Aug 23 14:03:51.467810 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
    ↳ de Telegram.
43 Aug 23 14:03:51.469086 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
    ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg
44 Aug 23 14:03:53.777267 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
    ↳ cmd/foto | Payload: 0x0A,8816825051
45 Aug 23 14:03:53.781394 ramondapi python[1443]: [ACCION] Encolando pedido de
    ↳ fotografia para el nodo 10...
46 Aug 23 14:03:53.781394 ramondapi python[1443]: [LOGICA] Comando 7 encolado para el
    ↳ nodo Oxa
47 Aug 23 14:03:54.040304 ramondapi python[1443]: [DEBUG BUS] Encolando evento a la
    ↳ entrada evento 7
48 Aug 23 14:03:54.040645 ramondapi python[1443]: [PARSER - CAMARA] El nodo Oxa
    ↳ reporta fotografia tomada
49 Aug 23 14:03:54.041040 ramondapi python[1443]: [MQTT PUB SUCCESS] sbc/status/Oxa ->
    ↳ OK_FOTO
50 Aug 23 14:03:54.047312 ramondapi python[1179]: [BOT-MQTT IN] Evento:
    ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
51 Aug 23 14:03:55.945333 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
    ↳ de Telegram.
52 Aug 23 14:03:55.946913 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
    ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg
53 Aug 23 14:03:58.906688 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
    ↳ cmd/foto | Payload: 0x0A,8816825051
54 Aug 23 14:03:58.908683 ramondapi python[1443]: [ACCION] Encolando pedido de
    ↳ fotografia para el nodo 10...
55 Aug 23 14:03:58.908683 ramondapi python[1443]: [LOGICA] Comando 7 encolado para el
    ↳ nodo Oxa
56 Aug 23 14:03:59.712528 ramondapi python[1443]: [DEBUG BUS] Encolando evento a la
    ↳ entrada evento 7
57 Aug 23 14:03:59.712955 ramondapi python[1443]: [PARSER - CAMARA] El nodo Oxa
    ↳ reporta fotografia tomada
58 Aug 23 14:03:59.713443 ramondapi python[1443]: [MQTT PUB SUCCESS] sbc/status/Oxa ->
    ↳ OK_FOTO
59 Aug 23 14:03:59.724259 ramondapi python[1179]: [BOT-MQTT IN] Evento:
    ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
60 Aug 23 14:04:01.759635 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
    ↳ de Telegram.
61 Aug 23 14:04:01.764670 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
    ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg
62 Aug 23 14:04:03.922796 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
    ↳ cmd/foto | Payload: 0x0A,8816825051
63 Aug 23 14:04:03.924604 ramondapi python[1443]: [ACCION] Encolando pedido de
    ↳ fotografia para el nodo 10...
64 Aug 23 14:04:03.924604 ramondapi python[1443]: [LOGICA] Comando 7 encolado para el
    ↳ nodo Oxa
65 Aug 23 14:04:04.224985 ramondapi python[1443]: [DEBUG BUS] Encolando evento a la
    ↳ entrada evento 7
66 Aug 23 14:04:04.225316 ramondapi python[1443]: [PARSER - CAMARA] El nodo Oxa
    ↳ reporta fotografia tomada

```

```

67 Aug 23 14:04:04.225719 ramondapi python[1443]: [MQTT PUB SUCCESS] sbc/status/0xa ->
    ↳ OK_FOTO
68 Aug 23 14:04:04.231149 ramondapi python[1179]: [BOT-MQTT IN] Evento:
    ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
69 Aug 23 14:04:07.461395 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
    ↳ de Telegram.
70 Aug 23 14:04:07.462909 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
    ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg
71 Aug 23 14:04:08.065896 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
    ↳ cmd/foto | Payload: 0x0A,8816825051
72 Aug 23 14:04:08.066737 ramondapi python[1443]: [ACCION] Encolando pedido de
    ↳ fotografia para el nodo 10...
73 Aug 23 14:04:08.068590 ramondapi python[1443]: [LOGICA] Comando 7 encolado para el
    ↳ nodo Oxa
74 Aug 23 14:04:08.364421 ramondapi python[1443]: [DEBUG BUS] Encolando evento a la
    ↳ entrada evento 7
75 Aug 23 14:04:08.364805 ramondapi python[1443]: [PARSER - CAMARA] El nodo Oxa
    ↳ reporta fotografia tomada
76 Aug 23 14:04:08.365378 ramondapi python[1443]: [MQTT PUB SUCCESS] sbc/status/0xa ->
    ↳ OK_FOTO
77 Aug 23 14:04:08.372234 ramondapi python[1179]: [BOT-MQTT IN] Evento:
    ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
78 Aug 23 14:04:10.146220 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
    ↳ de Telegram.
79 Aug 23 14:04:10.149386 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
    ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg
80 Aug 23 14:04:12.247455 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
    ↳ cmd/foto | Payload: 0x0A,8816825051
81 Aug 23 14:04:12.251376 ramondapi python[1443]: [ACCION] Encolando pedido de
    ↳ fotografia para el nodo 10...
82 Aug 23 14:04:12.252983 ramondapi python[1443]: [LOGICA] Comando 7 encolado para el
    ↳ nodo Oxa
83 Aug 23 14:04:12.534476 ramondapi python[1443]: [DEBUG BUS] Encolando evento a la
    ↳ entrada evento 7
84 Aug 23 14:04:12.534808 ramondapi python[1443]: [PARSER - CAMARA] El nodo Oxa
    ↳ reporta fotografia tomada
85 Aug 23 14:04:12.535390 ramondapi python[1443]: [MQTT PUB SUCCESS] sbc/status/0xa ->
    ↳ OK_FOTO
86 Aug 23 14:04:12.540637 ramondapi python[1179]: [BOT-MQTT IN] Evento:
    ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
87 Aug 23 14:04:14.529655 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
    ↳ de Telegram.
88 Aug 23 14:04:14.531205 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
    ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg
89 Aug 23 14:04:16.702689 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
    ↳ cmd/foto | Payload: 0x0A,8816825051
90 Aug 23 14:04:16.704520 ramondapi python[1443]: [ACCION] Encolando pedido de
    ↳ fotografia para el nodo 10...
91 Aug 23 14:04:16.704520 ramondapi python[1443]: [LOGICA] Comando 7 encolado para el
    ↳ nodo Oxa
92 Aug 23 14:04:16.995929 ramondapi python[1443]: [DEBUG BUS] Encolando evento a la
    ↳ entrada evento 7
93 Aug 23 14:04:16.996387 ramondapi python[1443]: [PARSER - CAMARA] El nodo Oxa
    ↳ reporta fotografia tomada
94 Aug 23 14:04:16.996896 ramondapi python[1443]: [MQTT PUB SUCCESS] sbc/status/0xa ->
    ↳ OK_FOTO
95 Aug 23 14:04:17.003112 ramondapi python[1179]: [BOT-MQTT IN] Evento:
    ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
96 Aug 23 14:04:18.801091 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
    ↳ de Telegram.
97 Aug 23 14:04:18.802369 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
    ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg
98 Aug 23 14:04:21.841138 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
    ↳ cmd/foto | Payload: 0x0A,8816825051
99 Aug 23 14:04:21.842624 ramondapi python[1443]: [ACCION] Encolando pedido de
    ↳ fotografia para el nodo 10...
100 Aug 23 14:04:21.842624 ramondapi python[1443]: [LOGICA] Comando 7 encolado para el
    ↳ nodo Oxa
101 Aug 23 14:04:22.075529 ramondapi python[1443]: [DEBUG BUS] Encolando evento a la
    ↳ entrada evento 7
102 Aug 23 14:04:22.075979 ramondapi python[1443]: [PARSER - CAMARA] El nodo Oxa
    ↳ reporta fotografia tomada

```

```

103 Aug 23 14:04:22.076644 ramondapi python[1443]: [MQTT PUB SUCCESS] sbc/status/0xa ->
    ↳ OK_FOTO
104 Aug 23 14:04:22.084633 ramondapi python[1179]: [BOT-MQTT IN] Evento:
    ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
105 Aug 23 14:04:24.322393 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
    ↳ de Telegram.
106 Aug 23 14:04:24.323571 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
    ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg

```

12.2.2. Ensayo a 115200 baudios sin intervalo entre encuestas

Traza de servicio en Raspberri Pi Zero 2 W

```

1 Aug 23 13:55:21.973671 ramondapi python[1350]: [SERIAL WARN] Nodo 0x1e reporta un
    ↳ timeout
2 Aug 23 13:55:21.974058 ramondapi python[1350]: [SERIAL WARN] Nodo 0x1e pasó a
    ↳ OFFLINE
3 Aug 23 13:55:21.974058 ramondapi python[1350]: [LOGICA ALERTA] El nodo 0x1e se ha
    ↳ ido OFFLINE.
4 Aug 23 13:55:21.974806 ramondapi python[1350]: [MQTT PUB SUCCESS] sbc/notify -> {"
    ↳ destino": "all", "evento": "alerta", "nodo": "0x1e", "data": "DESCONECTADO"}
5 Aug 23 13:55:22.873450 ramondapi python[1350]: [DEBUG BUS] Encolando evento a la
    ↳ entrada evento 101
6 Aug 23 13:55:22.874699 ramondapi python[1350]: [PARSER - WIFI] El nodo 0xa reporta
    ↳ IP: 192.168.0.10
7 Aug 23 13:55:22.874699 ramondapi python[1350]: [MQTT PUB SUCCESS] sbc/status/0xa ->
    ↳ 192.168.0.10
8 Aug 23 13:55:23.222441 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
    ↳ de Telegram.
9 Aug 23 13:55:23.223604 ramondapi python[1179]: [BOT-MQTT IN] Evento: ALERTA | Nodo:
    ↳ Acceso Bicicletero | Destino: all
10 Aug 23 13:55:24.762161 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
    ↳ de Telegram.
11 Aug 23 13:55:59.595348 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
    ↳ cmd/foto | Payload: 0x0A,8816825051
12 Aug 23 13:55:59.597604 ramondapi python[1350]: [ACCION] Encolando pedido de
    ↳ fotografia para el nodo 10...
13 Aug 23 13:55:59.597604 ramondapi python[1350]: [LOGICA] Comando 7 encolado para el
    ↳ nodo 0xa
14 Aug 23 13:55:59.948801 ramondapi python[1350]: [DEBUG BUS] Encolando evento a la
    ↳ entrada evento 7
15 Aug 23 13:55:59.948801 ramondapi python[1350]: [PARSER - CAMARA] El nodo 0xa
    ↳ reporta fotografía tomada
16 Aug 23 13:55:59.949564 ramondapi python[1350]: [MQTT PUB SUCCESS] sbc/status/0xa ->
    ↳ OK_FOTO
17 Aug 23 13:55:59.958144 ramondapi python[1179]: [BOT-MQTT IN] Evento:
    ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
18 Aug 23 13:56:01.911067 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
    ↳ de Telegram.
19 Aug 23 13:56:01.912250 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
    ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg
20 Aug 23 13:56:12.796450 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
    ↳ cmd/foto | Payload: 0x0A,8816825051
21 Aug 23 13:56:12.802352 ramondapi python[1350]: [ACCION] Encolando pedido de
    ↳ fotografia para el nodo 10...
22 Aug 23 13:56:12.802352 ramondapi python[1350]: [LOGICA] Comando 7 encolado para el
    ↳ nodo 0xa
23 Aug 23 13:56:12.937809 ramondapi python[1350]: [DEBUG BUS] Encolando evento a la
    ↳ entrada evento 7
24 Aug 23 13:56:12.938232 ramondapi python[1350]: [PARSER - CAMARA] El nodo 0xa
    ↳ reporta fotografía tomada
25 Aug 23 13:56:12.938804 ramondapi python[1350]: [MQTT PUB SUCCESS] sbc/status/0xa ->
    ↳ OK_FOTO
26 Aug 23 13:56:12.945142 ramondapi python[1179]: [BOT-MQTT IN] Evento:
    ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
27 Aug 23 13:56:14.296655 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
    ↳ de Telegram.
28 Aug 23 13:56:14.296776 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
    ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg
29 Aug 23 13:56:19.033597 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
    ↳ cmd/foto | Payload: 0x0A,8816825051

```



```

30 Aug 23 13:56:19.035783 ramondapi python[1350]: [ACCION] Encolando pedido de
    ↳ fotografia para el nodo 10...
31 Aug 23 13:56:19.035783 ramondapi python[1350]: [LOGICA] Comando 7 encolado para el
    ↳ nodo Oxa
32 Aug 23 13:56:19.419832 ramondapi python[1350]: [DEBUG BUS] Encolando evento a la
    ↳ entrada evento 7
33 Aug 23 13:56:19.420362 ramondapi python[1350]: [PARSER - CAMARA] El nodo Oxa
    ↳ reporta fotografía tomada
34 Aug 23 13:56:19.420893 ramondapi python[1350]: [MQTT PUB SUCCESS] sbc/status/Oxa ->
    ↳ OK_FOTO
35 Aug 23 13:56:19.429720 ramondapi python[1179]: [BOT-MQTT IN] Evento:
    ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
36 Aug 23 13:56:21.448553 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
    ↳ de Telegram.
37 Aug 23 13:56:21.449648 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
    ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg
38 Aug 23 13:56:29.916643 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
    ↳ cmd/foto | Payload: 0x0A,8816825051
39 Aug 23 13:56:29.918732 ramondapi python[1350]: [ACCION] Encolando pedido de
    ↳ fotografia para el nodo 10...
40 Aug 23 13:56:29.918732 ramondapi python[1350]: [LOGICA] Comando 7 encolado para el
    ↳ nodo Oxa
41 Aug 23 13:56:30.061775 ramondapi python[1350]: [DEBUG BUS] Encolando evento a la
    ↳ entrada evento 7
42 Aug 23 13:56:30.062206 ramondapi python[1350]: [PARSER - CAMARA] El nodo Oxa
    ↳ reporta fotografía tomada
43 Aug 23 13:56:30.062744 ramondapi python[1350]: [MQTT PUB SUCCESS] sbc/status/Oxa ->
    ↳ OK_FOTO
44 Aug 23 13:56:30.069050 ramondapi python[1179]: [BOT-MQTT IN] Evento:
    ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
45 Aug 23 13:56:31.359851 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
    ↳ de Telegram.
46 Aug 23 13:56:31.360193 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
    ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg
47 Aug 23 13:56:38.315212 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
    ↳ cmd/foto | Payload: 0x0A,8816825051
48 Aug 23 13:56:38.317281 ramondapi python[1350]: [ACCION] Encolando pedido de
    ↳ fotografia para el nodo 10...
49 Aug 23 13:56:38.317281 ramondapi python[1350]: [LOGICA] Comando 7 encolado para el
    ↳ nodo Oxa
50 Aug 23 13:56:38.611383 ramondapi python[1350]: [DEBUG BUS] Encolando evento a la
    ↳ entrada evento 7
51 Aug 23 13:56:38.611814 ramondapi python[1350]: [PARSER - CAMARA] El nodo Oxa
    ↳ reporta fotografía tomada
52 Aug 23 13:56:38.612585 ramondapi python[1350]: [MQTT PUB SUCCESS] sbc/status/Oxa ->
    ↳ OK_FOTO
53 Aug 23 13:56:38.622530 ramondapi python[1179]: [BOT-MQTT IN] Evento:
    ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
54 Aug 23 13:56:40.569596 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
    ↳ de Telegram.
55 Aug 23 13:56:40.570722 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
    ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg
56 Aug 23 13:56:44.328580 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
    ↳ cmd/foto | Payload: 0x0A,8816825051
57 Aug 23 13:56:44.330456 ramondapi python[1350]: [ACCION] Encolando pedido de
    ↳ fotografia para el nodo 10...
58 Aug 23 13:56:44.330456 ramondapi python[1350]: [LOGICA] Comando 7 encolado para el
    ↳ nodo Oxa
59 Aug 23 13:56:44.641038 ramondapi python[1350]: [DEBUG BUS] Encolando evento a la
    ↳ entrada evento 7
60 Aug 23 13:56:44.641459 ramondapi python[1350]: [PARSER - CAMARA] El nodo Oxa
    ↳ reporta fotografía tomada
61 Aug 23 13:56:44.641979 ramondapi python[1350]: [MQTT PUB SUCCESS] sbc/status/Oxa ->
    ↳ OK_FOTO
62 Aug 23 13:56:44.648803 ramondapi python[1179]: [BOT-MQTT IN] Evento:
    ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
63 Aug 23 13:56:46.621091 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
    ↳ de Telegram.
64 Aug 23 13:56:46.622307 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
    ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg
65 Aug 23 13:56:50.547194 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
    ↳ cmd/foto | Payload: 0x0A,8816825051

```

```

66 Aug 23 13:56:50.553747 ramondapi python[1350]: [ACCION] Encolando pedido de
    ↳ fotografia para el nodo 10...
67 Aug 23 13:56:50.555744 ramondapi python[1350]: [LOGICA] Comando 7 encolado para el
    ↳ nodo 0xa
68 Aug 23 13:56:50.697575 ramondapi python[1350]: [DEBUG BUS] Encolando evento a la
    ↳ entrada evento 7
69 Aug 23 13:56:50.698001 ramondapi python[1350]: [PARSER - CAMARA] El nodo 0xa
    ↳ reporta fotografía tomada
70 Aug 23 13:56:50.698539 ramondapi python[1350]: [MQTT PUB SUCCESS] sbc/status/0xa ->
    ↳ OK_FOTO
71 Aug 23 13:56:50.704815 ramondapi python[1179]: [BOT-MQTT IN] Evento:
    ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
72 Aug 23 13:56:51.985045 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
    ↳ de Telegram.
73 Aug 23 13:56:51.986472 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
    ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg
74 Aug 23 13:56:57.507172 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
    ↳ cmd/foto | Payload: 0x0A,8816825051
75 Aug 23 13:56:57.509217 ramondapi python[1350]: [ACCION] Encolando pedido de
    ↳ fotografia para el nodo 10...
76 Aug 23 13:56:57.509217 ramondapi python[1350]: [LOGICA] Comando 7 encolado para el
    ↳ nodo 0xa
77 Aug 23 13:56:57.806429 ramondapi python[1350]: [DEBUG BUS] Encolando evento a la
    ↳ entrada evento 7
78 Aug 23 13:56:57.806857 ramondapi python[1350]: [PARSER - CAMARA] El nodo 0xa
    ↳ reporta fotografía tomada
79 Aug 23 13:56:57.807432 ramondapi python[1350]: [MQTT PUB SUCCESS] sbc/status/0xa ->
    ↳ OK_FOTO
80 Aug 23 13:56:57.814451 ramondapi python[1179]: [BOT-MQTT IN] Evento:
    ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
81 Aug 23 13:56:59.615960 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
    ↳ de Telegram.
82 Aug 23 13:56:59.617135 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
    ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg
83 Aug 23 13:57:03.400791 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
    ↳ cmd/foto | Payload: 0x0A,8816825051
84 Aug 23 13:57:03.402498 ramondapi python[1350]: [ACCION] Encolando pedido de
    ↳ fotografia para el nodo 10...
85 Aug 23 13:57:03.402498 ramondapi python[1350]: [LOGICA] Comando 7 encolado para el
    ↳ nodo 0xa
86 Aug 23 13:57:03.676382 ramondapi python[1350]: [DEBUG BUS] Encolando evento a la
    ↳ entrada evento 7
87 Aug 23 13:57:03.676713 ramondapi python[1350]: [PARSER - CAMARA] El nodo 0xa
    ↳ reporta fotografía tomada
88 Aug 23 13:57:03.677586 ramondapi python[1350]: [MQTT PUB SUCCESS] sbc/status/0xa ->
    ↳ OK_FOTO
89 Aug 23 13:57:03.682373 ramondapi python[1179]: [BOT-MQTT IN] Evento:
    ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
90 Aug 23 13:57:05.442759 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
    ↳ de Telegram.
91 Aug 23 13:57:05.443844 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
    ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg
92 Aug 23 13:57:10.195167 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
    ↳ cmd/foto | Payload: 0x0A,8816825051
93 Aug 23 13:57:10.197535 ramondapi python[1350]: [ACCION] Encolando pedido de
    ↳ fotografia para el nodo 10...
94 Aug 23 13:57:10.199389 ramondapi python[1350]: [LOGICA] Comando 7 encolado para el
    ↳ nodo 0xa
95 Aug 23 13:57:10.311923 ramondapi python[1350]: [DEBUG BUS] Encolando evento a la
    ↳ entrada evento 7
96 Aug 23 13:57:10.312378 ramondapi python[1350]: [PARSER - CAMARA] El nodo 0xa
    ↳ reporta fotografía tomada
97 Aug 23 13:57:10.312906 ramondapi python[1350]: [MQTT PUB SUCCESS] sbc/status/0xa ->
    ↳ OK_FOTO
98 Aug 23 13:57:10.322326 ramondapi python[1179]: [BOT-MQTT IN] Evento:
    ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
99 Aug 23 13:57:11.614423 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
    ↳ de Telegram.
100 Aug 23 13:57:11.615977 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
    ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg

```

12.2.3. Ensayo a 115200 baudios con retardo entre encuestas de 10 ms

Traza de servicio en Raspberri Pi Zero 2 W

```
1 Aug 23 13:40:21.251279 ramondapi systemd[1]: Stopping sbc-bot.service - SBC Bot
  ↳ Process...
2 Aug 23 13:40:22.481206 ramondapi systemd[1]: sbc-bot.service: Deactivated
  ↳ successfully.
3 Aug 23 13:40:22.481811 ramondapi systemd[1]: Stopped sbc-bot.service - SBC Bot
  ↳ Process.
4 Aug 23 13:40:22.482002 ramondapi systemd[1]: sbc-bot.service: Consumed 3.847s CPU
  ↳ time.
5 Aug 23 13:40:22.488596 ramondapi systemd[1]: Started sbc-bot.service - SBC Bot
  ↳ Process.
6 Aug 23 13:40:24.202446 ramondapi python[1179]: /home/jose/PROYECTOFINAL_MASTER/SBC/
  ↳ bot/mods/estados.py:106: PTBUserWarning: If 'per_message=False', '
  ↳ CallbackQueryHandler' will not be tracked for every message. Read this FAQ
  ↳ entry to learn more about the per_* settings: https://github.com/python-
  ↳ telegram-bot/python-telegram-bot/wiki/Frequently-Asked-Questions#what-do-the
  ↳ -per_-settings-in-conversationhandler-do.
7 Aug 23 13:40:24.202446 ramondapi python[1179]: return ConversationHandler(
8 Aug 23 13:40:24.210713 ramondapi python[1179]: [BOT-MQTT] Cliente MQTT persistente
  ↳ conectado y en escucha sobre 'sbc/notify'
9 Aug 23 13:40:24.211321 ramondapi python[1179]: [BOT] Servidor SBC escuchando
  ↳ peticiones en Telegram...
10 Aug 23 13:40:34.103434 ramondapi python[1179]: [AUTH] ID 8816825051 vinculada
  ↳ exitosamente a 'Jose'
11 Aug 23 13:42:01.604258 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
  ↳ cmd/foto | Payload: 0x0A,8816825051
12 Aug 23 13:42:01.606486 ramondapi python[1117]: [ACCION] Encolando pedido de
  ↳ fotografia para el nodo 10...
13 Aug 23 13:42:01.606486 ramondapi python[1117]: [LOGICA] Comando 7 encolado para el
  ↳ nodo 0xa
14 Aug 23 13:42:01.952665 ramondapi python[1117]: [DEBUG BUS] Encolando evento a la
  ↳ entrada evento 7
15 Aug 23 13:42:01.953008 ramondapi python[1117]: [PARSER - CAMARA] El nodo 0xa
  ↳ reporta fotografía tomada
16 Aug 23 13:42:01.953273 ramondapi python[1117]: [MQTT PUB SUCCESS] sbc/status/0xa ->
  ↳ OK_FOTO
17 Aug 23 13:42:01.971360 ramondapi python[1179]: [BOT-MQTT IN] Evento:
  ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
18 Aug 23 13:42:03.891609 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
  ↳ de Telegram.
19 Aug 23 13:42:03.892795 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
  ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg
20 Aug 23 13:46:50.547399 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
  ↳ cmd/foto | Payload: 0x0A,8816825051
21 Aug 23 13:46:50.551845 ramondapi python[1117]: [ACCION] Encolando pedido de
  ↳ fotografia para el nodo 10...
22 Aug 23 13:46:50.554187 ramondapi python[1117]: [LOGICA] Comando 7 encolado para el
  ↳ nodo 0xa
23 Aug 23 13:46:50.903265 ramondapi python[1117]: [DEBUG BUS] Encolando evento a la
  ↳ entrada evento 7
24 Aug 23 13:46:50.903748 ramondapi python[1117]: [PARSER - CAMARA] El nodo 0xa
  ↳ reporta fotografía tomada
25 Aug 23 13:46:50.904203 ramondapi python[1117]: [MQTT PUB SUCCESS] sbc/status/0xa ->
  ↳ OK_FOTO
26 Aug 23 13:46:50.920823 ramondapi python[1179]: [BOT-MQTT IN] Evento:
  ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
27 Aug 23 13:46:52.849567 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
  ↳ de Telegram.
28 Aug 23 13:46:52.850662 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
  ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg
29 Aug 23 13:46:57.600386 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
  ↳ cmd/foto | Payload: 0x0A,8816825051
30 Aug 23 13:46:57.602686 ramondapi python[1117]: [ACCION] Encolando pedido de
  ↳ fotografia para el nodo 10...
31 Aug 23 13:46:57.602686 ramondapi python[1117]: [LOGICA] Comando 7 encolado para el
  ↳ nodo 0xa
32 Aug 23 13:46:57.720084 ramondapi python[1117]: [DEBUG BUS] Encolando evento a la
  ↳ entrada evento 7
33 Aug 23 13:46:57.720530 ramondapi python[1117]: [PARSER - CAMARA] El nodo 0xa
  ↳ reporta fotografía tomada
```

```

34 Aug 23 13:46:57.720890 ramondapi python[1117]: [MQTT PUB SUCCESS] sbc/status/Oxa ->
    ↳ OK_FOTO
35 Aug 23 13:46:57.729583 ramondapi python[1179]: [BOT-MQTT IN] Evento:
    ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
36 Aug 23 13:46:59.024011 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
    ↳ de Telegram.
37 Aug 23 13:46:59.025231 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
    ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg
38 Aug 23 13:47:01.404047 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
    ↳ cmd/foto | Payload: 0x0A,8816825051
39 Aug 23 13:47:01.407751 ramondapi python[1117]: [ACCION] Encolando pedido de
    ↳ fotografia para el nodo 10...
40 Aug 23 13:47:01.407751 ramondapi python[1117]: [LOGICA] Comando 7 encolado para el
    ↳ nodo Oxa
41 Aug 23 13:47:01.700220 ramondapi python[1117]: [DEBUG BUS] Encolando evento a la
    ↳ entrada evento 7
42 Aug 23 13:47:01.700551 ramondapi python[1117]: [PARSER - CAMARA] El nodo Oxa
    ↳ reporta fotografia tomada
43 Aug 23 13:47:01.700834 ramondapi python[1117]: [MQTT PUB SUCCESS] sbc/status/Oxa ->
    ↳ OK_FOTO
44 Aug 23 13:47:01.708319 ramondapi python[1179]: [BOT-MQTT IN] Evento:
    ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
45 Aug 23 13:47:03.693518 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
    ↳ de Telegram.
46 Aug 23 13:47:03.695297 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
    ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg
47 Aug 23 13:47:15.183682 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
    ↳ cmd/abrir | Payload: 0x0A,8816825051
48 Aug 23 13:47:15.185847 ramondapi python[1117]: [ACCION] Ejecutando apertura del
    ↳ nodo 10 por bus serie...
49 Aug 23 13:47:15.185847 ramondapi python[1117]: [LOGICA] Comando 3 encolado para el
    ↳ nodo Oxa
50 Aug 23 13:47:15.194869 ramondapi python[1117]: [DEBUG BUS] Encolando evento a la
    ↳ entrada evento 3
51 Aug 23 13:47:15.195404 ramondapi python[1117]: [PARSER - PUERTA] El nodo Oxa
    ↳ reporta APERTURA DE LA PUERTA.
52 Aug 23 13:47:15.195750 ramondapi python[1117]: [MQTT PUB SUCCESS] sbc/status/Oxa ->
    ↳ OK_PUERTA
53 Aug 23 13:47:28.914248 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
    ↳ cmd/abrir | Payload: 0x0A,8816825051
54 Aug 23 13:47:28.917987 ramondapi python[1117]: [ACCION] Ejecutando apertura del
    ↳ nodo 10 por bus serie...
55 Aug 23 13:47:28.919364 ramondapi python[1117]: [LOGICA] Comando 3 encolado para el
    ↳ nodo Oxa
56 Aug 23 13:47:28.926149 ramondapi python[1117]: [DEBUG BUS] Encolando evento a la
    ↳ entrada evento 3
57 Aug 23 13:47:28.926485 ramondapi python[1117]: [PARSER - PUERTA] El nodo Oxa
    ↳ reporta APERTURA DE LA PUERTA.
58 Aug 23 13:47:28.926670 ramondapi python[1117]: [MQTT PUB SUCCESS] sbc/status/Oxa ->
    ↳ OK_PUERTA
59 Aug 23 13:47:32.815520 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
    ↳ cmd/foto | Payload: 0x0A,8816825051
60 Aug 23 13:47:32.817670 ramondapi python[1117]: [ACCION] Encolando pedido de
    ↳ fotografia para el nodo 10...
61 Aug 23 13:47:32.817670 ramondapi python[1117]: [LOGICA] Comando 7 encolado para el
    ↳ nodo Oxa
62 Aug 23 13:47:33.003200 ramondapi python[1117]: [DEBUG BUS] Encolando evento a la
    ↳ entrada evento 7
63 Aug 23 13:47:33.003643 ramondapi python[1117]: [PARSER - CAMARA] El nodo Oxa
    ↳ reporta fotografia tomada
64 Aug 23 13:47:33.004005 ramondapi python[1117]: [MQTT PUB SUCCESS] sbc/status/Oxa ->
    ↳ OK_FOTO
65 Aug 23 13:47:33.011139 ramondapi python[1179]: [BOT-MQTT IN] Evento:
    ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
66 Aug 23 13:47:34.332203 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
    ↳ de Telegram.
67 Aug 23 13:47:34.333749 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
    ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg
68 Aug 23 13:47:46.719246 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
    ↳ cmd/foto | Payload: 0x0A,8816825051
69 Aug 23 13:47:46.721698 ramondapi python[1117]: [ACCION] Encolando pedido de
    ↳ fotografia para el nodo 10...

```

```

70 Aug 23 13:47:46.721698 ramondapi python[1117]: [LOGICA] Comando 7 encolado para el
    ↳ nodo Oxa
71 Aug 23 13:47:46.985558 ramondapi python[1117]: [DEBUG BUS] Encolando evento a la
    ↳ entrada evento 7
72 Aug 23 13:47:46.985985 ramondapi python[1117]: [PARSER - CAMARA] El nodo Oxa
    ↳ reporta fotografía tomada
73 Aug 23 13:47:46.986300 ramondapi python[1117]: [MQTT PUB SUCCESS] sbc/status/Oxa ->
    ↳ OK_FOTO
74 Aug 23 13:47:46.993643 ramondapi python[1179]: [BOT-MQTT IN] Evento:
    ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
75 Aug 23 13:47:48.927601 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
    ↳ de Telegram.
76 Aug 23 13:47:48.928772 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
    ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg
77 Aug 23 13:47:53.440657 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
    ↳ cmd/foto | Payload: 0x0A,8816825051
78 Aug 23 13:47:53.442903 ramondapi python[1117]: [ACCION] Encolando pedido de
    ↳ fotografia para el nodo 10...
79 Aug 23 13:47:53.442903 ramondapi python[1117]: [LOGICA] Comando 7 encolado para el
    ↳ nodo Oxa
80 Aug 23 13:47:53.592192 ramondapi python[1117]: [DEBUG BUS] Encolando evento a la
    ↳ entrada evento 7
81 Aug 23 13:47:53.592620 ramondapi python[1117]: [PARSER - CAMARA] El nodo Oxa
    ↳ reporta fotografía tomada
82 Aug 23 13:47:53.592994 ramondapi python[1117]: [MQTT PUB SUCCESS] sbc/status/Oxa ->
    ↳ OK_FOTO
83 Aug 23 13:47:53.600902 ramondapi python[1179]: [BOT-MQTT IN] Evento:
    ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
84 Aug 23 13:47:54.894014 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
    ↳ de Telegram.
85 Aug 23 13:47:54.896389 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
    ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg
86 Aug 23 13:48:05.173985 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
    ↳ cmd/foto | Payload: 0x0A,8816825051
87 Aug 23 13:48:05.176271 ramondapi python[1117]: [ACCION] Encolando pedido de
    ↳ fotografia para el nodo 10...
88 Aug 23 13:48:05.176271 ramondapi python[1117]: [LOGICA] Comando 7 encolado para el
    ↳ nodo Oxa
89 Aug 23 13:48:05.496234 ramondapi python[1117]: [DEBUG BUS] Encolando evento a la
    ↳ entrada evento 7
90 Aug 23 13:48:05.496719 ramondapi python[1117]: [PARSER - CAMARA] El nodo Oxa
    ↳ reporta fotografía tomada
91 Aug 23 13:48:05.497048 ramondapi python[1117]: [MQTT PUB SUCCESS] sbc/status/Oxa ->
    ↳ OK_FOTO
92 Aug 23 13:48:05.504226 ramondapi python[1179]: [BOT-MQTT IN] Evento:
    ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
93 Aug 23 13:48:07.301542 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
    ↳ de Telegram.
94 Aug 23 13:48:07.302633 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
    ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg
95 Aug 23 13:48:28.384460 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
    ↳ cmd/foto | Payload: 0x0A,8816825051
96 Aug 23 13:48:28.386641 ramondapi python[1117]: [ACCION] Encolando pedido de
    ↳ fotografia para el nodo 10...
97 Aug 23 13:48:28.386641 ramondapi python[1117]: [LOGICA] Comando 7 encolado para el
    ↳ nodo Oxa
98 Aug 23 13:48:28.681137 ramondapi python[1117]: [DEBUG BUS] Encolando evento a la
    ↳ entrada evento 7
99 Aug 23 13:48:28.681137 ramondapi python[1117]: [PARSER - CAMARA] El nodo Oxa
    ↳ reporta fotografía tomada
100 Aug 23 13:48:28.681879 ramondapi python[1117]: [MQTT PUB SUCCESS] sbc/status/Oxa ->
    ↳ OK_FOTO
101 Aug 23 13:48:28.689161 ramondapi python[1179]: [BOT-MQTT IN] Evento:
    ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
102 Aug 23 13:48:30.485355 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
    ↳ de Telegram.
103 Aug 23 13:48:30.486450 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
    ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg
104 Aug 23 13:48:42.508110 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
    ↳ cmd/foto | Payload: 0x0A,8816825051
105 Aug 23 13:48:42.510420 ramondapi python[1117]: [ACCION] Encolando pedido de
    ↳ fotografia para el nodo 10...

```

```

106 Aug 23 13:48:42.510420 ramondapi python[1117]: [LOGICA] Comando 7 encolado para el
    ↳ nodo Oxa
107 Aug 23 13:48:42.790581 ramondapi python[1117]: [DEBUG BUS] Encolando evento a la
    ↳ entrada evento 7
108 Aug 23 13:48:42.790581 ramondapi python[1117]: [PARSER - CAMARA] El nodo Oxa
    ↳ reporta fotografía tomada
109 Aug 23 13:48:42.791313 ramondapi python[1117]: [MQTT PUB SUCCESS] sbc/status/Oxa ->
    ↳ OK_FOTO
110 Aug 23 13:48:42.798155 ramondapi python[1179]: [BOT-MQTT IN] Evento:
    ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
111 Aug 23 13:48:44.927221 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
    ↳ de Telegram.
112 Aug 23 13:48:44.928371 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
    ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg
113 Aug 23 13:48:55.661616 ramondapi python[1179]: [BOT-MQTT CMD ENVIADO] Topico: sbc/
    ↳ cmd/foto | Payload: 0x0A,8816825051
114 Aug 23 13:48:55.663828 ramondapi python[1117]: [ACCION] Encolando pedido de
    ↳ fotografia para el nodo 10...
115 Aug 23 13:48:55.663828 ramondapi python[1117]: [LOGICA] Comando 7 encolado para el
    ↳ nodo Oxa
116 Aug 23 13:48:55.939267 ramondapi python[1117]: [DEBUG BUS] Encolando evento a la
    ↳ entrada evento 7
117 Aug 23 13:48:55.939716 ramondapi python[1117]: [PARSER - CAMARA] El nodo Oxa
    ↳ reporta fotografía tomada
118 Aug 23 13:48:55.939993 ramondapi python[1117]: [MQTT PUB SUCCESS] sbc/status/Oxa ->
    ↳ OK_FOTO
119 Aug 23 13:48:55.947208 ramondapi python[1179]: [BOT-MQTT IN] Evento:
    ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
120 Aug 23 13:48:57.685325 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
    ↳ de Telegram.
121 Aug 23 13:48:57.686422 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
    ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg
122 Aug 23 13:53:19.175862 ramondapi python[1117]: [DEBUG BUS] Encolando evento a la
    ↳ entrada evento 7
123 Aug 23 13:53:19.175862 ramondapi python[1117]: [PARSER - CAMARA] El nodo Oxa
    ↳ reporta fotografía tomada
124 Aug 23 13:53:19.177845 ramondapi python[1117]: [MQTT PUB SUCCESS] sbc/status/Oxa ->
    ↳ OK_FOTO
125 Aug 23 13:53:19.225453 ramondapi python[1179]: [BOT-MQTT IN] Evento:
    ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
126 Aug 23 13:53:21.430350 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
    ↳ de Telegram.
127 Aug 23 13:53:21.431495 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
    ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg
128 Aug 23 13:53:42.965951 ramondapi python[1117]: [DEBUG BUS] Encolando evento a la
    ↳ entrada evento 7
129 Aug 23 13:53:42.968080 ramondapi python[1117]: [PARSER - CAMARA] El nodo Oxa
    ↳ reporta fotografía tomada
130 Aug 23 13:53:42.968080 ramondapi python[1117]: [MQTT PUB SUCCESS] sbc/status/Oxa ->
    ↳ OK_FOTO
131 Aug 23 13:53:43.013336 ramondapi python[1179]: [BOT-MQTT IN] Evento:
    ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
132 Aug 23 13:53:44.286239 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
    ↳ de Telegram.
133 Aug 23 13:53:59.059710 ramondapi python[1117]: [DEBUG BUS] Encolando evento a la
    ↳ entrada evento 7
134 Aug 23 13:53:59.061649 ramondapi python[1117]: [PARSER - CAMARA] El nodo Oxa
    ↳ reporta fotografía tomada
135 Aug 23 13:53:59.061649 ramondapi python[1117]: [MQTT PUB SUCCESS] sbc/status/Oxa ->
    ↳ OK_FOTO
136 Aug 23 13:53:59.105157 ramondapi python[1179]: [BOT-MQTT IN] Evento:
    ↳ FOTO_ESPONTANEA_OK | Nodo: Acceso Frente | Destino: all
137 Aug 23 13:54:01.024976 ramondapi python[1179]: [BOT-MQTT SUCCESS] Despachado a API
    ↳ de Telegram.
138 Aug 23 13:54:01.026069 ramondapi python[1179]: [BOT-MQTT] Imagen temporal eliminada
    ↳ : /home/jose/PROYECTOFINAL_MASTER/SBC/fotos/captura_actual.jpeg

```

12.3. Anexo 3: Hoja de datos NTAG21x - Fragmento

A continuación se presentan algunas páginas relevantes de la hoja de datos de las etiquetas NFC **NTAG213**, **NTAG215** y **NTAG216**, utilizados en la configuración y la lectura de los tags.

Se incluye

- Mapa de registros de NTAG213 y NTAG215 (NTAG216 se omite).
- Detalle de registros de configuración.
- Función ASCII MIRROR.
- Función ASCII MIRROR aplicada al contador por hardware.
- Protección por contraseña de los tag.
- Comando de lectura rápida
- Comando de escritura
- Comando de autorización mediante contraseña

8.5 Memory organization

The EEPROM memory is organized in pages with 4 bytes per page. NTAG213 variant has 45 pages, NTAG215 variant has 135 pages and NTAG216 variant has 231 pages in total. The memory organization can be seen in [Figure 4](#), [Figure 5](#) and [Figure 6](#), the functionality of the different memory sections is described in the following sections.

Page Adr		Byte number within a page				Description
Dec	Hex	0	1	2	3	
0	0h	serial number				Manufacturer data and static lock bytes
1	1h	serial number				
2	2h	serial number	internal	lock bytes	lock bytes	
3	3h	Capability Container (CC)				Capability Container
4	4h	user memory				User memory pages
5	5h					
...	...					
38	26h					
39	27h					
40	28h	dynamic lock bytes			RFUI	Dynamic lock bytes
41	29h	CFG 0				Configuration pages
42	2Ah	CFG 1				
43	2Bh	PWD				
44	2Ch	PACK		RFUI		

aaa-008087

Fig 4. Memory organization NTAG213

The structure of manufacturing data, lock bytes, capability container and user memory pages are compatible to NTAG203.

Page Adr		Byte number within a page				Description
Dec	Hex	0	1	2	3	
0	0h	serial number				Manufacturer data and static lock bytes
1	1h	serial number				
2	2h	serial number	internal	lock bytes	lock bytes	
3	3h	Capability Container (CC)				Capability Container
4	4h	user memory				User memory pages
5	5h					
...	...					
128	80 h					
129	81 h	dynamic lock bytes				Dynamic lock bytes
130	82 h					
131	83 h	CFG 0				Configuration pages
132	84 h	CFG 1				
133	85 h	PWD				
134	86 h	PACK		RFUI		

aaa-008088

Fig 5. Memory organization NTAG215

8.5.7 Configuration pages

Pages 29h to 2Ch for NTAG213, pages 83h to 86h for NTAG215 and pages E3h to E6h for NTAG216 are used to configure the memory access restriction and to configure the UID ASCII mirror feature. The memory content of the configuration pages is detailed below.

Table 7. Configuration Pages

Page Address ^[1]		Byte number			
Dec	Hex	0	1	2	3
41/131/ 227	29h/83h /E3h	MIRROR	RFUI	MIRROR_PAGE	AUTH0
42/132/ 228	2Ah/84 h/E4h	ACCESS	RFUI	RFUI	RFUI
43/133/ 229	2Bh/85 h/E5h	PWD			
44/134/ 230	2Ch/86 h/E6h	PACK		RFUI	RFUI

[1] Page address for resp. NTAG213/NTAG215/NTAG216

Table 8. MIRROR configuration byte

Bit number						
7	6	5	4	3	2	1
MIRROR_CONF		MIRROR_BYTE		RFUI	STRG_MOD_EN	RFUI

Table 9. ACCESS configuration byte

Bit number						
7	6	5	4	3	2	1
PROT	CFGLCK	RFUI	NFC_CNT_EN	NFC_CNT_PWD_PROT	AUTHLIM	

Table 10. Configuration parameter descriptions

Field	Bit	Default values	Description
MIRROR_CONF	2	00b	Defines which ASCII mirror shall be used, if the ASCII mirror is enabled by a valid the MIRROR_PAGE byte 00b ... no ASCII mirror 01b ... UID ASCII mirror 10b ... NFC counter ASCII mirror 11b ... UID and NFC counter ASCII mirror
MIRROR_BYTE	2	00b	The 2 bits define the byte position within the page defined by the MIRROR_PAGE byte (beginning of ASCII mirror)
STRG_MOD_EN	1	1b	STRG MOD_EN defines the modulation mode 0b ... strong modulation mode disabled 1b ... strong modulation mode enabled

Table 10. Configuration parameter descriptions

Field	Bit	Default values	Description
MIRROR_PAGE	8	00h	MIRROR_Page defines the page for the beginning of the ASCII mirroring A value >03h enables the ASCII mirror feature
AUTH0	8	FFh	AUTH0 defines the page address from which the password verification is required. Valid address range for byte AUTH0 is from 00h to FFh. If AUTH0 is set to a page address which is higher than the last page from the user configuration, the password protection is effectively disabled.
PROT	1	0b	One bit inside the ACCESS byte defining the memory protection 0b ... write access is protected by the password verification 1b ... read and write access is protected by the password verification
CFGLCK	1	0b	Write locking bit for the user configuration 0b ... user configuration open to write access 1b ... user configuration permanently locked against write access, except PWD and PACK
NFC_CNT_EN	1	0b	NFC counter configuration 0b ... NFC counter disabled 1b ... NFC counter enabled If the NFC counter is enabled, the NFC counter will be automatically increased at the first READ or FAST_READ command after a power on reset
NFC_CNT_PWD_PROT	1	0b	NFC counter password protection 0b ... NFC counter not protected 1b ... NFC counter password protection enabled If the NFC counter password protection is enabled, the NFC tag will only respond to a READ_CNT command with the NFC counter value after a valid password verification
AUTHLIM	3	000b	Limitation of negative password verification attempts 000b ... limiting of negative password verification attempts disabled 001b-111b ... maximum number of negative password verification attempts
PWD	32	FFFFFFFFh	32-bit password used for memory access protection
PACK	16	0000h	16-bit password acknowledge used during the password verification process
RFUI	-	all 0b	Reserved for future use - implemented. Write all bits and bytes denoted as RFUI as 0b.

Remark: The CFGLCK bit activates the permanent write protection of the first two configuration pages. The write lock is only activated after a power cycle of NTAG21x. If write protection is enabled, each write attempt leads to a NAK response.

8.6 NFC counter function

NTAG21x features a NFC counter function. This function enables NTAG21x to automatically increase the 24 bit counter value, triggered by the first valid

- READ command or
- FAST-READ command

after the NTAG21x tag is powered by an RF field.

Once the NFC counter has reached the maximum value of FF FF FF hex, the NFC counter value will not change any more.

The NFC counter is enabled or disabled with the NFC_CNT_EN bit (see [Section 8.5.7](#)).

The actual NFC counter value can be read with

- READ_CNT command or
- NFC counter mirror feature

The reading of the NFC counter (by READ_CNT command or with the NFC counter mirror) can also be protected with the password authentication. The NFC counter password protection is enabled or disabled with the NFC_CNT_PWD_PROT bit (see [Section 8.5.7](#)).

8.7 ASCII mirror function

NTAG21x features a ASCII mirror function. This function enables NTAG21x to virtually mirror

- 7 byte UID (see [Section 8.7.1](#)) or
- 3 byte NFC counter value (see [Section 8.7.2](#)) or
- both, 7 byte UID and 3 byte NFC counter value with a separation byte (see [Section 8.7.3](#))

into the physical memory of the IC in ASCII code. On the READ or FAST READ command to the involved user memory pages, NTAG21x will respond with the virtual memory content of the UID and/or NFC counter value in ASCII code.

The required length of the reserved physical memory for the mirror functions is specified in [Table 11](#). If the ASCII mirror exceeds the user memory area, the data will not be mirrored.

Table 11. Required memory space for ASCII mirror

ASCII mirror	Required number of bytes in the physical memory
UID mirror	14 bytes
NFC counter	6 bytes
UID + NFC counter mirror	21 bytes (14 bytes for UID + 1 byte separation + 6 bytes NFC counter value)

The position within the user memory where the mirroring of the UID and/or NFC counter shall start is defined by the MIRROR_PAGE and MIRROR_BYTE values.

The MIRROR_PAGE value defines the page where the ASCII mirror shall start and the MIRROR_BYTE value defines the starting byte within the defined page.

The ASCII mirror function is enabled with a MIRROR_PAGE value >03h.

The MIRROR_CONF bits (see [Table 8](#) and [Table 10](#)) define if ASCII mirror shall be enabled for the UID and/or NFC counter.

If both, the UID and NFC counter, are enabled for the ASCII mirror, the UID and the NFC counter bytes are separated automatically with an "x" character (78h ASCII code).

Table 14. UID ASCII mirror - Virtual memory content

Page address		Byte number				ASCII
dec.	hex.	0	1	2	3	
0	00h	04	E1	41	2C	
1	01h	12	4C	28	80	
2	02h	F6	internal		lock bytes	
3	03h	E1	10	12	00	
4	04h	01	03	A0	0C	
5	05h	34	03	28	D1	4.(.
6	06h	01	24	55	01	.\$U.
7	07h	6E	78	70	2E	nxp.
8	08h	63	6F	6D	2F	com/
9	09h	69	6E	64	65	inde
10	0Ah	78	2E	68	74	x.ht
11	0Bh	6D	6C	3F	6D	ml?m
12	0Ch	3D	30	34	45	=04E
13	0Dh	31	34	31	31	1411
14	0Eh	32	34	43	32	24C2
15	0Fh	38	38	30	FE	880.
16	10h	00	00	00	00	
...	...					
39	27h	00	00	00	00	
40	28h		dynamic lock bytes		RFUI	
41	29h	54	RFUI	0C	AUTH0	
42	2Ah	Access				
43	2Bh			PWD		
44	2Ch		PACK		RFUI	

8.7.2 NFC counter mirror function

This function enables NTAG21x to virtually mirror the 3 byte NFC counter value in ASCII code into the physical memory of the IC. The length of the NFC counter mirror requires 6 bytes to mirror the NFC counter value in ASCII code. On the READ or FAST READ command to the involved user memory pages, NTAG21x will respond with the virtual memory content of the NFC counter in ASCII code.

The position within the user memory where the mirroring of the NFC counter shall start is defined by the MIRROR_PAGE and MIRROR_BYTE values.

The MIRROR_PAGE value defines the page where the NFC counter mirror shall start and the MIRROR_BYTE value defines the starting byte within the defined page.

The NFC counter mirror function is enabled with a MIRROR_PAGE value >03h and the MIRROR_CONF bits are set to 10b.

If the NFC counter is password protected with the NFC_CNT_PWD_PROT bit set to 1b (see [Section 8.5.7](#)), the NFC counter will only be mirrored into the physical memory, if a valid password authentication has been executed before.

8.8 Password verification protection

The memory write or read/write access to a configurable part of the memory can be constrained by a positive password verification. The 32-bit secret password (PWD) and the 16-bit password acknowledge (PACK) response are typically programmed into the configuration pages at the tag personalization stage.

The AUTHLIM parameter specified in [Section 8.5.7](#) can be used to limit the negative verification attempts.

In the initial state of NTAG21x, password protection is disabled by a AUTH0 value of FFh. PWD and PACK are freely writable in this state. Access to the configuration pages and any part of the user memory can be restricted by setting AUTH0 to a page address within the available memory space. This page address is the first one protected.

Remark: The password protection method provided in NTAG21x has to be intended as an easy and convenient way to prevent unauthorized memory accesses. If a higher level of protection is required, cryptographic methods can be implemented at application layer to increase overall system security.

8.8.1 Programming of PWD and PACK

The 32-bit PWD and the 16-bit PACK need to be programmed into the configuration pages, see [Section 8.5.7](#). The password as well as the password acknowledge are written LSByte first. This byte order is the same as the byte order used during the PWD_AUTH command and its response.

The PWD and PACK bytes can never be read out of the memory. Instead of transmitting the real value on any valid READ or FAST_READ command, only 00h bytes are replied.

If the password verification does not protect the configuration pages, PWD and PACK can be written with normal WRITE and COMPATIBILITY_WRITE commands.

If the configuration pages are protected by the password configuration, PWD and PACK can be written after a successful PWD_AUTH command.

The PWD and PACK are writable even if the CFGLOCK bit is set to 1b. Therefore it is strongly recommended to set AUTH0 to the page where the PWD is located after the password has been written. This page is 2Bh for NTAG213, page 85h for NTAG215 and page E5h for NTAG216.

Remark: To improve the overall system security, it is advisable to diversify the password and the password acknowledge using a die individual parameter of the IC, that is the 7-byte UID available on NTAG21x.

10.3 FAST_READ

The FAST_READ command requires a start page address and an end page address and returns the all n*4 bytes of the addressed pages. For example if the start address is 03h and the end address is 07h then pages 03h, 04h, 05h, 06h and 07h are returned. If the addressed page is outside of accessible area, NTAG21x replies a NAK. For details on those cases and the command structure, refer to [Figure 16](#) and [Table 30](#).

[Table 31](#) shows the required timing.

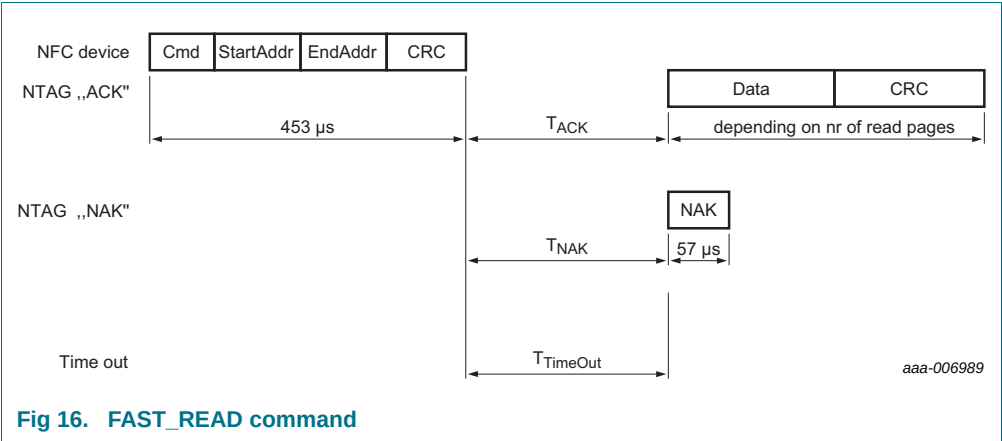


Fig 16. FAST_READ command

Table 30. FAST_READ command

Name	Code	Description	Length
Cmd	3Ah	read multiple pages	1 byte
StartAddr	-	start page address	1 byte
EndAddr	-	end page address	1 byte
CRC	-	CRC according to Ref. 1	2 bytes
Data	-	data content of the addressed pages	n*4 bytes
NAK	see Table 22	see Section 9.3	4-bit

Table 31. FAST_READ timing

These times exclude the end of communication of the NFC device.

	T _{ACK/NAK} min	T _{ACK/NAK} max	T _{TimeOut}
FAST_READ	n=9 ^[1]	T _{TimeOut}	5 ms

[1] Refer to [Section 9.2 "Timings"](#).

In the initial state of NTAG21x, all memory pages are allowed as StartAddr parameter to the FAST_READ command.

- page address 00h to 2Ch for NTAG213
- page address 00h to 86h for NTAG215
- page address 00h to E6h for NTAG216

Addressing a memory page beyond the limits above results in a NAK response from NTAG21x.

10.4 WRITE

The WRITE command requires a block address, and writes 4 bytes of data into the addressed NTAG21x page. The WRITE command is shown in [Figure 17](#) and [Table 32](#).

[Table 33](#) shows the required timing.

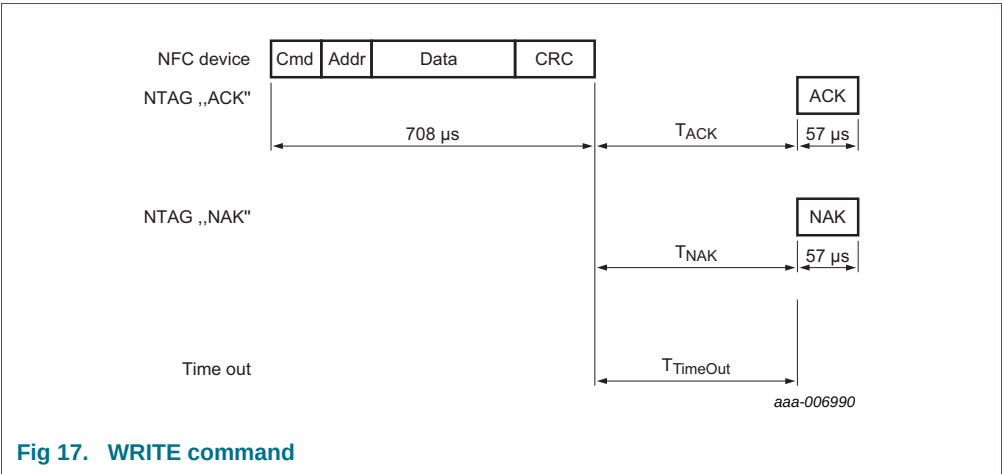


Fig 17. WRITE command

Table 32. WRITE command

Name	Code	Description	Length
Cmd	A2h	write one page	1 byte
Addr	-	page address	1 byte
CRC	-	CRC according to Ref. 1	2 bytes
Data	-	data	4 bytes
NAK	see Table 22	see Section 9.3	4-bit

Table 33. WRITE timing

These times exclude the end of communication of the NFC device.

	T _{ACK/NAK} min	T _{ACK/NAK} max	T _{TimeOut}
WRITE	n=9 ^[1]	T _{TimeOut}	10 ms

[1] Refer to [Section 9.2 "Timings"](#).

In the initial state of NTAG21x, the following memory pages are valid Addr parameters to the WRITE command.

- page address 02h to 2Ch for NTAG213
- page address 02h to 86h for NTAG215
- page address 02h to E6h for NTAG216

Addressing a memory page beyond the limits above results in a NAK response from NTAG21x.

Pages which are locked against writing cannot be reprogrammed using any write command. The locking mechanisms include static and dynamic lock bits as well as the locking of the configuration pages.

10.7 PWD_AUTH

A protected memory area can be accessed only after a successful password verification using the PWD_AUTH command. The AUTH0 configuration byte defines the protected area. It specifies the first page that the password mechanism protects. The level of protection can be configured using the PROT bit either for write protection or read/write protection. The PWD_AUTH command takes the password as parameter and, if successful, returns the password authentication acknowledge, PACK. By setting the AUTHLIM configuration bits to a value larger than 000b, the number of unsuccessful password verifications can be limited. Each unsuccessful authentication is then counted in a counter featuring anti-tearing support. After reaching the limit of unsuccessful attempts, the memory access specified in PROT, is no longer possible. The PWD_AUTH command is shown in [Figure 21](#) and [Table 38](#).

[Table 39](#) shows the required timing.

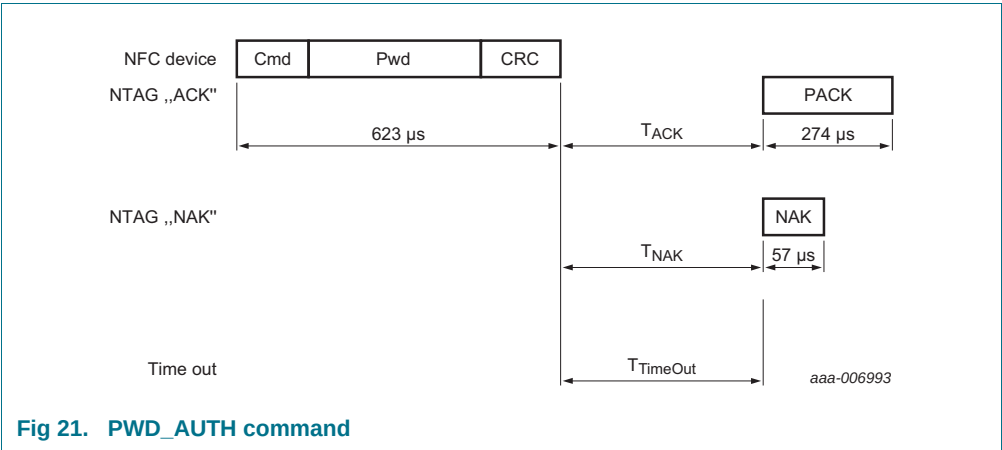


Table 38. PWD_AUTH command

Name	Code	Description	Length
Cmd	1Bh	password authentication	1 byte
Pwd	-	password	4 bytes
CRC	-	CRC according to Ref. 1	2 bytes
PACK	-	password authentication acknowledge	2 bytes
NAK	see Table 22	see Section 9.3	4-bit

Table 39. PWD_AUTH timing

These times exclude the end of communication of the NFC device.

	T _{ACK/NAK} min	T _{ACK/NAK} max	T _{TimeOut}
PWD_AUTH	n=9 ^[1]	T _{TimeOut}	5 ms

[1] Refer to [Section 9.2 "Timings"](#).

Remark: It is strongly recommended to change the password from its delivery state at tag issuing and set the AUTH0 value to the PWD page.

